Measuring and Repairing Quantization Damage in Small LLM Agents on Commodity Hardware

Project Workbook

By

Ekant Kapgate
Vritika Malhotra
Yashashav Devalapalli Kamalraj

September 2026

Advisor: Dr. Bertin Cordova-Diba

Team Member Contributions

All three team members contributed to this workbook. The table lists each member's primary project ownership (detailed in Chapter 9) and the workbook sections each member led. Items highlighted in yellow are proposed and must be confirmed by the team before submission.

Table 1. Individual contributions to the project and to this workbook.

Member	Primary project ownership	Workbook sections led
Ekant Kapgate	Episode Runner (C2), Inference Backends (C3), Step Logger (C5), grammar-constrained repair (R1), Cost Profiler (C9)	Ch. 1 (constrained decoding, State-of-the-Art Summary), Ch. 5, Ch. 8
Vritika Malhotra	Benchmark Adapter (C1), Simulated-User Cache (C4), Failure Classifier (C6), analysis plan and statistics (C10)	Ch. 1 (agent benchmarks, evaluation methodology), Ch. 3, Ch. 7
Yashashav Devalapalli Kamalraj	Added-bits repair (R2), QAD Training Pipeline and control (R3, C8), model conversion, Experiment Config & Registry (C11)	Ch. 1 (quantization, mixed precision, QAT/QAD), Ch. 2, Ch. 4, Ch. 6, Ch. 9

Chapter 1. Literature Search, State of the Art

Literature Search

This review covers weight quantization of large language models (LLMs) and the methods proposed to repair the damage it causes, with emphasis on what applies to 0.5 to 4B-parameter models that run as tool-calling agents on llama.cpp and mlx-lm. We searched the ACM Digital Library, IEEE Xplore, the ACL Anthology, arXiv, and the proceedings of NeurIPS, ICML, ICLR, and MLSys, and we added the technical reports, model releases, and runtime documentation through which industry ships quantized models [1, 2]. The review is organized around the three repair families the project compares: after the formats small models ship in and the evidence that aggregate scores hide quantization damage, this part covers adding bits and retraining, and grammar-constrained decoding [3] is reviewed with the tool-calling literature later in this chapter.

Post-Training Weight Quantization and On-Device Formats

Post-training quantization (PTQ) methods for LLMs follow two broad strategies. The first protects the few weights that dominate the error: LLM.int8() keeps a small set of emergent outlier feature dimensions in 16-bit while more than 99.9% of values are multiplied in 8-bit [4], AWQ uses activation statistics to find and rescale the roughly 1% of salient weight channels before rounding [5], and SpQR stores outlier weights at higher precision in a sparse side structure [6]. The second reduces the error of rounding everything: GPTQ quantizes one column at a time and uses approximate second-order information to compensate the remaining weights [7], and QuIP# makes weights incoherent with a randomized Hadamard transform before rounding them to lattice codebooks [8]. These methods were developed and mostly evaluated on models of 7B parameters and larger, using perplexity and zero-shot accuracy, and the SpQR authors note that 3 to 4-bit quantization usually causes moderate-to-high accuracy loss, especially in the 1 to 10B range suited to edge devices [6].

The formats small models actually ship in are simpler. llama.cpp stores models as GGUF files [1]. Its legacy `Q4_0` type rounds each block of 32 weights to 4 bits with one shared scale; its k-quant types group blocks into 256-weight super-blocks with quantized scales, giving 4.5 bits per weight for `Q4_K`; the widely used `Q4_K_M` recipe, as originally defined, also stores half of the attention value and feed-forward down-projection tensors in 6-bit `Q6_K`; and an importance matrix computed from activations on calibration text can weight the rounding error [1]. MLX quantizes weights affinely with a scale and bias per group, defaulting to 4 bits and groups of 64, and also supports the MXFP4 and NVFP4 microscaling modes [9]. Those two floating-point formats come from data-center hardware: both store 4-bit E2M1 values, but MXFP4 shares a power-of-two E8M0 scale across 32 values, whereas NVFP4 uses an FP8 E4M3 scale per 16 values plus a per-

tensor FP32 scale [10]. By our arithmetic, `Q4_0` and default 4-bit MLX with 16-bit scales and biases both cost about 4.5 bits per weight, so "a 4-bit model" in on-device practice usually means block-wise round-to-nearest quantization at about 4.5 bits, optionally guided by calibration statistics [1, 9].

Model vendors now publish their own 4-bit builds instead of leaving quantization to users. Google released Gemma 3 checkpoints trained for the `Q4_0` format used by llama.cpp, Ollama, and MLX [11]; Liquid AI released distillation-trained `Q4_0` files as updates to its PTQ builds of LFM2.5 models from 230M to 2.6B parameters [2]; NVIDIA publishes NVFP4 checkpoints recovered by distillation [12]; and Apple compresses its roughly 3B-parameter on-device model to 2 bits per weight [13]. Because the same "4-bit" label covers different block sizes, scale types, and mixed-precision recipes, our Inference Backends (C3) treat the format itself (`Q4_0`, `Q4_K_M`, MLX affine) as an experimental factor.

Measuring Quantization Damage Beyond Aggregate Scores

The common view that 4-bit weight quantization is nearly free rests mainly on aggregate benchmark scores. The largest such study, with over 500,000 evaluations of the Llama-3.1 family, found 4-bit weight-only quantization competitive with 8-bit formats [14], and a study of 11 model families from 125M to 180B parameters found 4-bit weight-only quantization to cost most models less than 2% [15]. Several peer-reviewed studies show why such averages mislead. Dutta et al. count *flips*, answers that change from correct to incorrect or the reverse: with accuracy within 0 to 2% of the 16-bit model, most quantization schemes flipped up to 13.6% of answers, and the authors propose flips and KL divergence as distance metrics to report alongside accuracy [16]. Jaiswal et al. show that perplexity misses the losses compressed models suffer on knowledge-intensive tasks [17], and Li et al. find that multiple-choice accuracy can rise under quantization through borderline cases and that multi-step reasoning tolerates quantization less than instruction following [15]. Distance metrics also reveal what training-based repairs do: NVIDIA reports a quantization-aware-trained model whose cross-entropy matches its BF16 teacher while its KL divergence from that teacher is 0.311, against 0.004 after distillation [12].

Damage grows as models shrink. Tolerance to weight-only quantization increases with model size [15], larger models flip fewer answers although a 70B model still flips 3 to 8% at 4 bits [16], and NVIDIA observes that models with hundreds of billions of parameters survive NVFP4 PTQ without training [12]. At the small end, under llama.cpp's `Q4_K_M`, Qwen3-0.6B without its thinking mode falls from 62.3% to 16.8% on BFCL's non-live function-calling split in the authors' instruction-following setup, while Qwen3-1.7B in the same mode loses only 2.3 points [18].

Evidence on agents is recent. ACBench, covering 15 models, found that 4-bit GPTQ and AWQ cost only 1 to 3% on workflow generation and tool use but 10 to 15%

on real-world application tasks [19]. On τ²-bench, for 9 to 35B models served with AWQ on vLLM, 4-bit quantization leaves the task score flat while amplifying the failure a model already makes by up to 2.5×, because the benchmark's error budget absorbs the extra failures [20]; whether the same masking occurs at 0.5 to 4B on llama.cpp and MLX has not been measured. Li et al. find that the per-step KL divergence between a 4-bit and a BF16 agent is low on most steps, with a small high-divergence mode of rare but consequential steps [21]. Inside the network, Hu and Ramachandran find damage to be diffuse: measured by restoring layers to 8 bits, 8 of 9 models need roughly half their layers restored to recover 75% of the accuracy gap, and cheap signals that correlate with damage do not identify where restored precision helps [22]. Together these results argue for measuring damage per step and per failure type against a full-precision reference, which is the job of our Step Logger (C5) and Failure Classifier (C6); benchmark-level evaluation methodology is reviewed with the agent benchmarks later in this chapter.

Adding Bits: Mixed-Precision and Adaptive Bit Allocation

The oldest repair gives the sensitive parts of a model more bits. HAWQ assigns per-layer precision from each layer's Hessian spectrum [23]; SqueezeLLM uses second-order sensitivity to place non-uniform quantization levels and keeps outliers and sensitive weights in a sparse full-precision matrix [24]; LLM.int8() and SpQR likewise hold outliers at higher precision [4, 6]. llama.cpp's `Q4_K_M` recipe and importance-matrix quantization are the deployed, heuristic form of the same idea [1]. All of these choose *where in the network* to spend bits, and all were validated with perplexity or aggregate accuracy.

Hu and Ramachandran test that premise causally on nine open models from 0.5B to 8B parameters, including Qwen3-0.6B and Qwen3-1.7B [22]. Starting from per-row 4-bit round-to-nearest quantization, they spend an extra 0.146 bits per weight either globally, by moving every layer to group-128 scales, or locally, by restoring the most recoverable whole layers to 8 bits. Global granularity recovered 21 to 52 percentage points more of the accuracy lost to 4-bit quantization than the best local repair, even with the local layers chosen by an oracle, on all eight models where it applied, including Qwen3-8B, the one model whose damage was concentrated [22]. Two caveats limit the transfer to our setting. Damage was measured by continuation scoring on 22 knowledge and reasoning tasks, with no generation or agent tasks [22]. On-device formats already use groups of 32 to 64 weights, so the next global step on llama.cpp or MLX is a higher-bit format everywhere, such as `Q5_K` or `Q6_K`, rather than finer scales [1, 9].

A second line of work varies precision over time instead of over layers. Any-Precision LLM overlays models of 3 to n bits in the memory of one n-bit model so that precision can be chosen per request [25]. Progressive Mixed-Precision Decoding runs the prefill at higher precision and lowers precision deeper into the generated sequence [26]. MoBiQuant adds a router that picks each token's precision from nested residual quantization and reports up to 1.34× the throughput of earlier any-precision methods [27].

Closest to agents, Dynamic Mixed-Precision Routing trains a router that sends each step of a multi-step episode to a BF16 or a 4-bit copy of Qwen3: with a budget of 40% of steps at full precision, Qwen3-4B with AWQ reached 95.1% success on ALFWorld (BF16 94.3%, all 4-bit 85.4%) and 61.7% on WebShop (BF16 65.2%, all 4-bit 56.4%) [21].

Step-selective precision is therefore possible, but the question our R2 arm asks remains untested: at a matched memory budget, does spending extra bits on selected conversation steps recover more agent failures than spreading them uniformly or placing them in selected layers? No study compares the three allocations on the same models. The step-routing study switches between two separate models, so both must be available at every step; it models cost as latency on a data-center GPU, reports no memory figures, and uses household and shopping environments rather than tool-calling benchmarks [21]. The nested formats that would avoid a second copy rely on their own kernels and are not among the documented GGUF or MLX quantization types [9, 25, 27]. On a laptop, the memory for a second copy is exactly what quantization was meant to save, so our Cost Profiler (C9) must charge it to the repair.

Retraining: Quantization-Aware Training and Distillation

Retraining lets the weights adapt to the quantization grid. LLM-QAT brought quantization-aware training (QAT) to LLMs with a data-free recipe that distills from the full-precision model's own generations, reaching 4-bit weights, activations, and KV cache on 7B to 30B models [28]. BitDistiller combines QAT with a confidence-aware KL self-distillation objective for 2 and 3-bit models [29]; EfficientQAT trains block by block and then only the quantization step sizes, producing a 2-bit Llama-2-70B on one A100 in 41 hours at 69.48 versus 72.41 average accuracy [30]; and ParetoQ finds a transition between 2 and 3 bits, above which fine-tuned models stay close to their pre-trained distributions [31]. Parameter-efficient variants freeze the quantized base and train low-rank adapters, as in QLoRA's 4-bit NormalFloat base [32] and Apple's quality-recovery adapters on its 2-bit on-device model [13]. Most of this work targets 2 to 3 bits on models of 7B parameters and more, where PTQ breaks down; repairing 4-bit agents below 4B parameters is much less studied.

Quantization-aware distillation (QAD) has become the industrial form of this idea. NVIDIA trains NVFP4 students on the KL divergence to their BF16 teacher and shows that plain QAT damages models post-trained with reinforcement learning: on AIME25, AceReason-Nemotron-1.1-7B scored 63.5 in BF16, 58.7 after PTQ, 46.1 after QAT, and 62.0 after QAD, with 0.3 to 6 billion training tokens depending on the model [12]. Google trained Gemma 3 for about 5,000 steps with the unquantized checkpoint's probabilities as targets and reports a 54% smaller perplexity drop at `Q4_0` [11]. Liquid AI's QAD closes 48.4% to 73.4% of the BF16-to-`Q4_0` gap for four LFM2.5 models, which retain 96.5% to 97.4% of BF16 performance [2]. Multiverse Computing's quantization-aware healing distills a structurally compressed MXFP4 student directly from the uncompressed original; its released 60B model matches or beats its BF16 source

on 7 of 9 benchmarks, and the method reaches a comparable peak about 7× faster than cross-entropy QAT [33].

Three gaps matter for agents. First, every evaluation above reports aggregate scores. Liquid AI reports a seven-task average in which BFCLv4 is one component [2], NVIDIA and Google report reasoning benchmarks and perplexity [11, 12], and the single agent result, 61.7 versus 59.4 on τ²-bench [33], uses the pass rate that can stay flat while failures grow [20]. NVIDIA also picks, among the ten checkpoints with the lowest validation loss, the one with the best average on the evaluation benchmarks themselves [12]. Second, none of these evaluations includes the control that separates retraining under quantization from simply training more. NVIDIA describes QAT as "effectively acting as an additional post-training stage" [12], and Multiverse attributes its 4-bit model's gains over BF16 to a second distillation pass that the BF16 checkpoint never received [33]. The control is not a formality: in pretraining, the loss-optimal split of compute between full-precision training and QAT shifts with the total budget [34], so a BF16 model distilled for the same steps and then quantized is a genuine competitor. Third, QAD keeps the format and therefore the inference cost, but its training cost for small models is not reported in usable terms: Liquid AI gives no training details and Google gives only a step count [2, 11]. Our QAD Training Pipeline (C8) therefore runs each QAD configuration next to a train-longer control at full precision. The table below summarizes the three families.

Table 2. The three repair families for quantized small-model agents, as evaluated in the literature.

Repair family	Representative methods	What is changed	Inference cost	How it has been evaluated	Gap for agents
R1 Restrict output	Grammar-constrained decoding [35, 36]; input-side schema compilation [3]	Decoder: invalid tokens masked	No extra weights; per-token masking overhead	Tool-call accuracy	Cannot fix wrong-but-valid calls; never compared with R2 or R3
R2 Add bits	`Q4_K_M`, SpQR, SqueezeLLM; any-precision, MoBiQuant, step routing [21, 22]	Precision of some weights, per layer, token, or step	More memory; step routing may keep a second model copy	Perplexity, continuation scoring, ALFWorld and WebShop success	Step vs. layer vs. uniform allocation untested at matched budget
R3 Retrain	QAT, QAD (NVIDIA, Google, Liquid AI, Multiverse) [2, 12]	Weight values within the same format	None at inference; training compute	Aggregate scores; τ²-bench pass rate	No train-longer control; failures by type unmeasured

Small Language Models as Tool-Calling Agents

Tool use turned language models from text generators into agents. Toolformer showed that a model can teach itself when to call an API and with which arguments [37]. Gorilla fine-tuned a model to write API calls and reduced hallucinated API usage [38],

and ToolLLM scaled training and evaluation to more than 16,000 real APIs [39]. ReAct established the loop these agents still run: reason, act, observe, repeat [40]. Each step is a model output that must parse into a valid call and then changes the environment, so one invented or malformed call alters every later step.

Running this loop on the user's device is now a design goal. TinyAgent fine-tuned 1.1B and 7B models for function calling, added tool retrieval and quantization, and ran the agent locally on a MacBook [41]. Paramanayakam et al. showed that shrinking the tool list offered to an edge model improves success, latency, and power [42]. The three families we study are released for this setting. Qwen3 ships dense models from 0.6B parameters upward, with switchable thinking and non-thinking modes [43]. LFM2's hybrid convolution-attention backbone was found by hardware-in-the-loop search for edge devices [44], and LFM2.5-1.2B is presented as a building block for on-device agents, with GGUF and MLX checkpoints at release [45]. SmolLM3 is a 3B model with tool calling and a dual reasoning mode [46]. At 16 bits they strain phone and laptop memory: Taheri et al. report 8.0 GB for Qwen3-4B at FP16 against 2.5 GB at llama.cpp's Q4_K_M format [18], so in practice they ship at about 4 bits.

The evidence on what 4 bits do to tool calling, reviewed earlier in this chapter, points in different directions: ACBench finds tool use almost unaffected by 4-bit GPTQ and AWQ [19], while Qwen3-0.6B in non-thinking mode loses most of its BFCL V3 non-live accuracy (62.3% to 16.8%) under Q4_K_M [18]. An edge study adds that the format matters: on BFCL, Llama-3.1-8B served through Ollama fell from 63.0% at full precision to 20.4% at q4_0 and 39.6% at q4_K_M [42]. The studies differ in model size, format, reasoning mode, and prompting, and all report single-turn or aggregate scores, so none shows where in a multi-turn conversation the damage lands.

Benchmarks and Failure Analysis for Tool-Calling Agents

Tool-use benchmarks have moved from single calls to stateful conversations. BFCL scores individual calls by abstract-syntax-tree matching and adds stateful multi-step categories [47]. Its V3 multi-turn set (base, missing parameters, missing functions, long context) checks the environment state and the response after every turn, returns execution errors to the model, and cuts off a turn after 20 steps [48]. τ-bench added an LLM-simulated user and a domain policy, graded the final database state, and introduced pass^k, the probability that all k independent trials succeed. With it, GPT-4o solved fewer than 50% of tasks and had pass^8 below 25% in retail [49]. τ²-bench adds a dual-control telecom domain in which the simulated user also acts on the shared state [50]. Amazon AGI's audit of that benchmark released τ²-Bench Verified, which corrects tasks whose expected actions contradicted the policy or the database [51]. Sierra has since folded external fixes, chiefly this audit, into its official τ³-bench release [52], so one benchmark name now covers several task versions and pinning the version is part of the measurement. The table below compares the benchmarks we considered.

Table 3. Agent benchmarks compared on the properties that matter for measuring quantization damage.

Benchmark	Multi-turn	Simulated user	Main metric	Error tolerance	Used by us
BFCL single-turn [47]	No	No	AST or execution match	None; one call scored	Candidate easier set
BFCL V3 multi-turn [48]	Yes	No (scripted turns)	State and response checks per turn	Errors returned; 20-step cap per turn	Yes
τ-bench [49]	Yes	Yes (LLM)	Final database state; pass^k	Errors returned; step cap	No
τ²-bench [50]	Yes	Yes (LLM, can act)	Final state and assertions; pass^k	Ends after 10 failed calls	Through Verified
τ²-Bench Verified [51]	Yes	Yes (LLM, can act)	As τ²-bench, corrected tasks	As τ²-bench	Yes
API-Bank [53]	Yes (annotated dialogues)	No	API-call accuracy; response ROUGE-L	None; each call scored	No
ToolBench [39, 54]	Multi-step, one query	No	LLM-judged pass and win rate	Not applicable	No
AgentBench [55]	Yes (environment turns)	No	Per-environment score	Varies by environment	No

The error-tolerance column is where quantization damage hides: τ²-bench ends an episode only after ten failed tool calls, and the agent can recover between failures [20]. Jang et al. ran two open-weight families at 16, 8, and 4 bits on two τ²-bench domains (456 episodes per cell). No change in final score survived multiple-comparison correction [20]. Underneath, 4-bit weights amplified failures the models already made at full precision, and only in models that made them. In the most affected cell (Gemma-4-31B, telecom), the share of tool calls naming a nonexistent tool rose from 19.5% to 38.3% (about 2.0×). Each tool name the 16-bit model had already invented was called about 2.5× more often, and only 0.18% of the 4-bit model's hallucinated calls named a tool the 16-bit model had never used [20]. Shrinking the budget from ten errors to two re-exposed a 17-point score gap, but only in the cell that had gained errors, and in that cell the 4-bit to 16-bit hallucination-rate ratio stayed between 1.3× and 2.0× at every turn depth [20]. This matches a general result for compressed models: matching accuracy can hide many answers that flip between correct and incorrect [16]. The study's scope leaves room for ours: 9B to 35B (total) parameter models, AWQ weights served by vLLM on A100 GPUs, one benchmark, and one prompt-based repair [20]. Small models on llama.cpp and mlx-lm have not been measured this way.

Failure taxonomies show why one score is not enough. MAST annotated more than 1,600 multi-agent traces into 14 failure modes with high inter-annotator agreement

($\kappa = 0.88$) and then scaled the annotation with an LLM judge [56]. Xu et al. split tool hallucination into tool-selection and tool-usage errors [57]. SABER found that deviations at state-changing ("mutating") steps decide task success, while deviations at read-only steps barely matter [51]. Failure types differ in consequence, and a pass/fail score merges them. Our Failure Classifier therefore labels each failed step (hallucinated tool, invalid arguments, malformed call, wrong tool, premature termination, policy violation) and records whether the step was mutating.

Restricting Output: Grammar-Constrained Decoding

Grammar-constrained decoding masks, at each step, every token that would violate a formal grammar, so the output is valid by construction. Willard and Louf compiled regular expressions and grammars into a finite-state index over the vocabulary, the basis of Outlines [58]. Geng et al. showed that input-dependent grammars let a model solve structured tasks without fine-tuning [35]. XGrammar prechecks tokens whose validity does not depend on context and interprets only the rest at runtime. It reports up to 100× faster mask generation and near-zero end-to-end overhead when co-designed with the serving engine [36]. llama.cpp exposes the same mechanism through GBNF grammars and converts JSON schemas to grammars in its server [1].

The repair is cheap in the best engines, but that does not make it free everywhere. In XGrammar's evaluation, llama.cpp's grammar engine was among the baselines slowed by longer preprocessing and per-token processing [36]. JSONSchemaBench tested six frameworks, including llama.cpp and XGrammar, on 10,000 real-world schemas and found that they differ in efficiency, in which schema features they support, and in output quality [59]. Constraints also change what the model says. Park et al. showed that greedy masking distorts the model's distribution, producing outputs that are grammatical but improbable under the model [60]. Tam et al. found that strict format restrictions degrade reasoning, and that stricter formats degrade it more [61].

What a grammar fixes is narrow and exact. A grammar that lists the available tool names makes a call to a nonexistent tool impossible, and so it removes by construction the failure that Jang et al. found amplified [20]. It also guarantees well-typed arguments; Apple's on-device framework uses exactly this to prevent hallucinated tool names and arguments [13]. It cannot choose the right tool among valid ones, supply the right record identifier, decide when to stop, or follow a policy. It may also turn a loud failure, a "tool not found" error that the agent can recover from, into a silent one: a valid but wrong call, the kind SABER identifies as decisive at mutating steps [51]. Whether grammars remove damage or only relabel it is a question for RQ3. TSCG works on the input side instead. It deterministically compiles JSON tool schemas into compact structured text, attributes most small-model tool-use failures at production catalog sizes to the JSON format itself, and lifts Phi-4 14B from 0% to 84.4% accuracy at 20 tools while saving 52 to 57% of schema tokens [3]. Neither TSCG nor any grammar engine above was evaluated on quantized agents.

Evaluation Methodology: Variance, Statistical Testing, and Pre-Registration

Agent benchmarks are small and noisy. A τ^2-bench domain has 114 tasks with binary outcomes, and repeated trials of a task are correlated [20]. Miller treats evaluations as experiments: report standard errors, cluster them when items are grouped (clustered errors can exceed three times the naive ones), compare models on per-question paired differences, and size the study by power analysis [62]. Dror et al. give a protocol for choosing significance tests in NLP: McNemar's test for paired binary outcomes, and paired bootstrap or permutation tests for other metrics [63]. Jang et al. apply this toolkit to quantized agents: a cluster bootstrap over tasks, paired-by-task comparisons, and two one-sided equivalence tests [64] at ±7.5 points to support a "no change" claim [20].

Two further problems apply to our design. First, crossing models, formats, repairs, benchmarks, and failure types produces hundreds of tests. We correct the confirmatory tests with Holm's procedure [65]; false-discovery-rate control [66] is the more powerful option for screening exploratory comparisons. Second, inference is not reliably deterministic. Greedy decoding can change between runs because kernels are not batch-invariant [67], and a repeated 456-episode τ^2-bench run on vLLM shifted the mean score by about 1.5 points [20]. We therefore repeat trials, record seeds, runtime versions, and hardware, and measure a noise floor for each backend instead of assuming determinism. External services add a third source of drift; StableToolBench replaced live APIs with a cache and simulators to keep scores stable [54], and our Simulated-User Cache applies the same idea to the simulated user.

Pre-registration addresses the remaining risk: choosing the analysis after seeing the data. It is established practice in the empirical sciences [68], and the NeurIPS 2020 pre-registration workshop reviewed and accepted ML protocols before any results existed [69]. The NeurIPS reproducibility program introduced a checklist covering code, data, and statistical reporting [70]. Our testbed follows both. The Step Logger stores every step, so per-failure-type rates, success under a shrinking error budget, and pass^k can all be recomputed without rerunning a model. The analysis plan (hypotheses, primary metrics, tests, correction, and the rule excluding items that the full-precision model already fails) is registered on OSF before the main runs. With three repair families and many candidate metrics, this keeps us from picking a winner by picking the metric.

Research Gap

Three literatures offer three repairs, each judged by a different yardstick. Output restriction is measured on syntactic validity and throughput [36, 58, 59], and its input-side relative TSCG on unquantized models [3]. Added-bits work has just shown, on continuation scoring rather than agent tasks, that choosing which layers receive extra precision works worse than spreading the bits evenly [22]. Choosing which steps get more precision has first evidence: a router that sends each ALFWorld or WebShop step to a BF16 or a 4-bit Qwen3 model [21], alongside token-adaptive any-precision kernels [27]. What remains untested is step-selective against uniform and layer-selective bits at a

matched memory budget, on tool-calling benchmarks, on llama.cpp and MLX, with deployment cost counted; the router keeps two full models and reports only modelled latency [21]. Quantization-aware distillation retrains the quantized model to imitate its full-precision self and now ships from NVIDIA, Multiverse Computing, and Liquid AI [2, 12, 33]. Its published evaluations report aggregate scores, and none that we reviewed includes a control that trains for the same number of steps without quantization, which is needed to separate quantization-aware retraining from simply training longer, a confound Multiverse's own write-up acknowledges [33].

The only process-level study of quantized agents shows that aggregate scores cannot see the damage [20], yet it covers 9B to 35B models on server GPUs and tests one prompt repair. The single-turn and aggregate studies disagree on how large the damage is [18, 19, 42]. No published work compares the three repairs on the same small models and multi-turn benchmarks, tests whether a repair removes failures or only restores the score, or reports deployment costs on the runtimes these models ship on. This project addresses four research questions:

1. **RQ1:** Where does 4-bit damage concentrate in small-agent conversations, by step type and by conversation depth?
2. **RQ2:** How do grammar-constrained decoding, added bits (uniform, layer-selective, and step-selective), and QAD retraining compare, each with its control, on the same models and benchmarks?
3. **RQ3:** Do repairs validated on overall scores fix the underlying failures, or only the score?
4. **RQ4:** What does each repair cost to deploy (memory, latency, extra tokens, and energy) on the real 4-bit code paths of an Apple M4 Pro?

State-of-the-Art Summary

On-device runtimes and formats. Small agents reach users through a few open runtimes. llama.cpp's GGUF format offers k-quants such as Q4_K_M (4.89 bits per weight on Llama-3.1-8B), i-quants from IQ1 to IQ4, and an importance matrix (imatrix) computed from calibration text to guide quantization, and its server accepts GBNF grammars or JSON schemas to constrain output [1]. On Apple silicon, MLX [9] and mlx-lm quantize and serve models, and mlx-lm's learned quantizers include DWQ, which tunes quantization scales and biases with the unquantized model as teacher, so a distillation-style repair already sits inside the runtime [71]. Meta ships quantized Llama 3.2 through ExecuTorch on Arm phone CPUs [72]. Desktop apps such as Ollama and LM Studio distribute these builds, and vendors now name them as launch targets [73, 74]. Apple's own on-device model shows how far this goes: about 3B parameters at 2 bits per weight, served through a framework whose guided generation prevents hallucinated tool names and arguments [13].

Vendor QAT and QAD. Retraining under quantization has moved from research papers into product releases, as the table below shows.

Table 4. Vendor quantization-aware training (QAT) and distillation (QAD) releases, 2024 to 2026.

Vendor	Release	Technique	Deployment target
Meta	Llama 3.2 1B/3B, Oct 2024	QAT with LoRA; SpinQuant	4-bit weights, 8-bit activations; ExecuTorch [72]
Google	Gemma 3 QAT, Apr 2025	QAT toward unquantized model's outputs	Q4_0 for llama.cpp, Ollama, MLX [11]
Apple	On-device model, 2025	2-bit QAT plus LoRA recovery adapters	Apple silicon [13]
NVIDIA	NVFP4 QAD, 2026	KL distillation from BF16 teacher	NVFP4 GPUs [12]
Google	Gemma 4 QAT, Jun 2026	QAT; Q4_0 plus a mobile format	E2B under 1 GB [73]
Multiverse	Quantization-aware healing, 2026	Distillation into a compressed 4-bit model	MXFP4 [33]
Liquid AI	LFM2.5 QAD, Aug 2026	QAD for 230M to 2.6B models	GGUF Q4_0 [2]

Structured output in production. Constraining output is now a standard serving feature. OpenAI's Structured Outputs guarantees schema adherence, including for function calls in strict mode [75]. XGrammar is integrated into SGLang and MLC-LLM [36] and is a structured-output backend in vLLM [76]. llama.cpp's GBNF [1] and Outlines [58] bring the same guarantee to open models. These engines are judged on validity and speed; whether a grammar helps a quantized agent pick the right tool is outside what they measure.

Agent evaluation and on-device agents. The BFCL leaderboard has grown from single calls to stateful multi-turn and agentic categories [47, 48]. Sierra's τ-bench leaderboard added a knowledge-retrieval banking domain, where the best model reached 25.5%, and a voice track in March 2026 [52]. Both rank models by task success, the metric that Jang et al. showed can stay flat while the rate of invented tool calls doubles [20]. Google's FunctionGemma (270M parameters) targets on-device function calling, and fine-tuning raised its Mobile Actions accuracy from 58% to 85% [74]. Liquid AI positions LFM2.5-1.2B as a building block for on-device agents [45].

What industry still lacks. Vendors validate quantized releases on aggregate scores; Apple combines 2-bit QAT, recovery adapters, and constrained tool calling without separating their contributions [13], and Liquid AI reports only a seven-task

average [2]. We found no vendor that reports per-failure-type agent results for its quantized builds, compares output restriction, added bits, and retraining against each other, or publishes each repair's cost on the runtime it ships with. This project supplies that measurement for small agents on llama.cpp and mlx-lm.

References

[1] Gerganov, G., & The ggml authors. (2023). *llama.cpp: LLM inference in C/C++* [Computer software]. GitHub. https://github.com/ggml-org/llama.cpp

The C/C++ inference engine and GGUF format on which many small models ship; it defines the Q4_0 and k-quant types (Q4_K at 4.5 bits per weight, the Q4_K_M mix with Q6_K for selected tensors), MXFP4, and importance-matrix-guided quantization. Its formats are the real 4-bit code paths our Inference Backends measure, and its mixed k-quant recipes are the deployed form of layer-selective added bits.

[2] Liquid AI. (2026, August 19). LFM2.5 Q4_0: Quantization-aware distillation for edge deployment. *Liquid AI Blog*. https://www.liquid.ai/blog/qad

Liquid AI releases QAD-trained Q4_0 GGUF checkpoints for LFM2.5-230M, 350M, 1.2B-Instruct, and 2.6B, closing 48.4% to 73.4% of the BF16-to-Q4_0 gap on a seven-task average that includes BFCLv4. It is direct industry evidence that QAD is shipped for exactly the small on-device models and llama.cpp format our testbed targets. The post reports only aggregate averages and gives no training data, teacher, or step details and no train-longer control.

[3] Sakizli, F. (2026). *TSCG: Deterministic tool-schema compilation for agentic LLM deployments* (arXiv:2605.04107). arXiv. https://doi.org/10.48550/arXiv.2605.04107 https://arxiv.org/abs/2605.04107

Compiles JSON tool schemas into compact structured text with eight composable operators, without model access or fine-tuning, and argues that the JSON format itself causes most tool-use failures of 4B to 14B models at production catalog sizes; it lifts Phi-4 14B from 0% to 84.4% at 20 tools with 52 to 57% token savings. It is an input-side, zero-inference-cost interface repair that complements grammar-constrained decoding (repair R1), and it has not been evaluated on quantized models.

[4] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. https://arxiv.org/abs/2208.07339

Identifies emergent outlier feature dimensions in transformer LLMs and keeps them in 16-bit through a mixed-precision decomposition while more than 99.9% of values are multiplied in 8-bit. It established that a small set of outliers dominates quantization error, the premise behind all later outlier-protecting and mixed-precision repairs that our R2 arm draws on.

[5] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*. https://arxiv.org/abs/2306.00978

Uses activation statistics to find the roughly 1% of salient weight channels and protects them by per-channel scaling rather than mixed precision, with the TinyChat runtime for on-

device 4-bit inference; MLSys 2024 best paper. It is one of the two 4-bit PTQ methods used in the agent-compression studies we cite and shows the systems community's focus on on-device weight-only quantization.

[6] Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefler, T., & Alistarh, D. (2024). SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2306.03078

Isolates outlier weights in a sparse higher-precision structure while quantizing the rest to 3 to 4 bits, and notes that such quantization hurts 1 to 10B edge-sized models most. It is both an example of outlier-based added bits (R2) and early evidence that smaller models are more fragile, which motivates our focus on 0.5 to 4B agents.

[7] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.17323

One-shot weight quantization that uses approximate second-order information to compensate rounding error, reaching 3 to 4 bits per weight on models up to 175B parameters. It is the reference PTQ algorithm in most quantization studies, including the agent studies our work builds on, and a baseline against which the simpler llama.cpp and MLX block formats can be compared.

[8] Tseng, A., Chee, J., Sun, Q., Kuleshov, V., & De Sa, C. (2024). QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235). https://arxiv.org/abs/2402.04396

Extends QuIP's incoherence processing with a randomized Hadamard transform and E8-lattice vector-quantization codebooks, reaching state-of-the-art quality at 4 bits and below. It represents the 'reduce rounding error everywhere' strategy of PTQ, which is consistent with the finding that damage is diffuse, although its codebook format is not among the types llama.cpp and MLX ship.

[9] Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on Apple silicon* [Computer software]. GitHub. https://github.com/ml-explore/mlx

Apple's array framework for Apple silicon, underlying mlx-lm. Its quantization defaults to affine 4-bit with a scale and bias per 64 weights (about 4.5 bits per weight) and also offers mxfp4, mxfp8, and nvfp4 modes; these are the MLX 4-bit code paths our testbed measures on the M4 Pro.

[10] Alvarez, E., Almog, O., Chung, E., Layton, S., Stosic, D., Krashinsky, R., & Aubrey, K. (2025, June 24). Introducing NVFP4 for efficient and accurate low-precision inference. *NVIDIA Technical Blog*. https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/

Defines NVFP4 (E2M1 values, an FP8 E4M3 scale per 16 values, and a per-tensor FP32 scale) and contrasts it with MXFP4's power-of-two E8M0 scale per 32 values. These microscaling formats are the target of NVIDIA's QAD and Multiverse's MXFP4 healing and are now exposed by MLX, so they bound which 4-bit formats our comparison can include on Apple silicon.

[11] Yvinec, E., & Culliton, P. (2025, April 18). Gemma 3 QAT models: Bringing state-of-the-art AI to consumer GPUs. *Google Developers Blog*. https://developers.googleblog.com/en/gemma-3-quantized-aware-trained-state-of-the-art-ai-to-consumer-gpus/

Google's release of Gemma 3 checkpoints trained for about 5,000 steps with the unquantized model's probabilities as targets, reducing the Q4_0 perplexity drop by 54% and shipping Q4_0 builds for llama.cpp, Ollama, and MLX. It shows a major vendor shipping distillation-style QAT for the same runtimes we measure, validated only by perplexity and aggregate preference scores.

[12] Xin, M., Priyadarshi, S., Xin, J., Kartal, B., Vavre, A., Thekkumpate, A. K., Chen, Z., Mahabaleshwarkar, A. S., Shahaf, I., Bercovich, A., Patel, K., Velury, S. V., Luo, C., Cheng, Z., Chen, J., Yu, C.-H., Ping, W., Rybakov, O., Tajbakhsh, N., . . . Mao, H. (2026). *Quantization-aware distillation for NVFP4 inference accuracy recovery* (arXiv:2601.20088). arXiv. https://doi.org/10.48550/arXiv.2601.20088 https://arxiv.org/abs/2601.20088

NVIDIA technical report that trains NVFP4-quantized students against the KL divergence to their BF16 teacher (QAD) and releases the resulting Nemotron checkpoints. It shows that cross-entropy QAT can match the teacher's loss while diverging in output distribution (KL 0.311 vs. 0.004) and can break RL-trained models, while QAD recovers near-BF16 scores. Evaluation uses aggregate reasoning, coding, and instruction-following scores, selects checkpoints by benchmark average, and has no train-longer-at-full-precision control, which are the gaps our R3 arm addresses.

[13] Apple. (2025). *Apple Intelligence foundation language models: Tech report 2025* (arXiv:2507.13575). arXiv. https://doi.org/10.48550/arXiv.2507.13575 https://arxiv.org/abs/2507.13575

Describes Apple's roughly 3B-parameter on-device model, compressed to 2 bits per weight with QAT plus LoRA quality-recovery adapters, and a developer framework with guided generation and constrained tool calling. It is an industry deployment that combines retraining, adapters, and output constraints, yet reports only aggregate quality (MMLU 67.8 to 64.4, IFEval 85.1 to 82.3) and no ablation separating the repairs.

[14] Kurtic, E., Marques, A. N., Pandit, S., Kurtz, M., & Alistarh, D. (2025). “Give me BF16 or give me death”? Accuracy-performance trade-offs in LLM quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (ACL 2025). https://aclanthology.org/2025.acl-long.1304/

Over 500,000 evaluations of FP8, INT8, and INT4 formats on the Llama-3.1 family find FP8 lossless and INT4 weight-only competitive with 8-bit, with deployment recommendations for vLLM. It is the strongest statement of the ‘quantization is nearly free’ view, which rests on aggregate scores for 8B and larger models and which our study tests for small agents.

[15] Li, S., Ning, X., Wang, L., Liu, T., Shi, X., Yan, S., Dai, G., Yang, H., & Wang, Y. (2024). Evaluating quantized large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235, pp. 28480-28524). https://proceedings.mlr.press/v235/li24bb.html

Evaluates weight, activation, and KV-cache PTQ on 11 model families from 125M to 180B parameters, finding 4-bit weight-only quantization nearly lossless on average, larger models more tolerant, and multi-step reasoning less tolerant than instruction following. It also documents that multiple-choice accuracy can rise under quantization through guessing on uncertain items, a concrete example of aggregate scores masking damage.

[16] Dutta, A., Krishnan, S., Kwatra, N., & Ramjee, R. (2024). Accuracy is not all you need. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://arxiv.org/abs/2407.09141

Shows that compressed models with accuracy within 0 to 2% of the baseline can still flip up to 13.6% of answers, that smaller models flip more, and proposes flips and KL divergence as distance metrics to report with accuracy. It is the peer-reviewed foundation for our claim that aggregate scores hide quantization damage and for measuring per-step disagreement with the full-precision reference.

[17] Jaiswal, A., Gan, Z., Du, X., Zhang, B., Wang, Z., & Yang, Y. (2024). Compressing LLMs: The truth is rarely pure and never simple. In *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2310.01382

Introduces the LLM-KICK benchmark and shows that perplexity fails to capture capability losses of compressed LLMs on knowledge-intensive tasks, with quantization faring better than pruning. It supports our argument that the metric used to validate a compression method determines whether damage is seen at all.

[18] Taheri, Y., Heydari, M. H., Naaman, E., & Fatemi, A. (2026). *Small reasoning models are instruction followers in function calling* (arXiv:2608.22472). arXiv. https://doi.org/10.48550/arXiv.2608.22472 https://arxiv.org/abs/2608.22472

Proposes Instruction-Followed Function Calling (IFFC), which delegates tool routing to a small reasoning model prompted in ordinary chat format, and tests it at FP16 and llama.cpp Q4_K_M. Under IFFC, Qwen3-0.6B in non-thinking mode falls from 62.3% to 16.8% on BFCL V3 non-live (single-turn), while thinking mode and models of 1.7B and larger lose little. It shows that 4-bit damage in small tool-calling models can be severe and depends on size and reasoning mode, which our damage map (RQ1) measures in multi-turn settings.

[19] Dong, P., Tang, Z., Liu, X., Li, L., Chu, X., & Li, B. (2025). Can compressed LLMs truly act? An empirical evaluation of agentic capabilities in LLM compression. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)* (Proceedings of Machine Learning Research, Vol. 267). https://proceedings.mlr.press/v267/dong25k.html

ACBench evaluates GPTQ, AWQ, and pruning on 12 agentic tasks and 15 models, finding that 4-bit quantization costs only 1 to 3% on workflow generation and tool use but 10 to 15% on real-world application tasks. It is the main peer-reviewed benchmark of compression effects on agents; it reports task-level accuracy rather than per-step failures and does not evaluate repairs.

[20] Jang, J., Yang, K., Lim, H., & Park, H. (2026). *Flat score, amplified failures: How the error budget masks damage in quantized LLM agents* (arXiv:2607.27275). arXiv. https://doi.org/10.48550/arXiv.2607.27275 https://arxiv.org/abs/2607.27275

Runs Gemma-4 and Qwen-3.5/3.6 agents (9B to 35B total parameters) at BF16, FP8, and AWQ INT4 on two τ^2-bench domains and finds no significant change in task score, while the failure each model already makes (tool-name hallucination in telecom) is amplified: the share of calls to nonexistent tools rises from 19.5% to 38.3% and each invented tool is called about 2.5× more often, hidden by the benchmark's ten-error budget. It is the direct motivation for our step-level logging and failure counting, and its scope (large models, vLLM on A100, one prompt repair) is exactly what our study extends to small models on llama.cpp and mlx-lm.

[21] Li, Y., Deng, J., Hu, J., Chen, T., Wang, S., & Yang, H. (2026). *Dynamic mixed-precision routing for efficient multi-step LLM interaction* (arXiv:2602.02711). arXiv. https://doi.org/10.48550/arXiv.2602.02711 https://arxiv.org/abs/2602.02711

Trains a router (KL-supervised, then GRPO-refined) that sends each step of an ALFWorld or WebShop episode to a BF16 or a 4-bit Qwen3-1.7B/4B model, after observing that step-wise KL divergence has a rare high-divergence mode. With 40% of steps at full precision, Qwen3-4B AWQ matches BF16 on ALFWorld (95.1% vs. 94.3%). It is the first evidence that step-level precision allocation can work for agents, but it keeps two separate models, reports latency only, uses no tool-calling benchmark, and does not compare against layer-selective or uniform added bits at a matched budget.

[22] Hu, J., & Ramachandran, S. (2026). *The structure of quantization damage in LLMs: Why the next bit should be spent globally* (arXiv:2609.01587). arXiv. https://doi.org/10.48550/arXiv.2609.01587 https://arxiv.org/abs/2609.01587

Uses causal mixed-precision intervention (restoring layers to 8-bit) on nine open models from 0.5B to 8B parameters, including Qwen3-0.6B and Qwen3-1.7B, and finds quantization damage diffuse across layers. At a matched +0.146 bits per weight, spending the budget globally on finer group-128 scales beats oracle-selected layer repair by 21 to 52 percentage points of recovered gap. It is the strongest current evidence against layer-selective added bits, but it uses continuation scoring only and no agent tasks, which motivates our R2 comparison of uniform, layer-selective, and step-selective precision.

[23] Dong, Z., Yao, Z., Gholami, A., Mahoney, M. W., & Keutzer, K. (2019). HAWQ: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2019)*. https://arxiv.org/abs/1905.03696

Selects each layer's precision from its Hessian spectrum, the classic sensitivity-driven approach to layer-wise mixed precision. It represents the 'choose which layers get more bits' idea that Hu and Ramachandran (2026) challenge and that our layer-selective R2 condition implements.

[24] Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., & Keutzer, K. (2024). SqueezeLLM: Dense-and-sparse quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235). https://arxiv.org/abs/2306.07629

Combines sensitivity-based non-uniform quantization with a dense-and-sparse decomposition that keeps outliers and sensitive weights in full precision, and argues that single-batch LLM decoding is memory-bandwidth bound. Both points apply to on-device agents: added bits must be weighed against the bandwidth they cost, which our Cost Profiler measures.

[25] Park, Y., Hyun, J., Cho, S., Sim, B., & Lee, J. W. (2024). Any-precision LLM: Low-cost deployment of multiple, different-sized LLMs. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)* (Proceedings of Machine Learning Research, Vol. 235, pp. 39682-39701). https://proceedings.mlr.press/v235/park24e.html

Overlays models quantized to 3 through n bits in the memory footprint of one n-bit model, with a serving engine that switches precision at runtime. It is the memory-efficient mechanism that would let a step-selective policy avoid holding two model copies, a cost our R2 comparison must account for on llama.cpp and MLX, which do not provide such nested formats.

[26] Chen, H. M., Tan, F., Kouris, A., Lee, R., Fan, H., & Venieris, S. I. (2025). Progressive mixed-precision decoding for efficient LLM inference. In *International Conference on Learning Representations (ICLR 2025)*. https://arxiv.org/abs/2410.13461

Allocates precision by inference phase, using higher precision for prefill and progressively lower precision deeper into the generated sequence, with task- or prompt-adaptive schedulers. It shows that precision can usefully vary over time within one generation, a finer-grained relative of the per-conversation-step allocation our R2 arm tests.

[27] Wang, D., Kim, J., Han, S., Gudovskiy, D., Nakata, Y., Okuno, T., Peong, K., Jeon, K. E., Ko, J. H., Chen, Y., & Yang, H. (2026). *MoBiQuant: Mixture-of-bits quantization for token-adaptive any-precision LLM* (arXiv:2602.20191). arXiv. https://doi.org/10.48550/arXiv.2602.20191 https://arxiv.org/abs/2602.20191

Proposes recursive residual quantization with a token-aware router that selects each token's inference precision, motivated by 'outlier migration' of sensitive tokens across bit-widths, and reports up to 1.34x throughput over prior any-precision methods. It is the closest

weight-level mechanism to the step-selective precision our R2 arm tests, but it needs its own kernels and was not evaluated on agents or on llama.cpp/MLX formats.

[28] Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., & Chandra, V. (2024). LLM-QAT: Data-free quantization aware training for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 467-484). https://aclanthology.org/2024.findings-acl.26/

First QAT recipe for LLMs; it distills from the full-precision model's own generations so that no original training data is needed, and quantizes weights, activations, and KV cache down to 4 bits on 7B to 30B models. It is the academic ancestor of the QAD recipes now shipped by industry and of the self-generated-data option in our QAD Training Pipeline.

[29] Du, D., Zhang, Y., Cao, S., Guo, J., Cao, T., Chu, X., & Xu, N. (2024). BitDistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (ACL 2024). https://aclanthology.org/2024.acl-long.7/

Combines QAT with a confidence-aware KL-divergence self-distillation objective and asymmetric clipping to improve 2 and 3-bit LLMs with less data. It is peer-reviewed evidence that a KL objective to the full-precision model is a better retraining target than hard labels, the same design choice behind industrial QAD.

[30] Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., & Luo, P. (2025). EfficientQAT: Efficient quantization-aware training for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 10081-10100). https://aclanthology.org/2025.acl-long.498/

Makes QAT affordable by training all parameters block by block and then only the quantization step sizes end to end, producing a 2-bit Llama-2-70B on one A100 in 41 hours. Its emphasis on single-GPU QAT cost is relevant to our constraint of free Colab/Kaggle GPUs, although it evaluates only aggregate accuracy.

[31] Liu, Z., Zhao, C., Huang, H., Chen, S., Zhang, J., Zhao, J., Roy, S., Jin, L., Xiong, Y., Shi, Y., Xiao, L., Tian, Y., Soran, B., Krishnamoorthi, R., Blankevoort, T., & Chandra, V. (2025). ParetoQ: Improving scaling laws in extremely low-bit LLM quantization. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. https://arxiv.org/abs/2502.02631

A unified QAT framework comparing 1 to 4-bit settings, including sub-1B models, which finds a transition between 2 and 3 bits: at 3 bits and above, fine-tuned models stay close to their pre-trained distributions. This suggests that 4-bit QAD for small models is a small correction rather than relearning, which is why a train-longer control is needed to separate the two.

[32] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2305.14314

Backpropagates through a frozen 4-bit NormalFloat (NF4) base model into low-rank adapters, making fine-tuning of quantized models feasible on a single GPU. It is the

template for parameter-efficient repair of a quantized model, a lower-cost alternative to full QAD that fits the free Colab/Kaggle GPUs our QAD Training Pipeline uses.

[33] Ryskulov, B., García-Ferrero, I., Montero, D., Jansen, D., Hashemi, A., Garcia, J. R., Tiene, A., & Orús, R. (2026). *Quantization-aware healing: A practical recipe for recovering compressed, 4-bit LLMs* (arXiv:2608.20953). arXiv. https://doi.org/10.48550/arXiv.2608.20953 https://arxiv.org/abs/2608.20953

Multiverse Computing's recipe that distills a structurally compressed MXFP4 student directly from the uncompressed original model, producing the open-weight Hypernova-60B; the student matches or beats its BF16 source on 7 of 9 benchmarks and peaks about 7 times faster than cross-entropy QAT. The authors attribute the 4-bit model's advantage to an extra distillation pass the BF16 checkpoint never received, which is exactly the confound our train-longer control isolates. Its one agent result is an aggregate tau2-bench score (61.7 vs. 59.4).

[34] Dremov, A., Grangier, D., Katharopoulos, A., & Hannun, A. (2026). Compute-optimal quantization-aware training. In *International Conference on Learning Representations (ICLR 2026)*. https://arxiv.org/abs/2509.22935

Studies how to split compute between a full-precision phase and a QAT phase for models from 86M to 2.2B parameters and finds that the loss-optimal QAT fraction grows with total compute. It shows that how much training happens at full precision versus under quantization matters, which is why our R3 arm pairs each QAD run with a train-longer-at-full-precision control.

[35] Geng, S., Josifoski, M., Peyrard, M., & West, R. (2023). Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 10932–10952). Association for Computational Linguistics. https://aclanthology.org/2023.emnlp-main.674/

Shows that grammar-constrained decoding, including input-dependent grammars, lets unmodified models solve structured prediction tasks. It is the peer-reviewed basis for our per-episode grammars built from each task's tool list.

[36] Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., & Chen, T. (2025). XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of Machine Learning and Systems 7 (MLSys 2025)*. https://proceedings.mlsys.org/paper_files/paper/2025/file/5c20ca4b0b20b0bd2f1d839dc605e70f-Paper-Conference.pdf

Splits the vocabulary into context-independent tokens checked ahead of time and context-dependent tokens checked at runtime, reporting up to 100× faster mask generation and near-zero end-to-end overhead in SGLang and MLC-LLM; its evaluation also found llama.cpp's grammar engine slower in preprocessing and per-token processing. It sets the performance bar for repair R1 and motivates measuring grammar overhead on our own runtimes (RQ4).

[37] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

Shows that a language model can learn in a self-supervised way which APIs to call, when, and with which arguments. It is the starting point of the tool-use literature and frames tool calling as a learned, and therefore perturbable, model behavior.

[38] Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2024). Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html

Fine-tunes a LLaMA model with retriever-aware training to write API calls, introduces APIBench, and reports reduced hallucination of API usage. It is the precursor of BFCL and an early treatment of hallucinated calls, the failure type our classifier counts first.

[39] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR 2024)*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html

Builds ToolBench from more than 16,000 real REST APIs, trains ToolLLaMA, and evaluates with an LLM-judged pass rate and win rate (ToolEval). We compare it in our benchmark table; its judge-based metric and single-query episodes make it less suited to per-step failure counting than BFCL multi-turn or τ^2-bench.

[40] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

Introduces the interleaved reason-act-observe loop that underlies current tool-calling agents. Our Episode Runner drives exactly this loop, and its step structure defines the unit our Step Logger records.

[41] Erdogan, L. E., Lee, N., Jha, S., Kim, S., Tabrizi, R., Moon, S., Hooper, C., Anumanchipalli, G., Keutzer, K., & Gholami, A. (2024). TinyAgent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 80–88). Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-demo.9/

Fine-tunes 1.1B and 7B models for function calling, adds tool retrieval and quantization, and runs a local assistant on a MacBook. It shows that small quantized tool-calling agents on Apple laptops are practical, which is the deployment setting our testbed measures.

[42] Paramanayakam, V., Karatzas, A., Anagnostopoulos, I., & Stamoulis, D. (2025). Less is more: Optimizing function calling for LLM execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE. https://arxiv.org/abs/2411.15399

Shows that offering an edge model fewer tools improves function-calling success, execution time, and power on NVIDIA Jetson hardware, and reports that Llama-3.1-8B served through Ollama drops from 63.0% on BFCL at full precision to 20.4% (q4_0) and 39.6% (q4_K_M). It is peer-reviewed evidence that GGUF 4-bit formats can damage tool calling, and that damage depends on the format.

[43] Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., . . . Qiu, Z. (2025). *Qwen3 technical report* (arXiv:2505.09388). arXiv. https://arxiv.org/abs/2505.09388

Describes the Qwen3 family (dense models from 0.6B, plus MoE models) with switchable thinking and non-thinking modes. Its 0.6B, 1.7B, and 4B dense models are among our study models, and the thinking-mode switch is a factor we must hold fixed because it changes quantization sensitivity.

[44] Amini, A., Banaszak, A., Benoit, H., Böök, A., Dakhran, T., Duong, S., Eng, A., Fernandes, F., Härkönen, M., Harrington, A., Hasani, R., Karwa, S., Khrustalev, Y., Labonne, M., Lechner, M., Lechner, V., Lee, S., Li, Z., Loo, N., . . . Tumma, N. (2025). *LFM2 technical report* (arXiv:2511.23404). arXiv. https://arxiv.org/abs/2511.23404

Describes Liquid AI's LFM2 family (350M to 8.3B), whose gated short-convolution and attention hybrid backbone was chosen by hardware-in-the-loop search under edge latency and memory limits. It documents the architecture of our LFM2.5 study models, which differs from the pure transformers of Qwen3 and SmolLM3 and may respond differently to 4-bit quantization.

[45] Liquid AI. (2026, January 5). *Introducing LFM2.5: The next generation of on-device AI* [Blog post]. https://www.liquid.ai/blog/introducing-lfm2-5-the-next-generation-of-on-device-ai

Announces LFM2.5-1.2B (Base, Instruct, and specialized variants) as a building block for on-device agentic applications, with GGUF and MLX checkpoints at release. It establishes that our LFM2.5 study model is built and shipped for exactly the runtimes our testbed targets.

[46] Bakouch, E., Patiño, C. M., Lozhkov, A., Beeching, E., Roucher, A., Tazi, N., et al. (2025, July 8). *SmolLM3: Smol, multilingual, long-context reasoner* [Blog post]. Hugging Face. https://huggingface.co/blog/smollm3

Releases SmolLM3, a 3B model with 128K context, tool calling, and /think and /no_think modes, with its full training recipe. It is our fully open study model, which lets the QAD arm and its train-longer control use public post-training data.

[47] Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., & Gonzalez, J. E. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267, pp. 48371–48392). PMLR. https://proceedings.mlr.press/v267/patil25a.html

Defines BFCL, which scores function calls by AST matching and evaluates abstention and reasoning in stateful multi-step settings; the public leaderboard is the de facto industry standard for function calling. BFCL multi-turn is one of our two primary benchmarks, and its single-turn categories are a candidate easier set for the smallest models.

[48] Mao, H., Yan, F., Ji, C. C.-J., Huang, J., Suresh, V., Huang, Y., Yu, X., Gonzalez, J. E., & Patil, S. G. (2024, September 19). *BFCL V3: Multi-turn & multi-step function calling* [Blog post]. Gorilla, University of California, Berkeley. https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html

Documents the BFCL V3 multi-turn categories (base, missing parameters, missing functions, long context, composite), state-based and response-based checks after each turn, error messages returned to the model, and a 20-step cap per turn. These rules define how much error BFCL multi-turn tolerates, which our Benchmark Adapter must reproduce exactly.

[49] Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2025). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR 2025)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html

Introduces conversations between a tool-using agent and an LLM-simulated user under a domain policy, grades the final database state, and proposes pass^k, the probability that all k trials succeed; even GPT-4o had pass^8 below 25% in retail. pass^k is one of the reliability metrics our Analysis & Stats component reports alongside per-failure-type rates.

[50] Barres, V., Dong, H., Ray, S., Si, X., & Narasimhan, K. (2026). τ²-Bench: Evaluating conversational agents in a dual-control environment. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*. https://arxiv.org/abs/2506.07982 https://icml.cc/virtual/2026/oral/71171

Extends τ-bench with a dual-control telecom domain (a Dec-POMDP in which the simulated user also acts on shared state), a compositional task generator, and a tool-constrained user simulator. Its default configuration ends an episode after 10 failed tool calls, the error budget that Jang et al. show masks quantization damage; our primary benchmark is its corrected Verified task set.

[51] Cuadron, A., Yu, P., Liu, Y., & Gupta, A. (2025). *SABER: Small actions, big errors — Safeguarding mutating steps in LLM agents* (arXiv:2512.07850). arXiv. Poster at the ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents). https://arxiv.org/abs/2512.07850

Amazon AGI analysis showing that deviations at state-changing (mutating) steps decide task success on τ-bench and SWE-bench Verified while read-only deviations barely matter; it also releases τ-Bench Verified, the corrected task set published as amazon-agi/tau2-

bench-verified. It is the paper behind our primary benchmark, and its mutating/non-mutating split is a step attribute our Failure Classifier records.

[52] Barres, V., Shi, B., Zytek, O., Ray, S., Dhandhania, K., & Razavi, P. (2026, March 18). *τ³-Bench: Advancing agent benchmarking to knowledge and voice* [Blog post]. Sierra. https://sierra.ai/blog/bench-advancing-agent-benchmarking-to-knowledge-and-voice

Sierra's release adding a knowledge-retrieval banking domain (best model 25.5%) and full-duplex voice evaluation to the τ-bench leaderboard, and folding external task fixes, especially Amazon's τ²-Bench Verified, into the official task sets. It shows that one benchmark name now covers several task versions, so our Experiment Registry must pin the exact benchmark release.

[53] Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., & Li, Y. (2023). API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 3102–3116). Association for Computational Linguistics. https://aclanthology.org/2023.emnlp-main.187/

Provides 314 annotated tool-use dialogues with 753 API calls over 73 executable APIs, scoring each call's accuracy against the annotation and responses by ROUGE-L. It is an early dialogue-based tool benchmark; because each call is scored given the annotated history, it cannot show how errors compound across an episode, which is why we use BFCL multi-turn and τ²-bench instead.

[54] Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., Liu, Z., Sun, M., & Liu, Y. (2024). StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 11143–11156). Association for Computational Linguistics. https://aclanthology.org/2024.findings-acl.664/

Makes ToolBench reproducible by replacing unstable live APIs with a virtual API server (a response cache plus LLM-based API simulators) and a stabilized GPT-4 evaluator for solvable pass and win rates. It is the direct precedent for our Simulated-User Cache: cached external responses make repeated runs comparable and free.

[55] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., . . . Tang, J. (2024). AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2308.03688

Evaluates LLMs as agents across eight interactive environments with environment-specific scores. We include it in our benchmark comparison; it tests general agency rather than tool-schema compliance, so it is not one of our benchmarks.

[56] Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*. https://neurips.cc/virtual/2025/poster/121528

Builds MAST, a taxonomy of 14 failure modes in three categories from more than 1,600 annotated multi-agent traces, validated by human agreement ($\kappa = 0.88$) and scaled with an LLM judge. It is the methodological model for our Failure Classifier: define failure types from traces, check annotator agreement, then automate.

[57] Xu, H., Zhu, Z., Pan, L., Wang, Z., Zhu, S., Ma, D., Cao, R., Chen, L., & Yu, K. (2025). Reducing tool hallucination via reliability alignment. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)* (Proceedings of Machine Learning Research, Vol. 267). PMLR. https://icml.cc/virtual/2025/poster/45001

Defines tool hallucination as tool-selection or tool-usage errors, introduces RelyToolBench, and proposes Relign, which lets a model defer, ask, or switch tools. Its two-way split informs our distinction between hallucinated or wrong tools and invalid arguments.

[58] Willard, B. T., & Louf, R. (2023). *Efficient guided generation for large language models* (arXiv:2307.09702). arXiv. https://arxiv.org/abs/2307.09702

Reformulates regex- and grammar-guided generation as transitions of a finite-state machine indexed over the vocabulary, the basis of the Outlines library. It is the foundational engineering paper for repair R1 (output restriction).

[59] Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., & Nori, H. (2025). *JSONSchemaBench: A rigorous benchmark of structured outputs for language models* (arXiv:2501.10868). arXiv. https://arxiv.org/abs/2501.10868

Evaluates six constrained-decoding frameworks (Guidance, Outlines, llama.cpp, XGrammar, OpenAI, Gemini) on about 10,000 real-world JSON schemas for efficiency, coverage, and output quality. It shows that engines differ in which schema features they support, a risk we check before relying on llama.cpp's JSON-schema conversion.

[60] Park, K., Wang, J., Berg-Kirkpatrick, T., Polikarpova, N., & D'Antoni, L. (2024). Grammar-aligned decoding. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/2bdc2267c3d7d01523e2e17ac0a754f3-Abstract-Conference.html

Shows that greedy grammar-constrained decoding distorts the model's distribution, producing grammatical but improbable outputs, and proposes the ASAp sampler that matches the grammar-conditioned distribution. It explains how a grammar repair can change which tool or argument a model picks, not only its syntax.

[61] Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H.-y., & Chen, Y.-N. (2024). Let me speak freely? A study on the impact of format restrictions on large language model performance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track* (pp. 1218–1236). Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-industry.91/

Finds that restricting output to formats such as JSON degrades reasoning, and that stricter restrictions degrade it more. It is the main evidence that repair R1 can have a quality cost, which our comparison must measure rather than assume away.

[62] Miller, E. (2024). *Adding error bars to evals: A statistical approach to language model evaluations* (arXiv:2411.00640). arXiv. https://doi.org/10.48550/arXiv.2411.00640 https://arxiv.org/abs/2411.00640

Treats LLM evaluations as experiments and recommends standard errors, clustered errors for grouped questions (which can be over three times the naive ones), paired per-question comparisons, and power analysis. These recommendations shape our paired, task-clustered analysis plan.

[63] Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1383–1392). Association for Computational Linguistics. https://aclanthology.org/P18-1128/

Gives a protocol for choosing significance tests in NLP, including McNemar's test for paired binary outcomes and paired bootstrap and permutation tests. It justifies our choice of paired tests for per-task pass/fail outcomes against the full-precision reference.

[64] Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science, 8*(4), 355–362. https://doi.org/10.1177/1948550617697177

Explains the two one-sided tests (TOST) procedure for showing that an effect lies within pre-specified equivalence bounds. We use TOST to test whether a repaired model's overall score is equivalent to the full-precision score (RQ3).

[65] Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70. https://www.jstor.org/stable/4615733

Introduces the sequentially rejective (Holm–Bonferroni) procedure, which controls the family-wise error rate and is uniformly more powerful than plain Bonferroni. It is our pre-registered multiple-comparison correction.

[66] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological), 57*(1), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x https://academic.oup.com/jrsssb/article/57/1/289/7035855

Introduces false-discovery-rate control, a more powerful alternative to family-wise correction when many hypotheses are tested. It is the reference for screening our many exploratory per-failure-type comparisons.

[67] He, H., & Thinking Machines Lab. (2025, September 10). Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

Shows that greedy LLM inference is nondeterministic mainly because kernels are not batch-invariant, so results depend on server load, and gives batch-invariant kernels that fix this. It is why our testbed repeats trials, logs seeds and runtime versions, and measures a noise floor for each backend.

[68] Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences, 115*(11), 2600–2606. https://doi.org/10.1073/pnas.1708274114 https://www.pnas.org/doi/10.1073/pnas.1708274114

Argues that specifying hypotheses and analyses before seeing the data separates confirmatory from exploratory findings. It motivates registering our analysis plan on OSF before the main data collection.

[69] Bertinetto, L., Henriques, J. F., Albanie, S., Paganini, M., & Varol, G. (Eds.). (2021). *NeurIPS 2020 Workshop on Pre-registration in Machine Learning* (Proceedings of Machine Learning Research, Vol. 148). PMLR. https://proceedings.mlr.press/v148/

Proceedings of a two-stage pilot in which ML papers were reviewed and accepted on their experimental protocol before results existed, and the results papers were published afterwards in PMLR. It is the ML-community precedent for our pre-registered analysis plan.

[70] Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research, 22*, 1–20. https://www.jmlr.org/papers/v22/20-303.html

Reports on the NeurIPS 2019 reproducibility program, including its code-submission policy and ML reproducibility checklist. Our testbed's release plan (code, step logs, configs, statistical reporting) follows that checklist.

[71] Apple (ml-explore). (2026). *MLX LM: Run LLMs with MLX* (Version 0.31.3) [Computer software]. GitHub. https://github.com/ml-explore/mlx-lm

Python package for running and converting LLMs with MLX: mlx_lm.convert quantizes Hugging Face checkpoints (-q, --q-bits, --q-group-size, --q-mode, and mixed-precision --

quant-predicate recipes), mlx_lm.server exposes an OpenAI-style endpoint with tool-call parsing, and generate_step accepts custom logits processors. We use it as the MLX inference backend and as the target for MLX exports of QAD checkpoints.

[72] Meta AI. (2024, October 24). *Introducing quantized Llama models with increased speed and a reduced memory footprint* [Blog post]. https://ai.meta.com/blog/meta-llama-quantized-lightweight-models/

Releases Llama 3.2 1B and 3B quantized with QAT plus LoRA adaptors (QLoRA) and with SpinQuant post-training quantization, using 4-bit group-wise weights and 8-bit dynamic activations, run through ExecuTorch on Arm phone CPUs with a 2 to 4× speedup. It is a vendor example of retraining under quantization for small on-device models, validated on aggregate benchmarks.

[73] Lacombe, O., & Sanseviero, O. (2026, June 5). *Gemma 4 QAT models: Optimizing model compression for mobile and laptop efficiency* [Blog post]. Google. https://blog.google/innovation-and-ai/technology/developers-tools/quantization-aware-training-gemma-4/

Releases QAT checkpoints for Gemma 4 in Q4_0 and a new mobile-specific format, bringing the text-only E2B model under 1 GB, with llama.cpp, Ollama, LM Studio, MLX, vLLM, SGLang, and LiteRT-LM listed as runtimes. It is the most recent vendor QAT release and shows that retraining now targets the same runtimes we measure.

[74] Black, K., & Kumar, R. (2025, December 18). *FunctionGemma: Bringing bespoke function calling to the edge* [Blog post]. Google. https://blog.google/innovation-and-ai/technology/developers-tools/functiongemma/

Releases FunctionGemma, a 270M Gemma 3 variant specialized for on-device function calling, whose Mobile Actions accuracy rises from 58% to 85% after fine-tuning, with support in LiteRT-LM, llama.cpp, Ollama, MLX, and LM Studio. It is a leading-edge on-device agent product in the parameter range just below our study models.

[75] OpenAI. (2024, August 6). *Introducing Structured Outputs in the API* [Blog post]. https://openai.com/index/introducing-structured-outputs-in-the-api/

Launches Structured Outputs, which guarantees that model outputs, including function calls in strict mode, adhere to a developer-supplied JSON Schema. It shows output restriction as a standard industry feature for cloud agents; our R1 repair brings the same guarantee to local runtimes.

[76] vLLM Project. (2026). *Structured outputs* [Documentation]. Retrieved September 27, 2026, from https://docs.vllm.ai/en/latest/features/structured_outputs/

Documents vLLM's structured-output support with xgrammar and guidance backends and an automatic backend choice. It confirms that XGrammar-style constrained decoding is standard in open serving engines.

[77] Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). *Small language models are the future of agentic AI* (arXiv:2506.02153). arXiv. https://doi.org/10.48550/arXiv.2506.02153 https://arxiv.org/abs/2506.02153

An NVIDIA position paper arguing that small language models are sufficiently capable, better suited, and more economical than large models for most invocations in agentic

systems. It supports our premise that small models are becoming the default engine for agents, which makes their quantized behavior a practical concern.

[78] Hugging Face. (2026). *Qwen/Qwen3-0.6B* [Model repository; download statistics]. Retrieved September 27, 2026, from https://huggingface.co/Qwen/Qwen3-0.6B

The Hugging Face model page for Qwen3-0.6B. On 27 September 2026 the Hub reported about 29.2 million downloads in the previous 30 days and about 229 million in total. We cite it as an indicator of demand for sub-1B models, not as a deployment count.

[79] EuroMLSys. (2026). *The 6th Workshop on Machine Learning and Systems (EuroMLSys 2026)*. Retrieved September 27, 2026, from https://euromlsys.eu/

Workshop website listing the 2026 dates: paper submission February 24, 2026, notification March 22, workshop April 27 in Edinburgh. We use it with the 2025 dates to estimate the unannounced 2027 deadline.

[80] EuroSys. (2026). *EuroSys 2027: European Conference on Computer Systems* and *Call for workshops*. Retrieved September 27, 2026, from https://2027.eurosys.org/

States that EuroSys 2027 takes place April 19 to 23, 2027 in Rabat, Morocco, with workshops on Monday, April 19, and that workshop proposals are due October 16, 2026. EuroMLSys 2027 and its deadline had not been announced as of September 27, 2026.

[81] Google. (2026). *Colaboratory: Frequently asked questions*. Retrieved September 27, 2026, from https://research.google.com/colaboratory/faq.html

States that free-tier Colab GPU types vary over time, that usage limits are dynamic and unpublished, and that free notebooks run for at most 12 hours. These terms drive our design of short, checkpointed QAD runs.

[82] Kaggle. (2026). *Notebooks* [Documentation]. Retrieved September 27, 2026, from https://www.kaggle.com/docs/notebooks

Kaggle's notebook documentation: 12 hours of execution per CPU or GPU session, a single P100 or two T4 GPUs with 29 GB of RAM, and a warning that free GPU availability is limited and may queue. It bounds how long an uninterrupted QAD run can be and motivates checkpoint and resume.

[83] NVIDIA. (2026). *NVIDIA Academic Grant Program for researchers*. Retrieved September 27, 2026, from https://www.nvidia.com/en-us/industries/higher-education-research/academic-grant-program/

NVIDIA's compute grant program for academic researchers, including an "AI Inference, Agents, and Systems Software" area. It requires a full-time faculty principal investigator and was not accepting new applications when checked. We name it as a possible funding route through our advisor, not as a commitment.

[84] Hugging Face. (2026). *Using GPU Spaces* [Documentation]. Retrieved September 27, 2026, from https://huggingface.co/docs/hub/spaces-gpus

Documents GPU hardware for Hugging Face Spaces and the community GPU grant that projects can request from a Space's settings. It is a possible no-cost host for our public damage-map report and demo.

[85] Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

Introduces Cohen's kappa, a chance-corrected agreement statistic for two raters assigning nominal categories. We use it to measure agreement between human labelers and between humans and the Failure Classifier (C6).

[86] Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall/CRC. https://doi.org/10.1201/9780429246593

Standard reference on bootstrap resampling and bootstrap confidence intervals. We use a cluster (task-level) bootstrap for effect-size confidence intervals.

[87] Connor, R. J. (1987). Sample size for testing differences in proportions for the paired-sample design. *Biometrics, 43*(1), 207–211. https://doi.org/10.2307/2531961

Gives a simple, conservative approximation for the sample size needed to test a difference between paired proportions. We use it to estimate how many full-precision-solvable tasks each model needs for 80% power, which drives the task-pool size and the inclusion rule.

[88] Liquid AI. (2026). *LFM Open License v1.0*. Retrieved September 27, 2026, from https://huggingface.co/LiquidAI/LFM2.5-2.6B/raw/main/LICENSE https://www.liquid.ai/lfm-license

The license of the LFM2.5 models. It is Apache-2.0-based and allows derivative works with the license and notices, but it does not license commercial use by entities with annual revenue of $10 million or more. It governs how we may release LFM2.5-derived checkpoints.

[89] Association for Computing Machinery. (2020). *Artifact review and badging* (Version 1.1, August 24, 2020). https://www.acm.org/publications/policies/artifact-review-and-badging-current

ACM's policy defining the Artifacts Available, Artifacts Evaluated (Functional and Reusable), and Results Reproduced/Replicated badges. It sets the packaging targets for our open-source testbed and datasets.

[90] Gerganov, G., & llama.cpp contributors. (2023). *Performance of llama.cpp on Apple Silicon M-series* (Discussion No. 4167) [Online forum post]. GitHub. Retrieved September 27, 2026, from https://github.com/ggml-org/llama.cpp/discussions/4167

Community benchmark table of llama.cpp prompt-processing and text-generation speed on every Apple M-series chip, including the M4 Pro (about 50 tokens/s decode and 360 to 440 tokens/s prefill for a 7B Q4_0 model). We scale these figures to estimate episode times and total machine-hours for our grid.

[91] OpenAI. (2026). *API pricing* [Documentation]. Retrieved September 27, 2026, from https://developers.openai.com/api/docs/pricing

List prices per million tokens for OpenAI models (for example GPT-4.1-mini at $0.40 input, $0.10 cached input, $1.60 output; GPT-4.1-nano at $0.10, $0.025, $0.40; GPT-5.1 at $1.25, $0.125, $10.00). We use them to estimate the simulated-user cost of τ^2-bench and to show why the $300 cap requires a mini- or nano-class simulator.

[92] Amazon AGI. (2025). *τ²-Bench-Verified* [Computer software and dataset]. GitHub. https://github.com/amazon-agi/tau2-bench-verified

A corrected, human-verified version of the τ²-bench task set that fixes task definitions, expected actions, and evaluation criteria that did not match the domain policies or databases; the evaluation code is otherwise identical to τ²-bench and uses LiteLLM for model calls. This is the tau2-bench-verified variant our Benchmark Adapter targets.

[93] Risdal, M. (2020, August 7). *[Notebooks update] More GPU hours: Introducing a "floating" GPU quota* [Online forum post]. Kaggle. https://www.kaggle.com/discussions/product-feedback/173129

Kaggle staff announcement that the weekly GPU quota, normally 30 hours, floats upward when demand allows and resets weekly. It is the source for our planning figure of about 30 GPU-hours per member per week for QAD training.

[94] IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). IEEE. https://doi.org/10.1109/IEEESTD.1998.88286 https://ieeexplore.ieee.org/document/720574

The IEEE recommended practice for writing software requirements specifications; section 4.3 defines eight characteristics of a good SRS (correct, unambiguous, complete, consistent, ranked, verifiable, modifiable, traceable). We use it to check the quality of our requirements. It was superseded by ISO/IEC/IEEE 29148:2011 but remains the reference named in the course template.

[95] Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2020). ZeRO: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* (pp. 1–16). IEEE. https://arxiv.org/abs/1910.02054

Analyzes the memory consumed by model states in data-parallel training and shows that mixed-precision Adam needs about 16 bytes per parameter (FP16 weights and gradients plus FP32 master weights, momentum, and variance). We use this figure to size full-parameter QAD runs against the 16 GB of a free T4 or P100 GPU.

[96] Liquid AI. (2026, August 4). *Deploy local agents everywhere with LFM2.5-2.6B* [Blog post]. Hugging Face. https://huggingface.co/blog/LiquidAI/lfm2-5-2-6b

Introduces LFM2.5-2.6B (2.69B parameters), post-trained with agentic reinforcement learning for tool calling and multi-step workflows, with day-one llama.cpp and MLX support. It is the larger LFM2.5 model in our study.

[97] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157. https://doi.org/10.1007/BF02295996

Introduces the test for a difference between paired proportions now known as McNemar's test. We use its exact form as a secondary test on seed-matched binary task outcomes between a repair and its control.

[98] Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)* (pp. 220–229). ACM. https://doi.org/10.1145/3287560.3287596 https://dl.acm.org/doi/10.1145/3287560.3287596

Proposes model cards, short documents that report a model's intended use, evaluation data, metrics, and caveats. Our released repaired checkpoints will carry model cards in this format, including quantization format and failure-rate results.

[99] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)* (pp. 611–626). Association for Computing Machinery. https://doi.org/10.1145/3600006.3613165 https://dl.acm.org/doi/10.1145/3600006.3613165

Introduces vLLM and PagedAttention, which manage the KV cache in fixed-size blocks to raise throughput when many requests are batched on GPU servers. We cite it to explain why vLLM, designed for high-throughput batched serving, is not the runtime we measure for single-user on-device agents.

[100] Zhang, R. H., Yuan, L. X., Chen, C. J., Curtin, E., & Goin, M. (2026, September 22). *Announcing vllm-metal: Concurrent serving on Apple silicon* [Blog post]. vLLM Blog. https://vllm.ai/blog/2026-09-22-vllm-metal-v0-28-0

Announces vllm-metal v0.28.0, a community-maintained plugin that brings vLLM's scheduler, paged KV cache, and OpenAI-compatible server to Apple silicon, with MLX and Metal handling model execution. It shows that vLLM on a Mac adds a serving layer on top of MLX, the runtime we already measure directly.

[101] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. https://www.pearson.com/en-us/subject-catalog/p/design-patterns-elements-of-reusable-object-oriented-software/P200000009480/9780321700698

The standard catalog of 23 object-oriented design patterns. Our class design uses its Strategy (backends, repairs), Adapter (benchmarks), Proxy (simulated-user cache), and Chain of Responsibility (failure rules) patterns.

[102] MacIver, D. R., Hatfield-Dodds, Z., & Many Other Contributors. (2019). Hypothesis: A new approach to property-based testing. *Journal of Open Source Software, 4*(43), 1891. https://doi.org/10.21105/joss.01891

Describes Hypothesis, the Python property-based testing library that generates and shrinks test inputs from declared strategies. We use it to fuzz the tool-call parser, schema validator, and cache-key functions.

[103] GitHub. (2026). *GitHub-hosted runners reference*. GitHub Docs. Retrieved September 27, 2026, from https://docs.github.com/en/actions/reference/runners/github-hosted-runners

Official list of GitHub Actions hosted runners. It documents arm64 macOS runners (labels macos-14, macos-15, macos-26, macos-latest) with 3 M1 cores and 7 GB RAM, free on public repositories, and notes that nested virtualization is unsupported because of Apple's Virtualization Framework. We use it to decide which tests run in hosted CI and which must run locally on Apple silicon.

[104] Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

Develops kappa-type agreement measures and proposes the widely used interpretation bands (0.81 to 1.00 as almost perfect agreement). We use it to justify our kappa ≥ 0.80 acceptance target for failure labels.

[105] EuroSys 2022 Artifact Evaluation Committee. (2022). *Badges*. Systems Research Artifacts. https://sysartifacts.github.io/eurosys2022/badges

Artifact-evaluation guidance for EuroSys 2022, the systems conference with which EuroMLSys is co-located. It awards the Available, Functional, and Reproduced badges defined by ACM v1.1 and gives author checklists for each, which we adopt as our release checklist.

[106] Apple. (2024). *MacBook Pro (14-inch, M4 Pro or M4 Max, 2024) - Tech specs*. Retrieved September 27, 2026, from https://support.apple.com/en-us/121553

Apple's specification page for the M4 Pro MacBook Pro, listing 24 GB as the base unified memory for M4 Pro configurations (48 GB optional) and 273 GB/s memory bandwidth. It bounds what our measurement machine can hold without us assuming its exact configuration.

[107] Kaggle. (2026). *Efficient GPU usage tips* [Documentation]. Retrieved September 27, 2026, from https://www.kaggle.com/docs/efficient-gpu-usage

Kaggle's documentation of its notebook GPU quota (about 30 GPU-hours per week, sometimes more depending on demand), accelerator options (P100 or two T4s, both drawing on the same quota), and 12-hour GPU sessions. It is the second free training platform for our QAD pipeline.

[108] EuroMLSys. (2025). *The 5th Workshop on Machine Learning and Systems (EuroMLSys 2025)*. Retrieved September 27, 2026, from https://2025.euromlsys.eu/

Workshop website listing the 2025 dates (submission February 11, 2025 after extension; workshop March 31 in Rotterdam) and the six-page, two-column paper format. Used for deadline estimation and paper planning.

Chapter 2. Project Justification

Small Agents Are Moving On-Device

Small open models now run tool-calling agents on personal devices. NVIDIA researchers argue that small models are capable enough, and far cheaper, for most agent calls [77]. Apple opens its roughly 3B-parameter on-device model, trained with 2-bit quantization-aware training, to third-party apps with constrained tool calling [13]. Demand is large: Qwen3-0.6B alone was downloaded about 29 million times from Hugging Face in the 30 days before 27 September 2026 [78]. Downloads are not deployments, but they show where practitioners are heading.

Quantization Is Unavoidable

Weight memory is roughly parameters × bits ÷ 8. A 4B model needs about 8 GB at 16 bits and about 2 to 2.5 GB at 4 to 5 bits (approximate; Taheri et al. report 8.0 GB and 2.5 GB for Qwen3-4B at FP16 and Q4_K_M [18]). On our 16 GB M4 laptop, the 16-bit model would take half of memory before the KV cache and other apps, and phones have less. At 0.5 to 4B parameters, 4-bit storage is the default.

The Damage Is Real and Hidden

Jang et al. showed the masking effect on τ^2-bench for 9B to 35B models quantized with AWQ and served with vLLM on A100 GPUs. In the most affected of ten cells, 4-bit weights doubled the share of calls to nonexistent tools (19.5% to 38.3%), amplifying errors the 16-bit model already made, yet the score did not change significantly because the ten-error budget absorbed them [20]. At small scale the score can collapse: Qwen3-0.6B (no-think) falls from 62.3% to 16.8% on single-turn BFCL (V3 non-live) under Q4_K_M, while Qwen3-1.7B loses 2.3 points [18]. Whether and where hidden damage appears at 0.5 to 4B on llama.cpp and MLX, the runtimes these models ship on, is unmeasured. RQ1 measures it.

Three Repairs, No Evidence to Choose

Practitioners can constrain decoding with a grammar [35, 36], add bits [22, 27], or retrain the quantized model to imitate its full-precision version, as NVIDIA, Liquid AI, Multiverse Computing, and Google already do [2, 11, 12, 33]. No study compares the three, and none reports deployment cost. Retraining results use overall scores and lack a train-longer control. Layer-selective bits lost to uniform bits, but only on continuation scoring [22]. Step routing between a 16-bit and a 4-bit model was tested on ALFWorld and WebShop with both models in memory [21], never at a matched memory budget on tool-calling tasks with measured cost.

Who Benefits

- **On-device app developers:** a repair choice matched to their memory and latency budget.
- **Vendors shipping retrained quantized models:** step-level validation with the missing control.
- **Runtime maintainers (llama.cpp, mlx-lm):** per-format damage maps, grammar overhead, and reproducible failing cases.
- **Benchmark designers:** evidence of how error budgets hide damage.

Why It Is Publishable

We contribute (1) the first controlled head-to-head comparison of the three repair families, each with its control; (2) a step-level measurement method, released as an open testbed with a failure taxonomy and a pre-registered analysis; (3) a matched-memory test of step-selective against uniform and layer-selective bits, where a null result is as informative as a positive one; and (4) deployment-cost figures on real 4-bit code paths. This fits EuroMLSys, whose 2026 call listed hardware-efficient ML, compression, LLM optimization, agentic systems, and benchmarking in a six-page format [79]. EuroSys 2027, its past host, runs 19 to 23 April 2027 in Rabat [80].

Feasibility

All models and benchmarks are openly licensed. BFCL multi-turn scripts its user turns and costs nothing. Only τ^2-bench-verified needs a paid simulated user, capped at $300 and replayed free from cache. Retraining uses free Colab and Kaggle GPUs in resumable sessions of at most 12 hours [81, 82]. Measurement needs one M4 Pro laptop.

Funding Prospects

We expect modest, not guaranteed, support. NVIDIA's Academic Grant Program has an "AI Inference, Agents, and Systems Software" track for faculty-led proposals; it is closed now, and our advisor could apply when it reopens [83]. Hugging Face community GPU grants can host our public damage-map report [84]. Vendors shipping retrained quantized models have a direct stake in step-level validation, and we will share results with them.

Chapter 3. Project Requirements

Requirements

This is a combined development and research project. The development part is a measurement testbed built from components C1 to C11. The research part is a controlled study run on that testbed to answer RQ1 to RQ4. This section specifies the testbed: who uses it, what it must do (functional requirements ranked Essential, Desired, and Optional), and how well it must do it (non-functional requirements). The Baseline Approaches section that follows specifies the references, repairs, controls, metrics, and hypotheses the testbed must support. Every requirement has an ID, an acceptance criterion or numeric target, and a trace to a component and a research question.

Stakeholders and Use Cases

- **Researcher (team member):** declares experiment grids, runs episodes, labels failures, and analyzes results.
- **Runtime maintainer (llama.cpp, mlx-lm):** uses per-format damage reports and minimal failing episodes.
- **Model vendor that ships retrained quantized models (for example Liquid AI or NVIDIA):** uses step-level validation of its checkpoints against controls.
- **On-device app developer:** uses the repair comparison and the cost figures to pick a repair.
- **Benchmark designer:** uses evidence on error budgets, pass^k, and hidden failures.
- **Reviewer or artifact evaluator:** reproduces figures and tables from the released artifacts.

Table 5. User stories (priority E = Essential, D = Desired).

ID	As a...	I want...	So that...	Priority
US-1	researcher	to declare a model × format × repair × benchmark × seed grid in one config and run it with one command	every result traces to a config, a commit, and a model-file hash	E
US-2	researcher	every step of every episode logged and labeled with a failure type	I can locate damage by step type and conversation depth, not only count failed tasks	E
US-3	researcher	a hard cap on simulated-user spending and free replays from a cache	the study never exceeds $300 and development runs cost nothing	E
US-4	researcher	retraining runs that checkpoint and resume across free GPU sessions	a 12-hour session limit never costs more than 30 minutes of training	E
US-5	on-device app developer	each repair's failure reduction next to its memory, latency, token, and energy cost on a laptop	I can choose a repair that fits my device budget	E
US-6	model vendor	step-level failure counts for a retrained checkpoint against plain quantization and a train-longer control	I know whether retraining fixes the failures or only the score	E
US-7	reviewer	a one-command re-run that uses cached simulated-user replies	I can regenerate every figure without API spending	E
US-8	runtime maintainer	per-format damage maps with minimal failing episodes	I can prioritize fixes to formats, kernels, grammars, or chat templates	D
US-9	benchmark designer	success re-scored at smaller error budgets, plus pass^k	I can see how much damage the default metric hides	D

Functional Requirements

Essential requirements are the minimum needed to answer RQ1 to RQ4 on the core grid defined under Baseline Approaches: five models, four 4-bit formats on two runtimes, three repair families, two multi-turn benchmarks, and one easier single-turn benchmark. Desired requirements widen coverage and test robustness. Optional requirements are stretch goals. The Comp. column names the component (C1 to C11) that implements each requirement.

Table 6. Essential functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-E1	Load all 800 BFCL v4 multi-turn tasks, τ^2-bench-verified (airline 50, retail 114, telecom 114), and 1,000 BFCL v4 non-live single-turn tasks into one task schema (tools as JSON Schema, initial state, checker).	All 2,078 tasks load and pass schema validation; counts match the pinned source files.	C1	All

Table 6. Essential functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-E2	Score episodes with the official BFCL and τ²-bench checkers, and check the harness against a published result before the main study.	100% verdict agreement with the official evaluators on ≥ 200 replayed trajectories; the Qwen3-0.6B Q4_K_M single-turn drop [18] is re-run with our prompts and its direction reported.	C1, C2	All
FR-E3	Drive multi-turn episodes (model turn, tool execution, scripted or simulated user turn) under each benchmark's limits (τ² defaults: 200 steps, 10 errors), with fixed seeds.	Killing and restarting a 500-episode batch leaves no missing or duplicate episodes.	C2	All
FR-E4	Offer one backend interface over llama-server (GGUF) and mlx-lm (MLX) with the model's own chat template, identical sampling settings, and one tool-call parser. Baseline requests must not apply any grammar.	Identical prompt token IDs on both runtimes for 100 sampled prompts; server logs show no grammar on baseline requests.	C3	All
FR-E5	Build every variant from the same original checkpoint with pinned tools: 16-bit references, Q4_0, Q4_K_M, IQ4_XS with imatrix, MLX 4-bit, and all R2 variants.	SHA-256 and measured bits per weight logged per file; imatrix calibration text shares no 13-gram with any benchmark task.	C3, C11	RQ1, RQ2
FR-E6	Route simulated-user calls through a content-addressed cache with a hard spending cap and a ledger.	Replaying a finished episode makes 0 API calls; the first call past the cap is refused; ledger within 5% of the provider's invoice.	C4	RQ1–RQ3
FR-E7	Write one record per step: episode, step index, depth, step type, prompt hash, raw output, parsed call, validity flags, tool result, token counts, TTFT, decode time, config ID, seed.	100% of records schema-valid; logged steps equal the runner's step count; Parquet export reloads without loss.	C5	All
FR-E8	Label each failed step with one class: malformed call, hallucinated tool, schema-invalid arguments, wrong tool, wrong argument value, premature termination, policy violation, repetition loop, user-facing response error (τ²-bench), or other.	Cohen's $\kappa \geq 0.80$ [85] between classifier and adjudicated human labels on 300 stratified steps by M3; codebook frozen and registered at M2.	C6	RQ1, RQ3
FR-E9	R1: compile each task's tool schemas into a grammar that admits free text or one schema-valid call; apply it with GBNF in llama.cpp and with XGrammar as an mlx-lm logits processor [36], at 16 and 4 bits.	0 malformed or hallucinated-tool calls in 1,000 constrained steps per runtime; results reported per runtime.	C7	RQ2–RQ4
FR-E10	R2: build uniform, layer-selective, and step-selective variants on both runtimes, with the matched variants at 5.5 bits per weight of memory (see the repairs table under Baseline Approaches).	Realized memory within ±0.1 bits per weight of 5.5; every routing decision logged per step.	C7	RQ2–RQ4

Table 6. Essential functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-E11	R3: run QAD-KL (KL to the model's own 16-bit output through fake quantization that reproduces llama.cpp Q4_0 rounding), QAD-KL+CE (adds a next-token loss), and a train-longer control that repeats QAD-KL+CE with fake quantization off. Resume across sessions.	Both arms match on data order, tokens, steps, optimizer, and seed (checked from manifests); a resumed run tracks the uninterrupted loss within 1%.	C7, C8	RQ2, RQ3
FR-E12	Export retrained weights to GGUF Q4_0 and to MLX affine 4-bit with group size 32, which represents Q4_0 exactly (scale d, bias −8d), and verify both on the real 4-bit code path.	Exported 4-bit codes and scales equal the trained ones bit for bit; runtime and PyTorch top-1 tokens agree on ≥ 99% of 10,000 held-out positions.	C8	RQ2, RQ3
FR-E13	Measure peak memory, TTFT, decode tokens/s, extra tokens, and energy per task on the M4 Pro for every baseline and repair on both runtimes.	≥ 10 repetitions per configuration; coefficient of variation ≤ 5% for decode tokens/s; profiler overhead ≤ 2%.	C9	RQ4
FR-E14	Compute every metric in the Metrics table and produce the damage map (failure rate by step type × depth for each model × format) and the repair comparison matrix.	Metric code recovers known values on synthetic logs; all tables and figures regenerate from logs with one command.	C10	RQ1–RQ4
FR-E15	Apply the pre-registered inclusion rule and statistical tests exactly as registered.	OSF timestamp precedes the first main-study run; registered scripts run unchanged; deviations logged with reasons.	C10	RQ1–RQ4
FR-E16	Keep grids as declarative configs; give every run an immutable ID and a manifest (commit, config hash, model SHA-256, runtime versions, OS, hardware, seeds, simulator model).	Every result row traces to a manifest; any run re-executes from its manifest with one command.	C11	All

Table 7. Desired functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-D1	Extend RQ1 and the vendor-QAD audit to LFM2.5-2.6B, and R1 and R2 to Qwen3-4B and SmolLM3-3B.	Same criteria as the Essential grid.	C7, C11	RQ1, RQ2
FR-D2	Compare Liquid AI's released QAD Q4_0 GGUFs (1.2B-Instruct, 2.6B) with Liquid's own published Q4_0 files, not our rebuilds, so that conversion differences cannot explain the result.	Step-level metrics and flip rates with CIs for both sizes.	C3, C10	RQ3
FR-D3	Replicate the R1 and R2 contrasts on Q4_K_M.	Contrasts and repair ranking reported for both base formats.	C7	RQ2
FR-D4	Hand-audit 100 episodes where the 16-bit model succeeded and the 4-bit model failed.	≥ 90% of audited failures fall in a taxonomy class other than "other"; gaps feed a codebook revision before v1.0.	C6	RQ1
FR-D5	Re-run a stratified 10% of τ²-bench episodes with a stronger simulated user.	Kendall's τ between repair rankings under the two simulators reported; any ranking flip flagged in the paper.	C4, C10	RQ2
FR-D6	Check whether correctness runs on the 16 GB M4 can be pooled with M4 Pro runs.	Greedy outputs token-identical on ≥ 99% of 200 calibration steps; otherwise all reported runs stay on the M4 Pro.	C3, C10	All
FR-D7	Add the BFCL multi-turn long-context category (200 tasks) to the RQ1 grid.	Scored as in FR-E2.	C1	RQ1
FR-D8	Build a static step-log explorer that filters failures by model, format, repair, type, and depth and links to raw steps.	Opens offline; summary for the full log loads in < 5 s on the M4; any step reachable in ≤ 3 clicks.	C10	All

Table 8. Optional functional requirements.

ID	Requirement	Acceptance criterion	Comp.	RQ
FR-O1	Run the RQ1 grid in thinking mode for Qwen3 and SmolLM3.	Same metrics reported for both modes.	C2, C3	RQ1
FR-O2	Combine repairs (R1 with R2, R1 with R3) to test whether effects add.	Interaction contrast with a 95% CI.	C7	RQ2
FR-O3	Add formats: MLX mxfp4 and nvfp4 modes, the mlx-lm mixed_4_6 recipe, and a 3-bit stress point.	Built and scored as in FR-E5.	C3	RQ1
FR-O4	Train an in-domain QAD variant on the τ²-bench-verified train split and evaluate it only on the test split.	Zero overlap between training and evaluation task IDs.	C8	RQ2
FR-O5	Replicate a subset on llama.cpp's CUDA or CPU backend on Kaggle.	Flip rate relative to Metal reported.	C3	RQ1
FR-O6	Measure the core formats on an iPhone with llama.cpp or MLX Swift.	Memory and decode tokens/s for B1 to B4 on one device.	C9	RQ4

Non-Functional Requirements

Table 9. Non-functional requirements with measurable targets.

ID	Attribute	Requirement and target	Verified by
NFR-1	Reproducibility	Pin all versions (uv lockfile, llama.cpp build tag, mlx and mlx-lm versions), seeds, and configs; cache simulated-user replies. Target: a second member re-runs a 200-episode sample from manifests on the same machine and reproduces ≥ 99% of task verdicts.	Scripted reproduction test before M7
NFR-2	Determinism and replay fidelity	Fix server settings (slots, batch size, prompt caching) for all correctness runs, because llama.cpp does not guarantee bit-identical logits across batch sizes with prompt caching on [1, 67]. Target: per model, measure a noise floor (flip rate between two identical 16-bit runs); claim an effect only if it exceeds this floor under the registered test.	Noise-floor runs in Stage 1
NFR-3	Measurement accuracy	Cost runs on AC power with Low Power Mode off, no other user applications, thermal pressure nominal at block start (powermetrics thermal sampler and `pmset -g therm`), 3 warm-up runs discarded, configurations interleaved in random order, powermetrics sampling every 200 ms. Target: ≥ 10 repetitions; coefficient of variation ≤ 5% for TTFT and decode tokens/s; profiler overhead ≤ 2%. Energy compared only within the M4 Pro, because powermetrics reports estimates.	C9 self-test; with/without-profiler A/B run
NFR-4	Cost	API spending hard-capped at $300 in C4, released in tranches: $75 for skeleton, pilots, and 16-bit references; $150 for the main study; $75 reserve. Training only on free Colab and Kaggle GPUs; checkpoints at most every 30 minutes. Target: estimated spend ≤ $182 (volume table below); no interrupted session loses more than 30 minutes.	C4 ledger; C8 resume test
NFR-5	Scalability and throughput	Essential grid ≈ 71,000 episodes, ≈ 785 single-stream machine-hours (volume table below). Target: aggregate ≥ 120 episodes/hour on the M4 Pro with parallel server slots, measured in the pilot, so the grid fits about 600 hours of overnight and weekend runs. If the pilot misses the target, apply the registered reduction ladder: 3 trials on BFCL, then 200 BFCL tasks, then R2 on two models.	Pilot throughput report at M2

Table 9. Non-functional requirements with measurable targets.

ID	Attribute	Requirement and target	Verified by
NFR-6	Portability	Run on macOS on Apple silicon with llama.cpp (Metal) and mlx-lm; runner, classifier, and analysis also run on Linux with the llama.cpp CPU backend; Python 3.12. Target: switching runtime changes one config field; CI passes on macOS and Linux.	CI matrix
NFR-7	Extensibility	Repairs, benchmarks, and backends are plugins registered by name behind fixed interfaces. Target: a new repair needs one module and one config file and no edits to C2, C5, or C10; R1 to R3 are themselves built this way.	Code review; plugin test
NFR-8	Statistical validity	Register on OSF before the main study [68]; Holm correction within each RQ family [65]; 95% bootstrap CIs over tasks with 10,000 resamples [86]; exploratory analyses labeled as such. Target: ≥ 80% power to detect a 10-point success difference after Holm correction over 10 contrasts, which needs about 200 16-bit-solvable tasks per model at 15% discordant pairs [87].	Power analysis after Stage 1
NFR-9	Data management and licensing	Honor upstream licenses: Qwen3 and SmolLM3 Apache-2.0; LFM2.5 LFM Open License v1.0, which allows derivatives with the license and notices but no commercial use by entities with ≥ $10M annual revenue [88]; BFCL Apache-2.0; τ^2-bench and τ^2-bench-verified MIT. Logs hold only synthetic benchmark data; simulated-user text is released only if the provider's terms allow. Target: every released file carries its upstream license; logs (about 1 to 3 GB of Parquet) on Hugging Face Datasets with a Zenodo DOI.	License checklist at M8
NFR-10	Security and isolation	API keys only in environment variables and never logged; benchmark tools are in-process mocks with no real side effects; llama-server bound to localhost. Target: a secret scan of logs and repository finds 0 keys.	Secret scan before each release
NFR-11	Release quality	Testbed under Apache-2.0 with documentation, examples, ≥ 80% line coverage on C2, C5, C6, and C10, semantic versioning, and a checklist for the ACM Available, Functional, and Reusable badges [89]. Target: a clean machine regenerates all figures from released logs with one command in < 1 hour.	Clean-machine test before M8

The volume estimate behind NFR-4 and NFR-5 is below. Its inputs are planning assumptions that the pilot replaces with measurements. The task pool is 578 multi-turn tasks per model: 300 BFCL multi-turn tasks (100 each from base, missing-function, and missing-parameter, sampled with a fixed seed) and all 278 τ²-bench-verified tasks. We assume 35% of tasks are solvable at 16 bits (≈ 202 per model, 97 from τ²), 4 trials per task and configuration, and mean episode times on the M4 Pro of 30 s at 4 to 8 bits and 60 s at 16 bits. These times scale down from llama.cpp's published M4 Pro results for a 7B Q4_0 model, about 50 tokens/s decode and 360 to 440 tokens/s prefill [90]. For τ² we assume about 25,000 input and 600 output tokens of simulated-user traffic per episode, 70% of it served from the provider's prompt cache.

Table 10. Planning estimate of experiment volume and simulated-user traffic for the Essential grid.

Stage	Configurations	Tasks × trials	Episodes	τ² episodes
Stage 1: 16-bit screening	5 models × BF16 (llama.cpp)	578 × 4	11,560	5,560
Stage 2: RQ1 formats	5 models × 5 (MLX bf16, Q4_0, Q4_K_M, IQ4_XS, MLX 4-bit)	202 × 4	20,200	9,700
Stage 3: RQ2 and RQ3 repairs	3 core models × 16, plus 1 vendor QAD	202 × 4	39,592	16,684
Total			71,352	31,944

The 16 Stage 3 configurations per core model are R1 with and without quantization on each runtime (4); Q5_0, Q6_K, Q8_0, layer-selective, and two-model step-selective on llama.cpp and MLX 5-bit, layer-selective, and step-selective on mlx-lm (8); and QAD-KL, QAD-KL+CE, and the train-longer control at 16 bits and at Q4_0 (4). The total is about 22,900 16-bit and 48,500 lower-precision episodes, or about 785 machine-hours at one episode at a time. The uniform Q6_K and Q8_0 rungs run on BFCL only, which is why Stage 3 has fewer τ² episodes. Simulated-user traffic comes to about 800 million input and 19 million output tokens. At OpenAI's September 2026 list prices [91], this costs about $182 with GPT-4.1-mini ($32 for Stage 1, $151 for the main study) and about $46 with GPT-4.1-nano. With GPT-5.1, the simulator used on the τ²-bench-verified leaderboard [92], it would cost about $561, far above the cap. We therefore fix one mini- or nano-class simulator for every condition, which keeps paired comparisons fair, and check robustness with a stronger simulator on 10% of episodes (FR-D5, about $56 at GPT-5.1 prices, paid from the reserve). The tightest calendar window is R1 and R2 between milestones M3 and M4: about 280 single-stream machine-hours in two weeks. It needs parallel slots (NFR-5) or pooling with the M4 (FR-D6).

R3 training volume is set in the pilot. Our planning figure is about 50 million tokens per run, or 7 to 10 GPU-hours on Kaggle's two T4s at an assumed 1,500 to 2,000 tokens/s. That makes three runs per core model (two QAD variants and one train-longer

control), nine runs in total, and 65 to 90 GPU-hours, about one week of the three members' combined Kaggle quota, which is about 30 GPU-hours per member per week [93].

Requirements Quality (IEEE 830)

IEEE Std 830-1998, section 4.3, lists eight characteristics of a good software requirements specification [94]. The table shows how this chapter meets each one.

Table 11. How the requirements meet IEEE 830 section 4.3.

Section	Characteristic	How this chapter satisfies it
4.3.1	Correct	Each requirement traces to a user story or an RQ; facts about formats, task counts, limits, and licenses were checked against the source repositories and documentation on 27 September 2026.
4.3.2	Unambiguous	Formats, task counts, thresholds, and budgets are named exactly; every metric is defined once in the Metrics table; "matched" always names the quantity matched.
4.3.3	Complete	Covers all components C1 to C11, all four RQs, both runtimes, and both benchmark types; invalid model outputs are logged and classified rather than crashing the runner.
4.3.4	Consistent	One vocabulary (component, format, and repair names) is used across chapters; no two requirements set conflicting targets.
4.3.5	Ranked for importance and stability	Essential, Desired, and Optional tiers; values expected to change (simulator model, QAD token budget, throughput) are flagged as pilot-set and frozen at M2.
4.3.6	Verifiable	Every FR has an acceptance criterion and every NFR a numeric target and a verification method.
4.3.7	Modifiable	One requirement per table row with a stable ID and no duplicated statements, so a change touches one row.
4.3.8	Traceable	Backward to US-1 to US-9 and RQ1 to RQ4; forward to components C1 to C11 and to the tests in the QA chapter.

Baseline Approaches

Every condition is compared with two anchors: the 16-bit reference and the 4-bit baseline that the repair tries to fix. Each repair also has a control that isolates the mechanism it claims. All conditions share tasks, prompts, chat templates, tool-call parsing, sampling settings, seeds, and the simulated-user model, so paired per-task comparisons are valid.

Reference and Quantized Baselines

The reference for each model is its original released weights at 16 bits in each runtime: BF16 GGUF for llama.cpp and bf16 for mlx-lm. Qwen3 and SmolLM3 run in

non-thinking mode, and every model uses the sampling settings its model card recommends for the 4 sampled trials. One extra greedy pass per task supports the determinism and flip-rate analyses. The quantized baselines are the formats publishers actually ship. On Hugging Face on 27 September 2026, Qwen's official GGUF repository for Qwen3-4B offered Q4_K_M, Liquid AI's LFM2.5 repositories offered Q4_0, Q4_K_M, and a QAD Q4_0, and both publishers offered MLX 4-bit builds. We rebuild every file ourselves from the original checkpoint with pinned tools, so any difference comes from the format and not from a third party's conversion.

Table 12. Reference and default 4-bit baselines (bits per weight include block scales).

ID	Runtime	Format	Bits per weight	Why included
B0-L	llama.cpp	BF16 GGUF	16	Reference on the GGUF path
B0-M	mlx-lm	bf16	16	Reference on the MLX path
B1	llama.cpp	Q4_0	4.5	Simplest grid (32 weights share one FP16 scale); target of vendor QAD [2] and of our R3; base format for RQ2
B2	llama.cpp	Q4_K_M	≈ 4.9	K-quant mix; the format behind the published 0.6B collapse [18]
B3	llama.cpp	IQ4_XS with imatrix	4.25	Non-linear 4-bit guided by an importance matrix
B4	mlx-lm	4-bit affine, group 64	4.5	MLX default quantization mode

Q4_0 and MLX 4-bit are 4.5 bits per weight by construction (4-bit codes plus a 16-bit scale per 32 weights, or a 16-bit scale and bias per 64). The Q4_K_M and IQ4_XS figures come from llama.cpp's quantize tool, where Q4_K_M measured 4.89 bits per weight on Llama 3.1 8B and varies slightly by model [1]. The MLX defaults (affine mode, group size 64, 4 bits) come from the MLX documentation [9].

Repair Baselines and Their Controls

Each repair is compared with its own control at a stated matched quantity. RQ2 uses Q4_0 and MLX 4-bit as base formats because both grids can be reproduced exactly during retraining; FR-D3 repeats R1 and R2 on Q4_K_M.

Table 13. Repairs: variants, controls, and matched quantities.

Repair	Variants	Control	What the control isolates	Matched quantity
R1 Grammar	Grammar built from the task's tool schemas: free text or one schema-valid call. GBNF in llama.cpp; XGrammar logits processor in mlx-lm.	(a) Same 4-bit model without grammar; (b) grammar applied at 16 bits	(a) grammar effect under 4-bit; (b) grammar effect without quantization; (a) minus (b) is the interaction	Weights, prompt, sampling, seeds
R2a Uniform bits	Q5_0 (5.5), Q6_K (6.56), Q8_0 (8.5); MLX 5-bit (5.5)	Q4_0; MLX 4-bit	Dose response of added bits	None (a ladder)
R2b Layer-selective	4-bit everywhere except the most sensitive 25% of linear weights at 8 bits (llama-quantize `--tensor-type`; mlx-lm quant predicate)	Uniform 5.5 (Q5_0; MLX 5-bit)	Whether where the bits go matters	Memory: 5.5 bits per weight
R2c Step-selective	MLX: 4-bit base plus 8-bit copies of the most sensitive ≈ 12% of linear weights, switched on only for routed steps. llama.cpp: Q4_0 and Q8_0 models both resident, routed per step.	Uniform 5.5; R2b	Whether when the bits are spent matters	MLX memory: 5.5; the llama.cpp variant reports its true 13
R3 Retraining (QAD)	QAD-KL: KL to the model's own 16-bit outputs through Q4_0 fake quantization. QAD-KL+CE: adds a next-token loss on the training mix.	(a) Plain Q4_0 of the original; (b) train-longer: the QAD-KL+CE run with fake quantization off, then plain Q4_0; (c) vendor QAD Q4_0 (LFM2.5)	(a) total gain; (b) training under quantization versus extra training	Data, order, tokens, steps, optimizer, seed, final format

Grammar scope. R1 is grammar-constrained decoding [35, 36, 58]. It needs no retraining and no extra weight memory; its latency overhead is measured under RQ4. Input-side tool-schema compilation such as TSCG [3] is complementary and out of scope, because it changes the prompt rather than the decoder.

Layer sensitivity. Hu and Ramachandran showed that cheap proxies can misidentify where extra precision helps [22]. We therefore rank layers by a causal test: we quantize one layer at a time to 4 bits and measure the KL divergence from the 16-bit model on calibration conversations that are disjoint from both benchmarks. The same ranking feeds R2b and R2c, so the two differ only in when the extra bits are used.

Step routing. R2c uses two pre-registered policies. The first routes tool-call steps, detected from the first generated token. The second routes high-entropy steps, with the threshold set on pilot data so that about 25% of steps route. Li et al. instead train a router and keep a full 16-bit model resident [21]. Our MLX variant holds memory at the

uniform budget, which is the comparison that has not been made. Its KV cache is shared across switches, so switching adds no re-prefill; the cached keys and values from earlier steps were, however, computed at the other precision, a mixing effect we log and discuss rather than remove. The llama.cpp two-model variant does re-prefill, and we count those tokens as extra tokens in RQ4.

Why the train-longer control uses the KL+CE loss. With pure self-distillation, a 16-bit student starts at zero loss and zero gradient, so a train-longer arm would not move. We confirm this with a unit test rather than spend GPU time on it. The control is therefore run for QAD-KL+CE, where the next-token term can change a 16-bit model. This is the confound that published retraining results leave open.

Scope set by free GPUs. Full-parameter training needs about 16 bytes of model state per parameter [95]. Full-parameter QAD therefore fits a 16 GB T4 or P100 only for Qwen3-0.6B, and only with gradient checkpointing. For LFM2.5-1.2B-Instruct and Qwen3-1.7B we use a parameter-efficient variant: a low-rank update is added to the frozen weights before fake quantization, so the exported Q4_0 grid is still exact. We add 8-bit optimizer states or a split across Kaggle's two T4s if needed. Neither GPU supports BF16 natively, so training uses FP16 autocast with loss scaling. Each control uses the same parameterization as its QAD arm. On mlx-lm, both R3 arms and their plain-Q4_0 control run as MLX affine 4-bit with group size 32, which holds the same Q4_0 codes exactly (FR-E12). The training mix is public function-calling and general text, filtered to share no 13-gram with any benchmark task.

Models, Benchmarks, and Metrics

Table 14. Model grid (size is the published name; stored parameters are counted from the Hugging Face checkpoints on 27 September 2026).

Model	Size (stored parameters)	License	Role
Qwen3-0.6B [43]	0.6B (0.75B)	Apache-2.0	Core: RQ1 to RQ4; full-parameter QAD
Qwen3-1.7B [43]	1.7B (2.03B)	Apache-2.0	Core: RQ1 to RQ4; parameter-efficient QAD
LFM2.5-1.2B-Instruct [45]	1.2B (1.17B)	LFM Open License v1.0	Core: RQ1 to RQ4; vendor QAD Q4_0 available
Qwen3-4B [43]	4B (4.02B)	Apache-2.0	RQ1; R1 and R2 in FR-D1
SmolLM3-3B [46]	3B (3.08B)	Apache-2.0	RQ1; R1 and R2 in FR-D1
LFM2.5-2.6B [96]	2.6B (2.70B)	LFM Open License v1.0	Desired (FR-D1, FR-D2); vendor QAD Q4_0 available

The three core models are the ones that the free GPUs can retrain. They include Qwen3-0.6B, the model with the largest published damage. LFM2.5 also comes in 230M and 350M sizes. Both are below our 0.5B floor and are excluded.

Table 15. Benchmarks.

Benchmark	Tasks in the grid	User turns	Success criterion	API cost
BFCL v4 multi-turn [47]	300 (100 each: base, missing function, missing parameter)	Scripted in the dataset	State and response checkers pass on every turn	None
τ²-bench-verified [50, 92]	278 (airline 50, retail 114, telecom 114)	LLM-simulated user	Final database state and required outputs match (reward 1)	Simulated user
BFCL v4 non-live single-turn [47]	1,000 (simple Python 400, multiple 200, parallel 200, parallel multiple 200)	None	Abstract-syntax-tree match	None

τ²-bench-verified is Amazon AGI's corrected release of τ²-bench. It fixes tasks whose expected actions contradicted the domain policy or database [51, 92]. The single-turn set is the easier benchmark for the smallest models and the setting of the published 0.6B result. **Inclusion rule (pre-registered):** a task stays in a model's analysis set if the 16-bit reference succeeds in at least 1 of 4 trials. The same set is used for every configuration of that model, including retrained ones.

Table 16. Metrics.

Metric	Definition	Level	RQ
Task success	The benchmark's native binary verdict for one trial; success rate is the mean over tasks and trials	Trial	1–3
pass^k	Mean over tasks of C(c, k) / C(n, k), with n = 4 trials and c successes, for k = 1, 2, 4 [49]	Task	1–3
Step failures by type	Count of steps that C6 labels with class f, per episode and per 100 steps	Step	1, 3
Amplification ratio	Mean failures of class f per task in a configuration divided by the same quantity at 16 bits, on the inclusion set, with a bootstrap CI	Model × format × class	1, 3
Flip rate	Share of (task, seed index) pairs whose verdict differs from the 16-bit reference, reported separately for success-to-failure and failure-to-success, next to the 16-bit noise floor	Configuration	1–3
Gap closed	(m_repair − m_4bit) / (m_16bit − m_4bit) for metric m, computed separately for success and for failures	Repair	2, 3
Budget-limited success	τ² success re-scored offline as if the run stopped after b tool errors, for b = 10 (default) and b = 2	Trial	3

Table 16. Metrics.

Metric	Definition	Level	RQ
Peak memory	Maximum footprint of the inference process during a task (weights, KV cache, runtime buffers), in GB	Task	4
TTFT	Time from request to first generated token, per step, in ms (median per task)	Step	4
Decode throughput	Generated tokens divided by decode time, per step, in tokens/s	Step	4
Extra tokens	Tokens prefilled plus generated per task, minus the base 4-bit configuration on the same task, including re-prefill after precision switches	Task	4
Energy per task	Integral of CPU, GPU, and ANE power from powermetrics (200 ms samples) over the task, minus a same-session idle baseline, in J	Task	4
Training cost	GPU-hours and training tokens to produce a retrained checkpoint, with GPU type	Run	4

Step type is the kind of output a step produces (tool call, text reply to the user, or end of conversation). Depth is the step's index within the episode, grouped into pre-registered bins. RQ4 cost is measured in two ways. Fixed-workload runs use identical prefixes and output lengths, so they isolate runtime cost. Natural episodes include behavioral effects such as loops or longer outputs.

Hypotheses

Every hypothesis, test, exclusion rule, and threshold below is registered on OSF before the main study (milestone M2). The unit of analysis is the task. Trials are averaged within a task before paired tests. Holm correction is applied within each RQ family [65], and every effect is reported with a 95% bootstrap CI over tasks [86].

- **H1 (RQ1).** 4-bit formats raise per-step failures above the 16-bit reference (amplification ratio > 1), and the excess concentrates in tool-call steps and at greater depth.
    - Test: for each model × format, a paired permutation test (10,000 permutations) on per-task failure counts. Concentration is tested with a Poisson GEE of failures on precision × step type and precision × depth, clustered by task.
- **H2 (RQ2).** Each repair reduces the excess failures relative to its control. We predict that R1 removes format failures (malformed call, hallucinated tool, schema-invalid arguments) but not semantic ones (wrong tool, wrong value, policy violation), and that QAD beats the train-longer control. For step-selective versus matched uniform bits we make no directional prediction.

    - Test: paired permutation tests on task-level success rates and failure counts; McNemar's exact test on seed-matched trials as a secondary test [97]. Gap closed is reported with bootstrap CIs. A null result for R2c is supported only if a two one-sided tests equivalence procedure with a ±3-point margin rejects [64].
- **H3 (RQ3).** Repairs close more of the score gap than of the failure gap, and their score gains shrink when τ^2 is re-scored with an error budget of 2.
    - Test: a paired bootstrap over tasks of the difference between gap closed on success and gap closed on failures; the 95% CI must exclude 0.
- **H4 (RQ4).** R1 adds no weight memory, and we predict it adds ≤ 5% to median per-step latency. XGrammar's authors report preprocessing and per-token overhead in llama.cpp's grammar engine [36], so this prediction may fail on llama.cpp. Uniform bits cost weight memory in proportion to bits (Q8_0 ≈ 1.9 × Q4_0). The llama.cpp two-model R2c costs about 2.9 × Q4_0 weight memory plus re-prefill tokens, while the MLX variant stays at the 5.5-bit budget. R3 adds no inference cost relative to Q4_0; its cost is training GPU-hours.
    - Test: 10 or more interleaved repetitions; medians with bootstrap CIs; equivalence tests with a ±5% margin for every "no overhead" claim; a Pareto frontier of failure reduction against each cost.
- **Mixed-effects sensitivity analysis (optional).** A logistic model of success on configuration with random intercepts for task, fitted in statsmodels, checks that task-level averaging does not change any conclusion.

Chapter 4. Dependencies and Deliverables

Dependencies

We list only showstoppers: dependencies whose failure would halt the project or remove a research question. Ordinary dependencies, such as Python packages, are pinned in the lockfile and are not listed. Each row names a fallback that keeps the study alive, even if in reduced form, and one owner who watches it. Likelihood is our estimate for the project period.

Table 17. Showstopper dependencies.

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
DEP-1	Open weights for Qwen3 (0.6B, 1.7B, 4B), LFM2.5 (1.2B-Instruct, 2.6B), and SmolLM3-3B, under licenses that allow modification and release: Apache-2.0 for Qwen3 and SmolLM3, LFM Open License v1.0 for LFM2.5 [88]	Without the original 16-bit weights we cannot build variants, retrain, or release derivatives	Low	Download and hash every checkpoint at project start and pin the revision. Release LFM2.5 derivatives only with its license and notices; that license bars commercial use by entities with ≥ $10M annual revenue. If a model becomes unusable, substitute another open 0.5 to 4B family and record the change in the pre-registration log.	Yashashav
DEP-2	BFCL v4 multi-turn (Apache-2.0) and τ²-bench-verified (MIT) remain available and runnable against a local OpenAI-compatible endpoint [47, 92]	They are the tasks and the official checkers; without them there is no study	Low	Fork and pin both repositories at a fixed commit on day one. If τ²-bench-verified breaks, fall back to the original τ²-bench (MIT) [50] and report the fixes we lose.	Vritika

Table 17. Showstopper dependencies.

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
DEP-3	Simulated-user API access for τ^2-bench within the $300 cap	Only τ^2 needs a paid simulated user; if the cap is hit, τ^2 results stop	Medium	Planning estimate is about $182 with GPT-4.1-mini and $46 with GPT-4.1-nano (see the requirements chapter). C4 enforces the cap, replays from cache for free, and releases money in tranches ($75, $150, $75 reserve). If costs overrun, cut τ^2 trials first. BFCL, which needs no API, can carry RQ1 to RQ3 alone.	Vritika
DEP-4	Free GPUs with enough memory for QAD: Colab and Kaggle sessions of at most 12 hours, about 30 GPU-hours per week on Kaggle, 16 GB T4 or P100, availability not guaranteed [81, 82, 93]	R3 cannot run without training. Full-parameter training needs about 16 bytes per parameter [95], so it fits a 16 GB GPU only for Qwen3-0.6B, and only with gradient checkpointing; LFM2.5-1.2B would need about 19 GB	Medium	Full-parameter QAD only for Qwen3-0.6B. For 1.2B and 1.7B, a low-rank update applied before fake quantization, 8-bit optimizer states, or a split across Kaggle's two T4s. FP16 autocast with loss scaling, because neither GPU has native BF16. Checkpoint to the Hugging Face Hub at most every 30 minutes and resume on either platform or on any member's account. Fallback: Liquid AI's released QAD Q4_0 checkpoints keep RQ3 alive for LFM2.5 [2].	Yashashav
DEP-5	Access to the Apple M4 Pro MacBook for about 600 to 785 machine-hours of runs and for exclusive, undisturbed cost-measurement sessions owner and unified-memory size of the M4 Pro MacBook, and that it is available for overnight runs	RQ4 is defined on the real 4-bit code path of this machine, and all reported correctness runs use it unless the pooling check passes	Medium	Book fixed overnight and weekend windows. Use parallel server slots to meet the throughput target. Pool correctness runs with the 16 GB M4 only if FR-D6 passes. If the M4 Pro is lost, measure cost on the M4 and relabel every claim to that device.	Ekant

Table 17. Showstopper dependencies.

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
DEP-6	llama.cpp and mlx-lm support the chosen models, formats, grammars, and tool-call templates, and stay stable during the main study [1, 9]	The runtimes are the object of study; a breaking change mid-study invalidates comparisons	Low for support, Medium for churn	Official or community GGUF and MLX builds exist for every chosen model. Pin one llama.cpp build tag and one mlx and mlx-lm version from M2 to M8, and run conformance tests in CI. llama.cpp uses GBNF; mlx-lm has no grammar option, so R1 on MLX uses XGrammar as a logits processor [36], and R1 is reported per runtime. Spike S3 checks that llama-server's native tool-call path applies no grammar to baseline requests.	Ekant
DEP-7	Retrained weights export to GGUF and MLX without leaving the Q4_0 grid	If export re-quantizes, the retraining gain can vanish and R3 would be measured on the wrong code path	Medium	Fake quantization reproduces Q4_0 rounding exactly (32-weight blocks, FP16 scale). Write the trained codes and scales directly rather than re-running quantization. Map Q4_0 exactly to MLX affine group-32 (scale d, bias −8d). Verify bit-exact codes and top-1 agreement ≥ 99% (FR-E12).	Yashashav
DEP-8	The 16-bit models solve enough tasks to analyze; the abstract flags that the smallest models may fail hard benchmarks even at full precision	The inclusion rule keeps only tasks the 16-bit model solves; too few tasks means no statistical power	High for Qwen3-0.6B on multi-turn; Low to Medium for the others	Include the easier BFCL single-turn set. Run the power analysis after Stage 1: about 200 solvable tasks give 80% power for a 10-point difference after Holm correction, and about 115 without it [87]. Models under 120 solvable tasks move to single-turn-only analysis, as pre-registered.	Vritika

Table 17. Showstopper dependencies.

ID	Dependency	Why it is a showstopper	Likelihood	Mitigation or fallback	Owner
DEP-9	Superuser access for powermetrics, macOS's documented tool for CPU, GPU, and ANE power	powermetrics refuses to run without superuser rights, so there would be no energy figures for RQ4	Low	The machine owner grants sudo for one scripted measurement command. Because powermetrics reports estimates, energy is compared only within the M4 Pro. If sudo is unavailable, we use a sudo-free reader of the same power counters only after cross-checking it against powermetrics on a machine where sudo works; otherwise we report energy as not measured. Memory, latency, and token costs stand either way.	Ekant
DEP-10	Nobody else publishes the same head-to-head comparison first; the abstract flags that this area moves fast	Loss of novelty for the paper	Medium	As the abstract commits: if a single repair is published first, we cite it and keep the comparison. The controls, the matched-memory step-selective test, and the measured costs remain new. The OSF registration (M2) timestamps our design. Each member tracks new work in one area every week.	All

Deliverables

These deliverables are project artifacts, not course assignments. Target dates follow the milestones in the schedule chapter: M1 walking skeleton (2026-10-23), M2 16-bit references and frozen OSF pre-registration (2026-11-06), M3 damage map v1 (2026-12-04), M4 R1 and R2 evaluated with repository v0.1 (2026-12-18), M5 QAD and train-longer control (2027-01-29), M6 cost profiling (2027-02-05), M7 workshop paper (2027-02-19), and M8 v1.0 release and final results (2027-05-14). M1 to M4 fall in CMPE 295A and M5 to M8 in CMPE 295B.

Table 18. Core project deliverables.

ID	Deliverable	Description	Form	Target	Owner
DL-1	Open-source testbed	C1 to C11: replays BFCL multi-turn and τ²-bench-verified against llama.cpp and mlx-lm, logs every step, and runs the repairs and profiler	GitHub repository, Apache-2.0, tagged releases, documentation, CI	v0.1 at M4; v1.0 with DOI at M8	Ekant (core, CI); Yashashav (v1.0 release, DOI)
DL-2	Failure taxonomy and classifier	Codebook of step-failure classes, rule-based classifier, 300 human-labeled steps, and Cohen's κ report	Code, PDF codebook, and labeled sample, all public	Codebook frozen at M2; κ report at M3	Vritika
DL-3	Pre-registered analysis plan	Hypotheses H1 to H4, metrics, inclusion rule, tests, corrections, power analysis, and analysis code	Frozen OSF registration	M2	Vritika
DL-4	Damage map (RQ1)	Failure rates by step type and depth for each model and 4-bit format, with amplification ratios and CIs	Figures, tables, and CSV in a report	v1 at M3; final at M8	Vritika
DL-5	Repair comparison matrix (RQ2, RQ3)	Every repair against its control: success, pass^k, failure counts, gap closed, and budget-limited	Tables and figures	R1 and R2 at M4; R3 at M5; with costs at M6	Ekant (R1), Yashashav (R2, R3), Vritika (statistics)
DL-6	QAD pipeline	Resumable QAD and train-longer training on Colab and Kaggle, exact Q4_0 export, and code-path verification	Notebooks and scripts in the repository	M5	Yashashav
DL-7	Cost measurements (RQ4)	Peak memory, TTFT, decode tokens/s, extra tokens, and energy per task for every baseline and repair on the M4 Pro, plus training GPU-hours	Dataset, figures, and measurement protocol	M6	Ekant
DL-8	Step-log dataset	Every main-study episode, one record per step, with run manifests	Parquet on Hugging Face Datasets with a dataset card; Zenodo DOI	v1 at M7; final at M8	Vritika
DL-9	Workshop paper	Six-page paper on RQ1 to RQ4 for EuroMLSys 2027	PDF	M7 (2027-02-19) EuroMLSys 2027 deadline	All

Additional Deliverables

These go beyond the minimum. Each has a check that anyone can apply.

Table 19. Additional deliverables and their acceptance checks.

ID	Deliverable	Acceptance check	Target	Owner
AD-1	Retrained checkpoints on Hugging Face: QAD-KL, QAD-KL+CE, and train-longer controls for the three core models, in GGUF Q4_0 and MLX	Each file downloads and loads in its runtime; its model card states base model, format, repair, control, license, and step-level failure rates with CIs [98]	M8	Yashashav
AD-2	One-command reproducibility package	On a clean machine, one command regenerates every paper figure and table from released logs in under 1 hour with no API or GPU cost; another re-runs any 200-episode sample from the cache; ACM badge checklist included [89]	M8	Yashashav (environments, DOI); Ekant (CI)
AD-3	Public damage-map report and leaderboard	Static site built only from released data; every number links to its config and run IDs; new models can be submitted as a config plus logs, validated by CI	M8	Vritika
AD-4	Practitioner decision guide ("which repair when")	Two to four pages and a flowchart keyed to memory budget, latency budget, and dominant failure class; every recommendation cites a cell of the comparison matrix	M8	Yashashav (lead), all
AD-5	Standalone tool-schema-to-grammar compiler	pip-installable; emits GBNF for llama.cpp and XGrammar grammars for mlx-lm; tests compile every tool schema in BFCL and τ^2-bench-verified	M4	Ekant
AD-6	Upstream bug reports and fixes	Every confirmed bug in llama.cpp or mlx-lm (grammar, tool-call parsing, quantization, export) is reported upstream with a minimal reproducer within two weeks of confirmation; links kept in the README	Ongoing to M8	Ekant (runtimes, grammar); Yashashav (quantization, export)
AD-7	arXiv preprints	Workshop version posted at submission; extended version with all RQ3 and RQ4 results and replications by the end of CMPE 295B	M7 and M8	All
AD-8	Expo poster and live demo	On the M4 Pro, 16-bit, 4-bit, and repaired agents run side by side on the same τ^2 task with live failure labels and cost meters, fully offline from the cache	CMPE 295B Expo	Ekant (demo), all (poster)
AD-9	Hypothesis outcome table	Each registered hypothesis listed with its result, every deviation from the plan, and exploratory findings labeled as such; published with the paper and on OSF	M7	Vritika

Chapter 5. Project Architecture

The testbed is an experiment pipeline built around a pluggable inference core. The figure shows all eleven components (C1 to C11) with their function, technology, and data flows. The same code runs on Ekant's 16 GB M4 for development, but every reported number comes from the Apple M4 Pro.

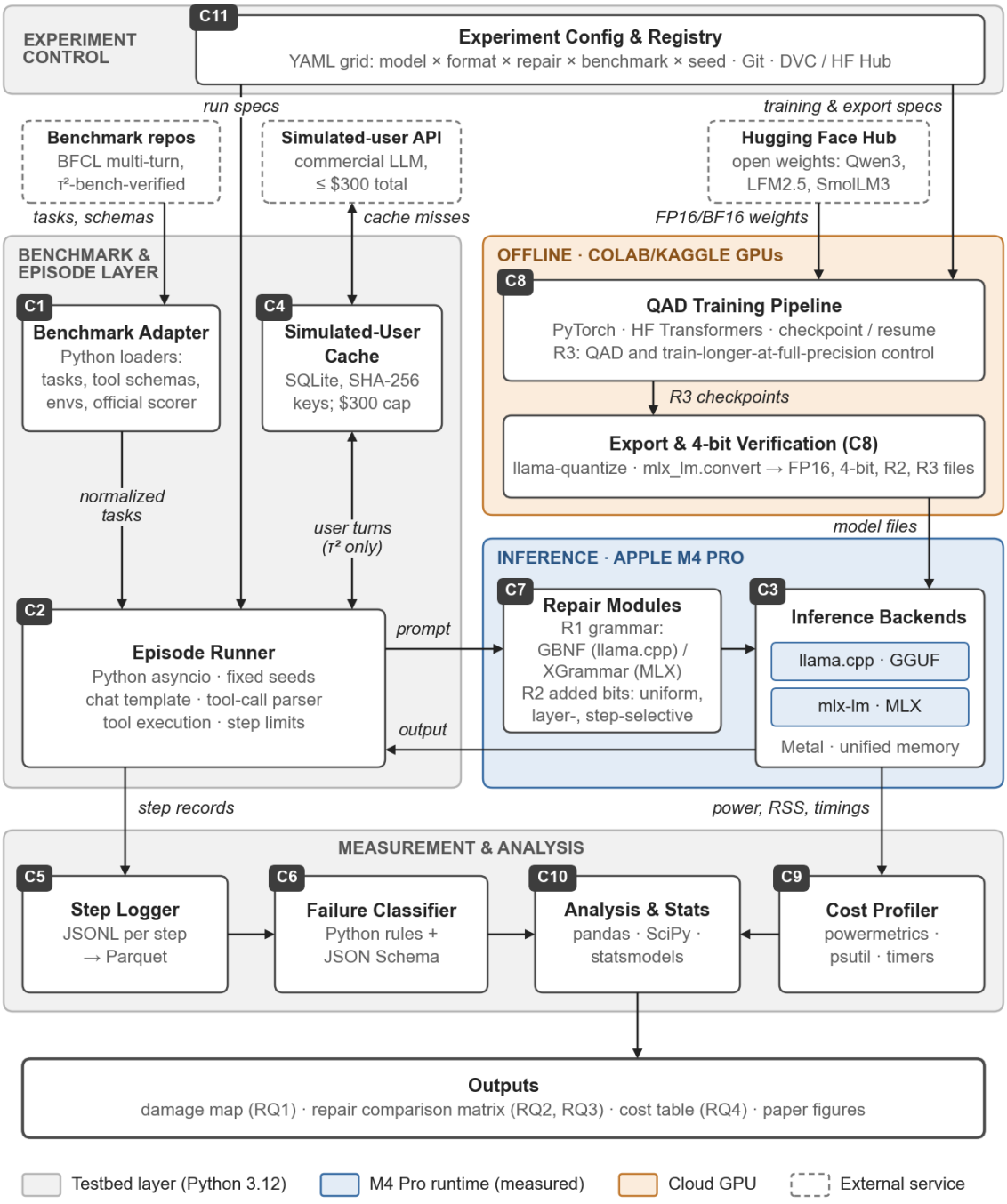


Figure 1. Testbed architecture. Components C1 to C11 are grouped by layer; each box gives the function (bold) and the technology (gray), and arrows show data flow. The blue zone is the measured on-device code path, and the orange zone runs on free cloud GPUs.

Architectural Style

The architecture combines three styles, each chosen for a need of a controlled measurement study.

- **Batch-sequential pipeline (pipe and filter).** Configs become episodes, episodes become step logs, logs become failure labels, and labels become statistics. Each stage persists its output before the next stage reads it, so any stage can rerun alone. Revising the failure codebook relabels frozen logs without rerunning a model, and re-analysis costs no inference or API spending.
- **Layered plugin architecture.** The Episode Runner (C2) depends only on the benchmark (C1), backend (C3), and repair (C7) interfaces. Implementations are registered by name, so changing one experimental factor changes one config field (NFR-7), which is what a paired comparison needs.
- **Client-server at the inference boundary, plus an offline training pipeline.** Each backend runs in its own local process that C2 calls over localhost. This separates the model's memory and energy from the harness's, and a backend crash cannot corrupt the logs. Training runs on other hardware and shares only versioned model files with the rest of the system.

One Agent Step, End to End

1. C11 expands the config into run specs. C2 starts an episode with a fixed seed, and C1 supplies the task, its tool schemas, and a mock tool environment.
2. C2 renders the conversation with the model's own chat template. The repair pre-hook (C7) attaches a grammar (R1) or picks the model copy for this step (R2).
3. C3 sends the request to llama-server or to the mlx-lm worker. The Cost Profiler (C9) marks the start and end and samples power and memory in between. The backend returns text, time to first token (TTFT), and token counts.
4. C2 parses any tool call, and the Failure Classifier's (C6) validators flag whether it parsed, whether the tool exists, and whether the arguments satisfy the schema.
5. C1's environment executes a tool call and returns the result. A message to the user gets the next user turn, scripted for BFCL or served by the Simulated-User Cache (C4) for τ^2-bench.
6. The Step Logger (C5) appends the step record to JSONL. At the end of the episode, C1's official checker records the verdict.
7. After the run, C5 writes Parquet, C6 assigns failure types using whole-episode context, and Analysis & Stats (C10) joins the C9 samples to the steps and runs the pre-registered tests.

Layers: Function and Technology

Experiment Control

The Experiment Config & Registry (C11) declares grids in YAML (Hydra-style) and gives every run an immutable ID and a manifest. The manifest records the commit, lockfile hash, model SHA-256, runtime versions, macOS build, and simulator model

(FR-E16). Configs live in Git; large artifacts are versioned with DVC or on the Hugging Face Hub.

Benchmark and Episode Layer

C1 wraps BFCL multi-turn and single-turn [47] and τ²-bench-verified [50, 92] behind one task schema: tools as JSON Schema, initial state, and the official checker. C2 is a Python 3.12 asyncio loop that enforces each benchmark's step and error limits and resumes a killed batch without duplicate episodes. C4 sits between τ²-bench's user simulator and the provider. It keys each request by a SHA-256 hash of the simulator model, sampling settings, seed, and conversation prefix, stores replies in SQLite, keeps a spending ledger, and refuses the first call past the $300 cap. BFCL's user turns are scripted, so BFCL never calls C4.

Inference Layer on the Apple M4 Pro

C3 exposes one `generate(messages, tools, constraint)` call over llama.cpp [1] and mlx-lm [9, 71] on the Metal GPU. References are BF16 GGUF and MLX bf16; the 4-bit baselines are Q4_0, Q4_K_M, IQ4_XS, and MLX 4-bit. C3 renders the chat template itself, so both runtimes receive identical prompt tokens (FR-E4). C7 plugs into C3 through the pre-hook. R1 compiles the task's tool schemas into a grammar that admits free text or one schema-valid call, applied as GBNF in llama.cpp and through XGrammar as an mlx-lm logits processor [36]. The engines differ, and XGrammar's authors report per-token overhead for llama.cpp's grammar engine [36], so R1 is reported per runtime and its latency is measured, not assumed. R2 supplies higher-precision weight files (uniform and layer-selective) or chooses per step which copy serves (step-selective). R3 checkpoints arrive as ordinary model variants.

Offline Training and Export

The QAD Training Pipeline (C8) runs PyTorch and Hugging Face Transformers on free Colab and Kaggle GPUs, checkpointing at least every 30 minutes. It trains the QAD arms [12, 33] and the train-longer control on identical data, order, and steps. At about 16 bytes of training state per parameter [95], full-parameter QAD fits a 16 GB T4 or P100 only for Qwen3-0.6B with gradient checkpointing. Larger core models use low-rank updates applied before fake quantization, and all training uses FP16 autocast because neither GPU supports BF16. The export step (`llama-quantize`, `mlx_lm.convert`) writes GGUF Q4_0 and MLX 4-bit files and verifies them on the real 4-bit code path (Chapter 7). The same path builds the baselines and R2 files, and vendor QAD checkpoints [2] enter as ordinary variants.

Measurement and Analysis

C5 appends JSONL during runs and writes typed Parquet afterward. C6 combines JSON Schema validation with rules to assign the failure taxonomy of Chapter 6. C9 reads llama-server timings and mlx-lm statistics, samples process memory with psutil, and

samples CPU, GPU, and ANE power with powermetrics every 200 ms. C10 uses pandas, SciPy, statsmodels, and matplotlib to produce the damage map, the repair comparison matrix, and the paper figures.

Key Decisions and Trade-offs

Table 20. Architectural decisions and the alternatives we rejected.

Decision	Choice	Rejected alternative	Reason
Runtimes	llama.cpp and mlx-lm	vLLM	Models ship as GGUF and MLX files; vLLM targets batched datacenter serving [99], and its Apple-silicon GPU support is a community plugin that executes models with MLX [100]
Backend process	llama-server over HTTP; an mlx-lm worker process of our own	In-process bindings by default	Separates model memory and energy from the harness, contains crashes, and matches deployment; llama-cpp-python is used only when we need logits in-process
Prompt path	Our own template rendering to the raw completion interface	Server chat endpoint	Identical token IDs on both runtimes; no implicit tool-call grammar can reach a baseline request
Simulated user	Content-addressed cache with a ledger	Live API calls	Replays and re-analysis are free; identical prefixes get identical user turns in every condition; the cap is enforced in code
Logs	JSONL during runs, Parquet after	A database server	Crash-safe appends and human-readable lines; columnar, typed files for analysis and release
Labeling	Validity flags online; failure types offline	Online-only labels	Some types need whole-episode context; the codebook can change without rerunning models

Step-selective precision (R2c). Li et al. route each agent step between a BF16 and a 4-bit model and keep both resident [21]. Our llama.cpp variant uses the same two-copy design with Q4_0 and Q8_0. Weights then occupy about 4.5 + 8.5 = 13 bits per parameter, 2.9 times Q4_0. Each copy keeps its own KV cache, so with prompt caching on, a switch re-prefills every token added since that copy last ran. C9 charges the second copy's memory and those tokens to the repair. Our MLX variant keeps a 4-bit base plus 8-bit copies of the most sensitive ≈ 12% of linear weights and swaps only those layers at step boundaries. It stays at the 5.5-bit budget of the uniform and layer-selective controls and shares one KV cache, at the price that cached keys and values from earlier steps come from the other precision. Any-precision weights [25] and per-token precision switching [27] would avoid the duplicate weights, but both rely on their own kernels rather than the llama.cpp or mlx-lm code paths we measure, so we treat them as related work.

Reproducibility and Measurement Validity

The architecture enforces validity by construction:

- **Traceability.** Run IDs are hashes of the config cell and seed, so rerunning a finished run is a no-op and every result row resolves to a manifest.
- **Pairing.** All conditions share tasks, prompts, parser, seeds, and simulator model. The cache makes user turns identical whenever conversation prefixes match.
- **Pinned server settings.** Slots, batch size, and prompt caching are fixed for all correctness runs, with caching off because llama.cpp does not guarantee bit-identical logits across batch sizes [1] (NFR-2). Cost runs are separate single-stream blocks with warm-up and randomized interleaving (NFR-3).
- **Clean baselines.** Baseline requests carry no grammar, as the server logs confirm, and a repair acts only through the C7 hook. Two conditions therefore differ by exactly the repair.
- **Registered analysis.** C10 reads only frozen logs, and its tests are registered on OSF before the main study [68].

Table 21. Components, their function and technology, data flow, and owners.

Component	Function	Technology	Inputs → Outputs	Owner
C1 Benchmark Adapter	Load tasks, tools, envs; official scoring	Python; BFCL, τ²-bench repos	Repos → tasks, envs, verdicts	Vritika
C2 Episode Runner	Drive model, tool, and user turns	Python 3.12, asyncio	Run specs, tasks → steps, episodes	Ekant
C3 Inference Backends	One generate call over two runtimes	llama-server, mlx-lm, Metal	Prompt, constraint → text, timings	Ekant
C4 Simulated-User Cache	Cached user turns; $300 cap	SQLite, SHA-256, LiteLLM	Prefix → user turn, ledger	Vritika
C5 Step Logger	One record per step	JSONL, Parquet (pyarrow)	Step events → step table	Ekant
C6 Failure Classifier	Validity flags; failure types	Python rules, JSON Schema	Steps, references → labels	Vritika
C7 Repair Modules	R1 grammar, R2 bits, R3 variants	GBNF, XGrammar, llama-quantize, mlx-lm	Request, context → constraint or copy	Ekant (R1); Yashashav (R2, R3)
C8 QAD Training Pipeline	QAD, control, export, verification	PyTorch, HF Transformers, llama.cpp tools	FP weights, data → GGUF, MLX files	Yashashav
C9 Cost Profiler	Memory, latency, tokens, energy	powermetrics, psutil, runtime timers	Markers, samples → cost table	Ekant
C10 Analysis & Stats	Registered tests, damage map, matrix	pandas, SciPy, statsmodels, matplotlib	Labels, costs → figures, tables	Vritika
C11 Experiment Config & Registry	Grids, run IDs, manifests	YAML (Hydra-style), Git, DVC or HF Hub	Config → run specs, manifests	Yashashav

Chapter 6. Project Design

This chapter is a preliminary design, written before implementation; we will refine it for Workbook Assignment 2.

Class Design

The class diagram shows the main abstractions of the testbed. The component ID in each class header (C1 to C11) links it to the architecture in Chapter 5. The testbed is written in Python 3.12: abstract classes are `abc.ABC` subclasses, and records such as Step and Episode are frozen dataclasses.

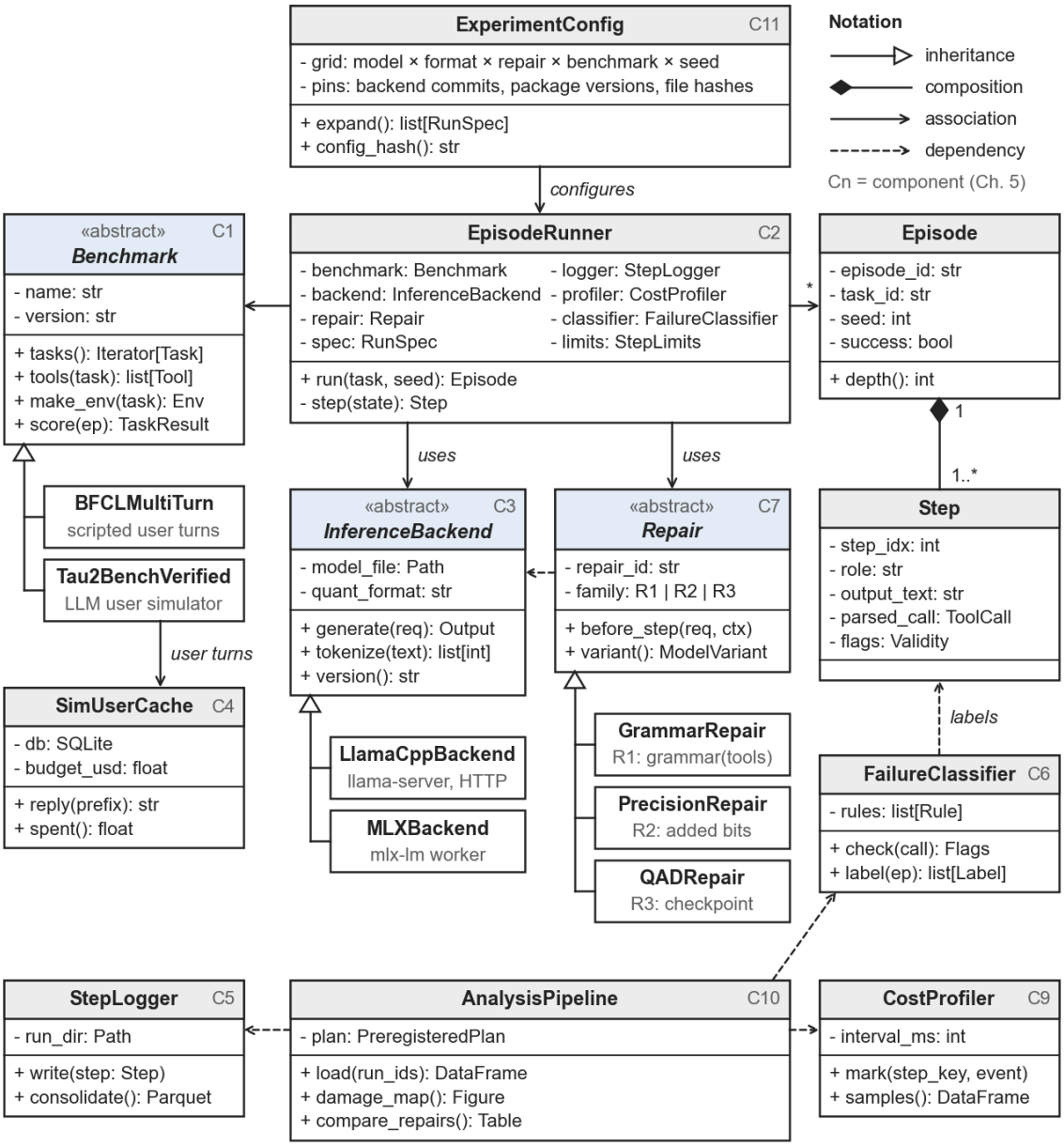


Figure 2. UML class diagram of the testbed's core abstractions. Italic names in blue headers are abstract classes; the component ID appears at the right of each header.

The design uses four patterns from the standard catalog [101]:

- **Strategy (backends and repairs).** EpisodeRunner holds an InferenceBackend and a Repair through their abstract types and never branches on the concrete class. A plugin registry maps config names to classes, so a new backend or repair needs one module and one config file and no change to C2, C5, or C10 (NFR-7). The request object passed to `generate` carries the messages, tools, and constraint of the `generate(messages, tools, constraint)` call in Chapter 5. LlamaCppBackend is an HTTP client for llama-server. It sends the rendered

prompt to the raw completion interface and passes R1 grammars in the request's grammar field. MLXBackend talks to a worker process that wraps mlx-lm's generation loop and accepts logits processors, which is how XGrammar is attached for R1.

- **Adapter (benchmarks).** BFCLMultiTurn and Tau2BenchVerified wrap the official repositories' task formats, environments, and checkers behind the Benchmark interface; a third adapter covers BFCL single-turn. The runner never imports benchmark code, and scoring always calls the unmodified official checker.
- **Proxy (simulated-user cache).** SimUserCache exposes the same completion call as the LLM client that τ²-bench's user simulator already uses. The simulator does not know it is cached. The proxy forwards only misses, and it also enforces the budget, so it is both a caching and a protection proxy.
- **Chain of responsibility (failure rules).** FailureClassifier evaluates its rules in a fixed precedence order, and the first match labels the step (see Failure Taxonomy Design).

Table 22. Interface contracts that the controlled comparison depends on.

Interface	Key methods	Contract
Benchmark	`tools`, `make_env`, `score`	Fresh environment per episode; `score` calls the official checker unchanged
InferenceBackend	`generate`, `tokenize`	Same prompt tokens, seed, and settings on both runtimes; returns text, token counts, TTFT, and decode time; raises a typed error on crash or timeout
Repair	`before_step`, `variant`	May add a constraint or choose a model copy; never changes prompt, sampling, or seed; logs every decision
SimUserCache	`reply`, `spent`	Same key returns the same reply; the first call past the cap raises a budget error
StepLogger	`write`, `consolidate`	Append-only, one line per step; consolidation is lossless

Sequence Design

The sequence diagram shows one agent step. The runner first calls the repair's pre-hook, which returns a grammar (R1), a model copy (R2 step-selective), or nothing. The runner then brackets the generation with Cost Profiler markers. While the backend generates, the profiler samples memory and power every 200 ms (the `loop` fragment). The validity flags returned by the Failure Classifier are recorded for analysis only. Control flow follows each benchmark harness's own rules, so the testbed observes failures without changing how episodes unfold. The `alt` fragment separates tool calls, which the benchmark environment executes, from messages to the user. On τ²-bench, the

adapter's user simulator answers those messages through the Simulated-User Cache, which calls the LLM API only on a miss (the `opt` fragment). For brevity, the diagram draws the simulator and its cache as one lifeline. On BFCL multi-turn, the next scripted user turn comes from the adapter instead.

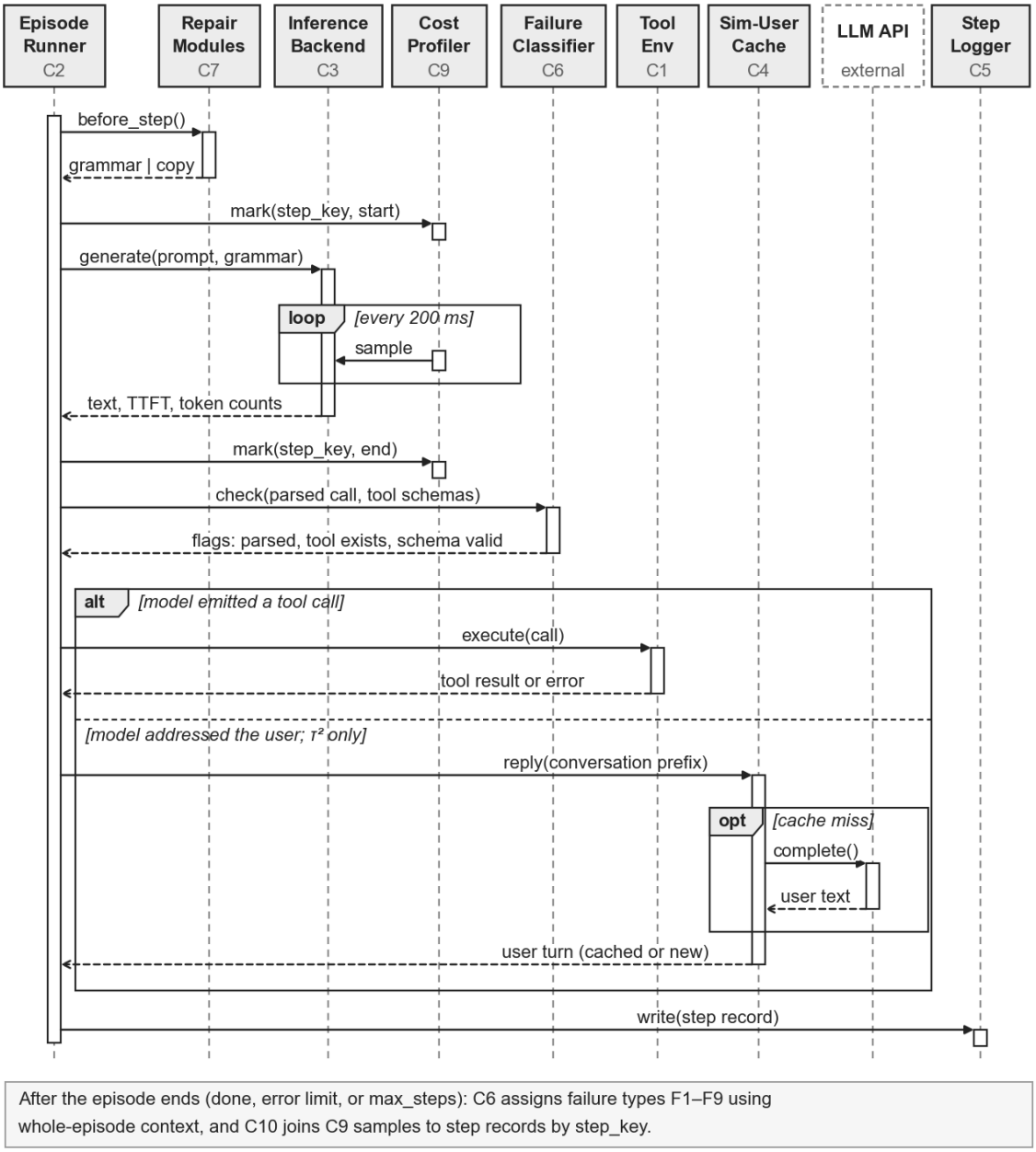


Figure 3. UML sequence diagram of one multi-turn agent step. Solid arrows are calls, dashed arrows are returns, and activation bars show when each participant is busy.

Error paths are part of the design, not afterthoughts:

- **Backend crash or timeout.** The runner retries up to three times with the same seed. If the step still fails, the episode is marked as an infrastructure error, excluded, and listed, as the exclusion rules in Chapter 7 specify.
- **Budget exhausted.** SimUserCache raises a budget error. The runner stops scheduling τ²-bench episodes and leaves them pending, so no truncated conversation enters the analysis.
- **Benchmark limits.** The τ²-bench step and error limits and BFCL's per-turn step limit (a turn with more than 20 steps is force-terminated and scored incorrect [48]) end episodes as outcomes, not exclusions.

Data Design

The step record is the central data structure. The Step Logger writes one JSON line per step to `runs/RUN_ID/steps.jsonl` next to the run's `manifest.json`. After the run, consolidation writes a Parquet dataset partitioned by benchmark, model, and format, with the schema version stored in the file metadata. Episode records (verdict, number of steps, termination reason), C9 cost samples, and the C4 reply store are kept in separate tables.

Table 23. Step-record schema (one row per conversation step).

Field	Type	Description
run_id	string	Hash of config cell and seed; key to the run manifest
config_hash	string	SHA-256 of the canonical experiment config
model_id	string	Hugging Face repository and revision of the original checkpoint
variant_id	string	SHA-256 of the weight file that produced this step
quant_format	enum	BF16, Q4_0, Q4_K_M, IQ4_XS, MLX 4-bit, Q8_0, and so on
backend	enum	llama.cpp or mlx-lm
backend_version	string	llama.cpp build and commit, or MLX and mlx-lm versions
repair_id	string	none, or the R1, R2, or R3 variant or control
benchmark	enum	BFCL multi-turn, τ²-bench-verified, or BFCL single-turn
task_id	string	Benchmark task identifier
seed	int	Trial seed, shared across conditions
episode_id	string	Hash of run_id and task_id
step_idx	int	Index of the step within the episode, from 0
depth_bin	enum	Pre-registered bin of step_idx
role	enum	assistant, tool, or user
step_type	enum	tool_call, user_reply, or end (assistant steps only)

Table 23. Step-record schema (one row per conversation step).

Field	Type	Description
prev_event	enum	task_start, user_turn, tool_result, or tool_error
prompt_sha256	string	SHA-256 of the exact prompt token IDs
prompt_tokens	int	Prompt length in tokens
output_text	string	Raw model output
completion_tokens	int	Generated tokens
parsed_call	JSON	Tool name and arguments, or null
tool_name	string	Name of the called tool, or null
call_parsed	bool	A well-formed call in the model's native format was found
tool_exists	bool	tool_name is in the task's tool list
args_valid	bool	Arguments parse as a JSON object
schema_valid	bool	Arguments satisfy the tool's JSON Schema
tool_result	string	Environment output or error text, or null
route	enum	R2c routing decision (low or high precision), or null
failure_type	enum	F0 to F9, or null; set offline by C6
latency_ms	float	Request sent to last token, client clock
ttft_ms	float	Request sent to first token
decode_ms	float	Generation time after the first token
energy_mj	float	Idle-subtracted CPU, GPU, and ANE energy in the step window (from C9)
peak_rss_mb	float	Peak resident memory of the backend process in the step window (from C9)
timestamp	string	Step start, ISO 8601 in UTC

The entity-relationship diagram shows how step records connect to the rest of the registry. An Experiment has many Runs, a Run has many Episodes, and an Episode has many Steps. Each Run applies one Repair (the unrepaired baseline is the repair "none") and loads one ModelVariant, or two for the llama.cpp step-selective variant, so each Step records which variant produced it. A ModelVariant produced by retraining points to its TrainingJob (QAD or the train-longer control). A Step has at most one Failure label and any number of CostSamples in its time window. Simulated-user replies live in a separate content-addressed store with no foreign keys, because one reply can serve many runs.

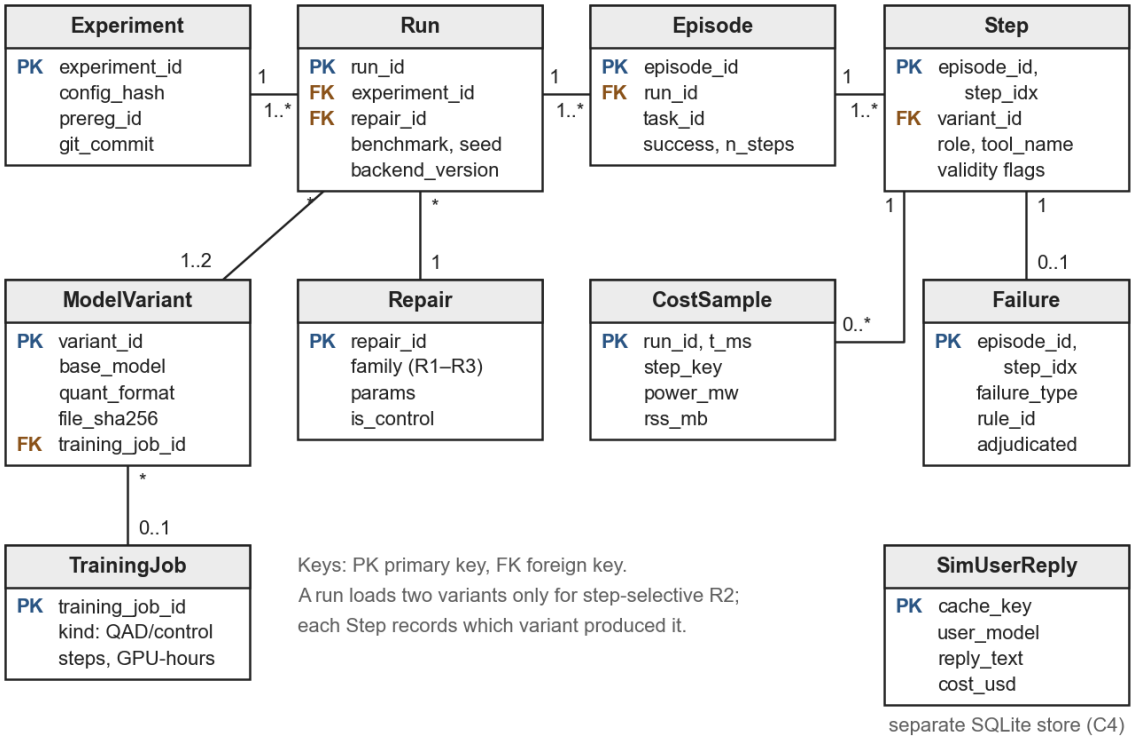


Figure 4. Entity-relationship diagram of the experiment registry and step logs. Multiplicities follow UML notation.

Failure Taxonomy Design

Each failed assistant step receives exactly one label. The classes below extend the list in Chapter 3 (FR-E8) with F9 for errors in what the agent tells the user, which τ^2-bench checks separately from tool calls. Rules run in the precedence order F2, F1, F3, F7, F4, F5, F8, F6, F9, and the first match wins. Format failures therefore take priority over semantic ones. A failure that belongs to the whole conversation is attached to the step where it becomes visible: the first repeated call for F7, and the ending step for F6.

Table 24. Failure taxonomy (codebook v0; frozen as v1 before the main study).

ID	Type	Definition	Detection rule	Example
F1	Hallucinated tool	Call names a tool that is not offered	call_parsed and not tool_exists	Calls a plausible tool name absent from the tool list
F2	Malformed call	Intended call cannot be parsed	Call markers present and not call_parsed, or not args_valid	Arguments cut off before the closing brace
F3	Schema-invalid arguments	Existing tool; arguments break its schema	tool_exists and not schema_valid	A required ID missing, or a string where an integer is required
F4	Wrong tool	Valid call to a tool that the reference does not expect at this point	Aligned to the reference trajectory, the tool name matches no pending expected action	Cancels a booking when the task requires changing it
F5	Wrong argument value	Expected tool, valid schema, wrong values	Name matches a pending action; normalized arguments differ	Correct tool with the wrong order ID or date
F6	Premature termination	Ends the task or turn while required actions remain	Episode, or BFCL turn, ends with expected actions still pending	Tells the user the change is done without calling the tool
F7	Repetition loop	Same call repeated with no new information	Identical name and normalized arguments at least 3 times in a row (threshold pre-registered)	Re-fetches the same user record until the step limit
F8	Policy violation (τ^2)	Action forbidden by the domain policy	State-changing action outside the expected set that a codebook policy rule flags	Performs a write action the policy forbids for this customer
F9	User-facing response error (τ^2)	Message to the user is wrong or omits required information	A required-output check fails, or the reply contradicts the latest tool result	Quotes an amount that differs from the tool's result
F0	Other	A failed step that matches no rule	Fallback; share monitored	Reviewed by hand; frequent patterns feed the next codebook

F1 to F3 need only the step itself. F4, F5, F6, and F8 need the reference: BFCL's ground-truth trajectories or τ^2-bench's expected actions. Both benchmarks can admit more than one valid path, so these rules align calls to the reference and send uncertain alignments to human review instead of guessing. The codebook defines each class with positive and negative examples. Two team members label a stratified sample of 300 pilot steps independently, blind to model, format, and repair. We require Cohen's κ of at least 0.80 between them [85]; the third member adjudicates disagreements. The classifier must then reach $\kappa \geq 0.80$ against the adjudicated labels (Chapter 7). If the share of F0 exceeds 10% of failures, we revise the codebook before freezing it.

Experiment Grid Design

An experiment config lists values for each axis plus constraints. C11 expands the Cartesian product, removes combinations that the constraints forbid, and applies the

inclusion set from Stage 1. Each remaining cell and seed becomes a run spec with a hash-derived run ID. The registry tracks each run spec as pending, running, done, or failed. A restart therefore resumes where it stopped, and a finished run is never recomputed. Correctness runs are grouped by model variant to avoid reloading weights. Cost runs are interleaved in random order.

Table 25. Axes of the Essential grid (details and episode counts in Chapter 3).

Axis	Values	Constraint
Model	Qwen3-0.6B, Qwen3-1.7B, LFM2.5-1.2B-Instruct (core); Qwen3-4B, SmolLM3-3B	R3 only on the core models
Runtime and format	llama.cpp: BF16, Q4_0, Q4_K_M, IQ4_XS; mlx-lm: bf16, 4-bit	Format must belong to the runtime
Repair	none; R1; R2a uniform; R2b layer-selective; R2c step-selective; R3 arms and control	Repairs start from Q4_0 or MLX 4-bit; R1 also runs at 16 bits as a control
Benchmark	BFCL multi-turn, τ^2-bench-verified, BFCL single-turn	Tasks limited to the per-model inclusion set
Trial	4 sampled trials plus 1 greedy pass	Same seeds in every condition

For example, one run spec is Qwen3-0.6B, Q4_0 on llama.cpp, R1 grammar, τ^2-bench-verified, trial 2. Its episodes are paired task by task with the same trial of the Q4_0 baseline and of the BF16 reference.

Pre-registration. Before the main study (milestone M2), we register the following on OSF [68]: hypotheses H1 to H4, the inclusion and exclusion rules, the step-type definitions and depth bins, codebook v1, the tests, the Holm families [65], and the commit hash of the analysis code. The Experiment entity stores the registration ID. C10 labels any analysis whose code hash differs from the registered one as exploratory, so a confirmatory result cannot silently come from edited code.

CLI and Report Mockup

The testbed has no end-user GUI. Researchers use a command-line interface built on Hydra-style overrides, and readers use a static report.

Table 26. Planned command-line interface.

Command	Purpose	Component
`uv run python -m testbed.run +experiment=NAME`	Expand the grid and run all pending episodes; resumes after interruption	C11, C2
`uv run python -m testbed.run +experiment=NAME dry_run=true`	List run specs, episode counts, and projected API spend without running	C11, C4
`uv run python -m testbed.replay run_id=ID`	Re-execute one run from its manifest with cached user turns	C11, C4
`uv run python -m testbed.cache report`	Hits, misses, and spend against the $300 cap	C4
`uv run python -m testbed.classify run_id=ID`	Consolidate logs to Parquet and assign failure types	C5, C6
`uv run python -m testbed.profile +experiment=NAME`	Cost runs with warm-up and interleaving	C9
`uv run python -m analysis.report +analysis=prereg`	Registered tests, damage map, comparison matrix, HTML report	C10
`python -m train.qad +train=NAME`	QAD or control training in a notebook, with checkpoint and resume	C8
`python -m train.export ckpt=PATH`	Export to GGUF and MLX and verify on the 4-bit path	C8

The powermetrics tool requires superuser rights. C9 starts it through a sudoers rule limited to that one binary, so the harness itself never runs as root.

The damage-map report is a static HTML site built by C10 from released logs, with PNG and PDF exports for the paper. A row of filters (model, format, repair, benchmark, failure type) sits above a heatmap of step type × depth bin. Each cell shows the failure-rate ratio against the 16-bit reference, and hovering shows counts, the 95% CI, and the run IDs behind the cell. Cells whose CI excludes 1 after Holm correction are outlined, so significance does not depend on color alone. Clicking a cell lists example steps with their raw outputs. A second view holds the repair comparison matrix. Its rows are repairs and controls, and its columns are the failure gap closed, the success gap closed, peak memory, TTFT overhead, extra tokens, and energy per task, with a Pareto plot of failure reduction against each cost. Every page ends with the config hash and commit that produced it.

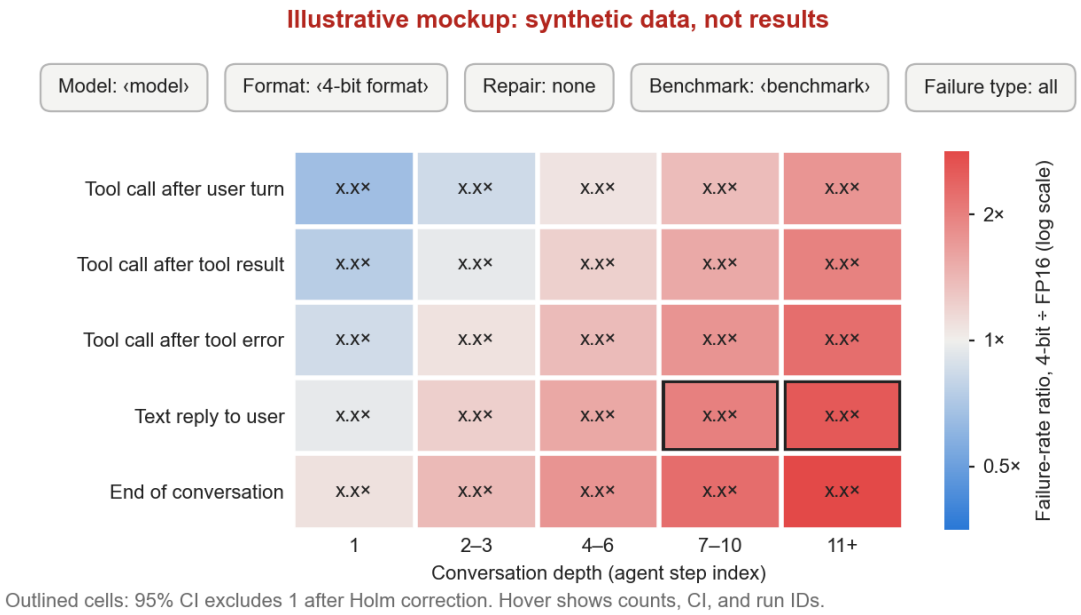


Figure 5. Illustrative mockup of the damage-map view. All values are synthetic placeholders, not results.

Chapter 7. QA, Performance, Deployment Plan

This chapter is a preliminary plan written before the testbed exists; we will revise it with test results and measured baselines for Workbook 2. Because the project is a measurement study, quality assurance covers both software correctness and measurement validity: the failure counts must describe the models, not the harness.

Quality Assurance and Testing

Software Testing

Unit tests (pytest) target the functions whose bugs would silently change failure counts: the tool-call parser for each model family's native format, the JSON-schema validator, each Failure Classifier (C6) rule, and the Simulated-User Cache (C4) key. Classifier rules get positive and negative fixtures from real pilot transcripts. The cache key must change exactly when an input to the simulated user's reply changes (conversation prefix, user model, sampling parameters, seed); a wrong key replays the wrong reply or wastes the $300 budget.

Property-based tests use Hypothesis [102] to generate malformed outputs (truncated JSON, nested quotes, Unicode, prose around the call). The parser must never crash, never accept a call the validator rejects, and must survive a serialize-then-parse round trip.

Integration tests run a full episode through C1 to C6 on Qwen3-0.6B with a fixed seed and cached simulated-user replies. A parity test sends the same conversation to llama.cpp and mlx-lm at FP16/BF16: the rendered chat template and token IDs must be identical, and greedy outputs must agree above a pilot-set threshold (exact equality is not expected, because the kernels differ).

Regression tests replay frozen golden episodes from the cache on the same machine, backend build, and model file; step logs must be byte-identical once wall-clock fields are stripped. A difference is either an intended change, regenerated in a reviewed commit, or new non-determinism to investigate.

Continuous integration on GitHub Actions runs unit, property, and CPU-only integration tests on Linux for every pull request. GitHub's arm64 macOS runners (for example `macos-15`, free for public repositories) are virtual machines with 3 M1 cores and 7 GB of RAM [103], too small for FP16 references and unrepresentative for timing, so they run only a smoke test. Metal-backend tests and all performance measurements run locally on Apple silicon, with reports committed to the repository.

Table 27. Software test plan.

Test level	What is tested	Tool	Pass criterion	Owner
Unit	Tool-call parser, schema validator	pytest	Every fixture yields the expected call or typed error	Ekant
Unit	Classifier rules (C6), cache keys (C4)	pytest	Rules fire only on positive fixtures; keys equal iff inputs equal	Vritika
Unit	Config expansion, run IDs (C11)	pytest	Grid expands as expected; IDs stable across machines	Yashashav
Property	Parser, validator, cache keys	Hypothesis	No crash; no invalid call accepted; round trip holds	Ekant, Vritika
Integration	End-to-end episode, Qwen3-0.6B, fixed seed	pytest, llama.cpp CPU	Episode completes; log passes schema	Ekant
Integration	llama.cpp vs mlx-lm parity at FP16/BF16	pytest, local Metal	Identical token IDs; agreement above pilot threshold	Ekant
Integration	QAD export (C8) on real 4-bit path	llama.cpp perplexity tool	KL and top-1 agreement within tolerance	Yashashav
Regression	Golden step logs replayed from cache	pytest	Deterministic fields byte-identical	Ekant
CI	Unit, property, CPU integration; macOS smoke	GitHub Actions	Green before merge to main	Ekant

Measurement Validity

Classifier accuracy. Before the main runs, two team members independently label a stratified sample of about 300 pilot steps (all failure classes plus valid steps) from a written codebook, blind to model, precision, and repair. Inter-rater Cohen's κ [85] must be at least 0.80, the edge of the top band on the usual interpretation scale [104]; the third member adjudicates. The classifier must then reach $\kappa \geq 0.80$ against the adjudicated labels, with per-class precision and recall reported; if it falls short, we revise the rules and re-test on a fresh sample. Assuming 90% raw and 30% chance agreement, 300 items give $\kappa \approx 0.86$ with a 95% CI half-width of about ± 0.05. A smaller check on main-study data tests whether classifier error differs by precision condition, which would bias amplification ratios directly.

Reproducing published numbers. Each FP16/BF16 reference must match published scores for the same model, benchmark version, and prompting mode within a pre-registered tolerance. If it does not, we audit the chat template, tool-call format, sampling settings, and benchmark version before any repair comparison. Because tau2-bench-verified corrects tasks of the original τ^2-bench, published scores are compared only with runs on the same task-set version.

Determinism. Decoding is greedy, seeds are fixed, and versions are pinned (llama.cpp commit, MLX and mlx-lm, macOS build, Python lockfile). Cost-measurement runs send one request at a time; correctness runs may use a fixed number of parallel `llama-server` slots (Chapter 3, NFR-5) only after the M2 pilot shows their greedy outputs match single-slot runs. Batch and context sizes are fixed, and measurement runs disable llama.cpp prompt caching, which its documentation warns can make results non-deterministic [1]. He attributes most run-to-run variation in LLM serving to kernels that are not batch-invariant [67], and Jang et al. report that repeating a 456-episode run on their non-deterministic stack shifted the mean score by about 1.5 points [20]. We run each FP reference cell twice, report the share of byte-identical episodes, and use the replicate spread as the noise floor.

Real 4-bit code path. QAD (C8) trains in PyTorch with simulated quantization, but deployment uses llama.cpp and MLX kernels; a mismatch in block size, scale format, or rounding would train one quantizer and measure another. For each exported checkpoint we (1) confirm that every quantized tensor in the GGUF or MLX file has the intended type; (2) compare next-token distributions of the fake-quant and exported models on held-out text and logged agent prompts (mean KL divergence, top-1 agreement, perplexity) with llama.cpp's perplexity tool [1]; and (3) require the export-vs-fake-quant KL to be small relative to the FP16-vs-4-bit KL (ratio fixed after the pilot, for example below 10%). The same audit covers the R2 mixed-precision variants and the train-longer control.

Statistical Analysis Plan

We will register hypotheses, metrics, exclusions, and analysis code on OSF before the main data collection [68]; pilot runs only set thresholds and check power.

- **H1 (RQ1):** 4-bit weights raise the per-step rate of at least one failure type relative to FP, and the rate ratio varies by step type and grows with conversation depth.
- **H2 (RQ2):** each repair lowers the unrepaired 4-bit failure rate and beats its control: R1 grammar vs unconstrained decoding with the same prompt; R2 step-selective vs uniform added bits at equal average bits per weight; R3 QAD vs train-longer-at-full-precision with equal data and steps.
- **H3 (RQ3):** for at least one repair whose overall score is equivalent to FP (two one-sided tests, pre-registered margin [64]), failure-type rates still differ from FP.
- **H4 (RQ4):** repairs differ in deployment cost; R1 keeps decode throughput within a pre-registered margin of unrepaired 4-bit, R2 raises memory, and R3 adds only one-time training cost.

The unit of analysis is the episode (task × seed), paired by task with the FP reference; failure rates are failures per agent step. Effect sizes are the rate ratio (amplification) and the difference in failures per 100 steps, with 95% CIs and tests from a paired cluster bootstrap that resamples tasks with all their seeds (10,000 resamples) [86],

as recommended for correlated LLM evaluation items [62] and as Jang et al. did [20]. A Poisson mixed model (task random effect, log-steps offset) is the sensitivity check; Holm–Bonferroni [65] corrects within each hypothesis family. BFCL multi-turn has no LLM-simulated user, so under greedy decoding its seeds give identical episodes and the task count bounds the sample.

Power. Given the total failure events of one type in a paired FP/4-bit comparison, the 4-bit share is 0.5 under the null and 0.6 at a 1.5× amplification (assuming equal step counts). Detecting this with 80% power at two-sided $\alpha = 0.05/12$ (Holm's first step with 12 tests per family) needs about 340 events, times an unknown clustering design effect D; the table assumes 10 agent steps per episode. For scale, Jang et al. ran 456 episodes per cell (114 tasks × 4 trials) and saw FP tool-name hallucination rates from below 1% to about 20% of tool calls [20]. At 456 episodes, a 10% base rate is detectable even at D = 3, 5% only if D is below about 1.7, and 2% not at all, so rare types are pooled or reported as exploratory. We will recompute the table from pilot data before registering.

Table 28. Approximate episodes (tasks × seeds) needed to detect a 1.5× rate ratio (80% power, $\alpha = 0.05/12$, 10 steps per episode).

FP base rate per step	Expected events per FP/4-bit episode pair	Episodes, D = 1	Episodes, D = 3
2%	0.50	681	2,043
5%	1.25	273	817
10%	2.50	137	409
20%	5.00	69	205

Exclusion rules, fixed in advance:

- Tasks the FP reference fails in every seed are excluded per model from confirmatory tests (they reflect missing capability, not quantization); we report the counts and an all-task sensitivity analysis. A model × benchmark cell with fewer than 30 eligible tasks is reported descriptively, and that model moves to the easier benchmark.
- Infrastructure errors (backend crash, API timeout) are retried up to three times with the same seed; episodes that still fail are excluded and listed.
- Episodes that hit the step or error limit are outcomes, not exclusions.

Performance Evaluation Plan

A repair's cost is what it adds relative to unrepaired 4-bit on the same task × seed, measured by the Cost Profiler (C9) on the M4 Pro (memory size of M4 Pro of unified

memory, which also bounds the largest FP16 reference). Ekant's 16 GB M4 is for development only.

Table 29. Cost metrics.

Metric	Definition	Tool	Protocol	Reported as
Peak memory	Max process RSS; peak Metal allocation	psutil sampler; mlx-lm peak memory; llama.cpp buffer log	Per episode	Median, IQR (GB)
Model size	Bytes on disk of GGUF file or MLX weights	File system	Once per artifact	GB; bits per weight
TTFT	Request sent to first streamed token	Client timer; llama-server `timings`	Per step, binned by prompt length	Median, IQR (ms)
Decode speed	Generated tokens ÷ decode time	llama-server `timings`; mlx-lm stats; llama-bench	Per step	Median, IQR (tokens/s)
Extra tokens	Tokens per task minus unrepaired 4-bit (grammar-forced, retries, longer turns)	Step Logger (C5)	Paired by task × seed	Median Δ and %
Energy per task	Sum of CPU + GPU + ANE power × interval, idle subtracted	powermetrics (sudo)	Per task, 200 ms samples	Median, IQR (J)
One-time cost	QAD and control training; conversion	C8 run logs	Per training run	GPU-hours by GPU type

llama-server returns per-request timings and mlx-lm reports throughput and peak memory [1, 9]; we cross-check both against client-side timers.

Energy. powermetrics must run as superuser and samples every 5,000 ms by default; we will sample every 200 ms and measure its own overhead. Its manual page calls its CPU, GPU, and ANE power values estimates that may be inaccurate and not comparable across devices, so we only compare repairs on one machine, subtract a same-session idle baseline, and never report whole-system energy. **One-time costs** record GPU type and GPU-hours, including resume overhead on free Colab and Kaggle tiers.

Protocol. The machine runs plugged in, with Low Power Mode off, one macOS build for the whole study, and user applications closed. Cost runs replay cached simulated-user replies, so API latency never enters TTFT, and keep prompt caching on, as a deployed agent would. Each block starts with a warm-up workload until decode throughput is stable (three consecutive windows within 5%), with thermal pressure logged. Three warm-up runs per configuration are discarded, then N = 10 repetitions (revised from pilot variance) run in randomized interleaved order so drift does not align with one repair. We report medians and IQRs, with bootstrap CIs for median ratios.

Deployment Plan

For a research artifact, deployment means a release others can run, check, and extend.

Open-source release. The testbed will be public on GitHub under Apache-2.0, chosen over MIT for its explicit patent grant and compatible with our dependencies: llama.cpp (MIT) [1], mlx-lm (MIT) [9], BFCL's Gorilla repository (Apache-2.0), and tau2-bench (MIT). Released weights follow the model licenses (Hugging Face, checked September 27, 2026): Qwen3 and SmolLM3 are Apache-2.0, but LFM2.5 uses the LFM Open License v1.0, which does not license commercial use by entities with $10 million or more in annual revenue, so LFM2.5-derived checkpoints keep that license. Releases are semantically versioned, and each paper table names its release. A Dockerfile covers the Linux/CUDA parts (QAD training, analysis, CI); the Metal parts run natively from a locked uv environment (conda file as fallback).

Artifacts. Step-log datasets (Parquet) go on the Hugging Face Hub with dataset cards; repaired checkpoints and controls go there in GGUF and MLX formats with model cards [98] stating base model, quantization format, repair, control, and failure rates with CIs. Configs (C11) live in the repository, and the pre-registration is frozen on OSF. Before publishing step logs, we confirm that the simulated-user API provider's terms allow redistributing its generated text.

Reproducibility packaging targets the ACM badges [89], the same three that EuroSys 2022 artifact evaluation awarded [105]: *Available* (releases archived with a DOI, for example through Zenodo), *Functional* (documented, complete, exercisable; one command regenerates every table and figure from released step logs on a laptop, with no API or GPU cost), and *Reproduced* (someone outside the team reruns a subset from scratch within our stated CIs). We do not assume EuroMLSys runs artifact evaluation.

Damage-map report. Analysis & Stats (C10) builds a static public report of the damage map, comparison matrix, and cost tables from the released data, linking each number to its config and run IDs.

Paper and demo. We target EuroMLSys 2027 (EuroMLSys 2027 submission deadline) and an Expo demo running FP, 4-bit, and repaired agents side by side on the M4 Pro with live failure labels and cost meters.

Table 30. Release checklist.

Item	Content	Owner	Target
Repository v0.1	C1 to C6, C11, tests, CI, license, README	Ekant	295A end
Environments	uv lockfile (macOS), Dockerfile (Linux/ CUDA)	Yashashav	295A end
Codebook and labeled sample	Taxonomy, 300 labeled pilot steps, κ report	Vritika	295A end
OSF pre-registration	Hypotheses, exclusions, analysis code	Vritika	295A end, before main runs
Repository v0.5	R1 to R3, C8 export checks, C9 profiler	Yashashav, Ekant	295B mid
Step-log dataset v1	FP and 4-bit runs with dataset card	Vritika	295B mid
Paper submission	EuroMLSys 2027	All	295B mid
Checkpoints	QAD and control in GGUF and MLX, model cards	Yashashav	295B end
Release v1.0 with DOI	Frozen code, data, configs	Yashashav	295B end
Damage-map report	Static site built from release data	Vritika	295B end
External reproduction	Outside rerun from cache and from scratch	All	295B end
Expo demo	Side-by-side agents on the M4 Pro	Ekant	295B end

Risks to Validity

Table 31. Threats to validity and mitigations.

Type	Threat	Mitigation
Internal	Simulated-user variance changes conversations between conditions	Shared seeds and cached replies across conditions; task-cluster bootstrap
Internal	Inference non-determinism mimics or hides damage	Greedy decoding, pinned versions, one request at a time, measured noise floor
Internal	QAD gains come from extra training, not training under quantization	Train-longer-at-FP control with equal data and steps
Internal	Classifier error differs by condition	Blind labeling; $\kappa \geq 0.80$; per-condition error check
Construct	Taxonomy misses or merges failure types	Codebook from pilot data; monitor the "other" share; manual review
Construct	Grammar conflicts with the chat template (native tool-call tags, reasoning blocks)	Grammar built from each model's native format; validated at FP first
Construct	powermetrics power is an estimate	Same-machine comparisons only; idle subtracted; limits stated
External	Benchmark contamination inflates absolute scores	Paired FP vs 4-bit contrasts; published-number sanity check
External	M4 Pro results may not transfer to phones	Report per-token and per-bit costs; state scope; no phone claims
External	Few models and benchmarks; runtimes change	Three model families; pinned versions; released configs for reruns
Statistical	Rare failure types underpowered; many tests	Power plan; Holm correction; rare types exploratory

Chapter 8. Implementation Plan and Progress

This chapter covers five things: the software environment the testbed runs in, the tools each member needs, the hardware we measure and train on, the small programs ("spikes") that show we can use each technology, and the prototype we build first to connect all of them. It closes with how we track progress. As of 2026-09-27 we have not run any experiments.

Programming and Execution Environment

The testbed runs in two execution contexts. The **Apple silicon context** (macOS on M-series laptops) runs everything that touches inference: the Benchmark Adapter (C1), Episode Runner (C2), Inference Backends (C3), Simulated-User Cache (C4), Step Logger (C5), Failure Classifier (C6), repairs R1 and R2 (C7), the Cost Profiler (C9), and Analysis & Stats (C10). The **CUDA notebook context** (Linux on free Colab and Kaggle GPUs) runs the QAD Training Pipeline (C8) and converts trained checkpoints to GGUF. Both contexts share one Git repository with top-level `testbed/`, `train/`, `configs/`, `analysis/`, and `spikes/` directories. Each context has its own lockfile. Yashashav maintains the lockfiles and a Dockerfile that packages the Linux/CUDA parts (training, analysis, CI) for reproducibility. The Colab and Kaggle notebooks install from the `train` lock directly, and Ekant maintains continuous integration on GitHub Actions (Chapter 7).

We use **Python 3.12** in every environment, managed by **uv**, with conda as a fallback only when a package has no wheel. Python 3.12 satisfies our 3.11+ policy and every benchmark harness we depend on. The upstream tau2-bench v1.0.1 requires Python ≥ 3.12 and < 3.14, while tau2-bench-verified and `bfcl-eval` require ≥ 3.10 [47, 50, 92]. The two benchmark harnesses live in separate uv environments because each pins its own dependencies (tau2-bench, for example, pins a LiteLLM range). The testbed talks to them only through HTTP endpoints and files, so a dependency conflict inside a harness cannot break the testbed.

llama.cpp is built from source at one pinned release tag for the whole study [1]. On macOS, Metal is enabled by default, so the build is `cmake -B build` followed by `cmake --build build --config Release`. Adding `-DGGML_METAL=OFF` gives a CPU-only build, used only for sanity checks. The build produces `llama-server`, `llama-cli`, `llama-quantize`, `llama-imatrix`, and `llama-perplexity`, and the repository provides `convert_hf_to_gguf.py`. We build the same tag in the Colab environment so that the GGUF files produced after training match the runtime that loads them. The Experiment Config & Registry (C11) writes the `llama-server --version` string (build number and commit) into every run record.

The testbed relies on four `llama-server` interfaces:

- the OpenAI-compatible `/v1/chat/completions` endpoint, whose native tool calling uses the model's Jinja chat template (`--jinja`);
- the per-request `grammar` and `json_schema` fields (and `response_format` with a JSON schema on the chat endpoint), which R1 uses. The server converts JSON schemas to GBNF grammars at request time;
- the server-wide `--grammar-file` and `-j`/`--json-schema` flags, which we use only in spikes;
- the `timings` object in each response (`prompt_ms`, `predicted_per_second`), which the Cost Profiler reads.

MLX and **mlx-lm** are installed with `pip install mlx-lm` (which pulls in `mlx`) [9, 71]. `mlx_lm.convert --hf-path MODEL_ID -q --q-bits 4 --q-group-size 64` produces the MLX 4-bit variant. Its `--q-mode` option (affine by default; `mxfp4`, `nvfp4`, `mxfp8`) and its mixed-precision `--quant-predicate` recipes (`mixed_2_6`, `mixed_3_4`, `mixed_3_6`, `mixed_4_6`) provide the MLX side of repair R2. `mlx_lm.server` parses tool calls but has no grammar option. R1 on MLX therefore uses **XGrammar** as a logits processor passed to mlx-lm's `generate_step`; XGrammar ships an mlx-lm example with a Metal masking kernel [36]. Because the two runtimes use different grammar engines, we report R1 separately for each runtime.

llama-cpp-python is optional. We install it with `CMAKE_ARGS="-DGGML_METAL=on" pip install llama-cpp-python` and use it only when we need logits in-process, for example to compare a PyTorch fake-quantized model with llama.cpp token by token. The default path is `llama-server` over HTTP because that is how these models are deployed.

Table 32. Execution environment, version policy, and the command that verifies each layer.

Layer	Tool	Version policy	Purpose	Verified by
Interpreter	Python 3.12 via uv (conda fallback)	3.12.x; `uv.lock` per environment	Testbed, harnesses, analysis	`uv run python -V`
GGUF runtime	llama.cpp, CMake build, Metal on	One release tag for the study	`llama-server`, `llama-cli`	`llama-server --version`
GGUF tooling	`convert_hf_to_gguf.py`, `llama-quantize`, `llama-imatrix`, `llama-perplexity`	Same tag as the runtime	Conversion, 4-bit variants, KL checks	`llama-quantize --help`
Binding (optional)	llama-cpp-python, Metal build	Pinned	In-process logits	`python -c "import llama_cpp"`
MLX runtime	mlx, mlx-lm	Pinned minor (mlx-lm 0.31.x today)	`mlx_lm.server`, `mlx_lm.convert`	`python -c "import mlx_lm; print(mlx_lm.__version__)"`

Table 32. Execution environment, version policy, and the command that verifies each layer.

Layer	Tool	Version policy	Purpose	Verified by
Constrained decoding	GBNF and JSON schema (llama.cpp); XGrammar (MLX)	Pinned	Repair R1	`uv pip show xgrammar`
Training	PyTorch, Transformers, Datasets, Accelerate (CUDA)	Separate `train` lock	QAD pipeline (C8)	`python -c "import torch; print(torch.cuda.is_available())"`
Benchmarks	`bfcl-eval`; tau2-bench-verified (LiteLLM)	Pinned commit, own environments	Reference scoring for C1	`bfcl --help`; `tau2 --help`
Analysis	pandas, pyarrow, SciPy, statsmodels, matplotlib	Pinned	Analysis & Stats (C10)	`uv run pytest tests/analysis`
Configuration	Hydra (`hydra-core`) over YAML	Pinned	Config & Registry (C11)	`python -m testbed.run --cfg job`
Quality	pytest, pre-commit (ruff)	Hooks pinned	Tests and lint before commit	`pre-commit run --all-files`
Collaboration	Git, GitHub (Projects, Actions), Hugging Face Hub	Hosted	Code, board, CI, model artifacts	`gh auth status`; `hf auth whoami`
Energy	`powermetrics` (macOS, sudo)	OS tool; macOS build frozen during measurement	Cost Profiler (C9)	`sudo powermetrics -n 1 --samplers cpu_power,gpu_power,ane_power`

Each run record also stores the Git commit, the lockfile hash, the SHA-256 hash of the model file, the macOS build, and a machine identifier, so we can rebuild any reported number from its configuration.

Development Tools per Team Member

Every member uses the same repository, GitHub Projects board, and Hugging Face organization (private model repositories until release). Beyond that, each member's tools follow the components they own (Chapter 9, Task Assignment Summary). Only one account in the project costs money: the simulated-user API key. C4 enforces its $300 cap in software, and we also cap the key at the provider where the provider allows it. Each member trains and runs notebooks only under their own Colab and Kaggle accounts.

Table 33. Development tools, accounts, and status per member.

Member	Machine	Role-specific tools	Accounts and quotas	Status
Ekant Kapgate	Apple M4 MacBook, 16 GB unified memory (verified; macOS 26.6)	llama.cpp (Metal build), mlx-lm, XGrammar, `powermetrics`, psutil, GitHub Actions	GitHub; Hugging Face (read token); own Colab and Kaggle as backup	Set up
Vritika Malhotra	laptop model and memory	`bfcl-eval`, tau2-bench-verified, SQLite, SciPy, statsmodels, Jupyter	GitHub; Hugging Face; OSF (pre-registration); simulated-user API key ($300 cap, provider simulated-user provider and model)	Set up
Yashashav Devalapalli Kamalraj	laptop model and memory	PyTorch, Transformers, Accelerate, llama.cpp conversion tools, `mlx_lm.convert`, uv, Docker	GitHub; Hugging Face (write token); Colab (free tier); Kaggle (about 30 GPU-hours/week)	Set up
Shared	Apple M4 Pro MacBook (measurement machine)	Pinned llama.cpp and mlx-lm builds; `powermetrics` under sudo	Owner: owner of the M4 Pro	Access arranged

If a member's laptop is not Apple silicon, that member develops against a CPU build of llama.cpp and runs MLX- and Metal-specific tests on the M4 Pro or on Ekant's M4.

Hardware Prototyping and Simulation Environment

The project has no custom hardware. Its "prototyping hardware" is the set of machines the models run and train on, and its "simulation environment" is the set of deterministic benchmark environments the agents act in.

Measurement machine: Apple M4 Pro

Every number we report comes from one Apple M4 Pro MacBook owned by owner of the M4 Pro with memory size of M4 Pro of unified memory. Apple sells the M4 Pro MacBook Pro with 24 GB of unified memory as the base configuration (48 GB optional) and 273 GB/s of memory bandwidth [106]. Whichever configuration the team has therefore holds every full-precision reference model in the table below. During measurement sessions the machine runs on AC power with other applications closed, and we defer macOS updates until the cost profiling milestone (M6) is complete, so the operating system does not change mid-study.

Development machines

Ekant's Apple M4 MacBook with 16 GB (verified on the machine) and the other members' laptops run the small models, replays, unit tests, and spikes. They never produce reported numbers. macOS lets Metal use only part of unified memory by default.

`llama-server` prints this limit (`recommendedMaxWorkingSetSize`) at startup, and spike S1 records the value on each machine so we do not rely on an assumption.

What fits in memory

Table 34. Approximate weight memory of candidate study models at 16 bits and at about 4.5 bits per weight.

Model	Parameters	FP16 weights	About 4.5-bit weights	FP16 reference runs on
Qwen3-0.6B	0.6 B	1.2 GB	0.34 GB	Either machine
LFM2.5-1.2B-Instruct	1.17 B	2.3 GB	0.66 GB	Either machine
Qwen3-1.7B	1.7 B	3.4 GB	0.96 GB	Either machine
LFM2.5-2.6B	2.69 B	5.4 GB	1.5 GB	M4 Pro (tight on 16 GB)
SmolLM3-3B	3.08 B	6.2 GB	1.7 GB	M4 Pro (tight on 16 GB)
Qwen3-4B	4.0 B	8.0 GB	2.3 GB	M4 Pro

Parameter counts come from the model cards and checkpoint metadata [43, 45, 46, 96]. Memory is parameters × bits ÷ 8, in GB of 10^9 bytes. The figures are approximate. They cover weights only. About 4.5 bits per weight is the density of llama.cpp Q4_0, which stores one 16-bit scale per block of 32 weights, and of MLX affine 4-bit with group size 64, which stores a 16-bit scale and a 16-bit bias per group. llama.cpp's quantization table lists Q4_K_M at about 4.9 bits per weight (measured on Llama-3.1-8B), so Q4_K_M files are about 10% larger. The KV cache comes on top. For Qwen3-4B (36 layers, 8 KV heads of 128 dimensions) an FP16 cache takes $2 \times 36 \times 8 \times 128 \times 2$ bytes ≈ 0.15 MB per token, or about 2.4 GB for a 16k-token conversation. At FP16, the 4B model therefore does not leave comfortable headroom on the 16 GB machine.

Free cloud GPUs for training

QAD training (C8) runs on free notebook GPUs. Google's Colab FAQ states that the GPU types offered for free vary over time, that usage limits are dynamic and unpublished, and that free notebooks run for at most 12 hours [81]. In practice the free tier usually assigns an NVIDIA T4 (16 GB). Kaggle documents a weekly GPU quota of about 30 hours, sometimes more depending on demand. Its accelerators are a P100 or two T4s, both drawing on the same quota, and GPU sessions run for up to 12 hours [107]. We treat both limits as subject to change and design for them: training runs are short, save a checkpoint to the Hugging Face Hub at a fixed step interval, and resume on either platform.

Neither the T4 nor the P100 supports BF16 natively, so we train with FP16 autocast and loss scaling (or FP32 for the smallest runs). Full-parameter training with

mixed-precision Adam holds about 16 bytes of model state per parameter [95]. By that estimate Qwen3-0.6B needs about 9.6 GB before activations and the 1.2 GB FP16 teacher, which fits a 16 GB GPU only with gradient checkpointing and small micro-batches. LFM2.5-1.2B needs about 19 GB and does not fit. For models of 1B parameters and up we will need 8-bit optimizer states, a split across Kaggle's two T4s, or low-rank updates. Spike S7 measures the real peak memory, and we choose among these options before M2.

Energy measurement

The Cost Profiler (C9) measures energy with macOS `powermetrics`, which needs sudo. It samples the `cpu_power`, `gpu_power`, and `ane_power` samplers every 200 ms in machine-readable `plist` format, for example `sudo powermetrics --samplers cpu_power,gpu_power,ane_power -i 200 -f plist -o run.plist`. C9 starts sampling before a generation, stops it with SIGINT afterward, aligns the samples with the step timestamps from the Step Logger, subtracts an idle baseline, and integrates power into energy per step and per generated token. The tool's own help text says its power values are estimates that should not be compared across devices. We therefore compare repairs with each other only on the same M4 Pro, never across machines. Memory comes from psutil (process resident size), llama.cpp's buffer-size log, and MLX's peak-memory counter. Latency comes from the `timings` object in `llama-server` responses and from mlx-lm's generation statistics.

Simulation environment

The agents act in the benchmarks' own simulated environments. BFCL multi-turn fixes its user turns in the dataset and scores the final state of a simulated backend, so it replays deterministically on any machine [47]. tau2-bench-verified uses an LLM-simulated user [50, 92]. We route that user through the Simulated-User Cache (C4), a content-addressed SQLite store behind an OpenAI-compatible proxy that LiteLLM calls. Once an episode has been recorded it replays free, and development never spends the budget.

Sample Programs and Technology Spikes

Each spike is a short script in `spikes/` with a README that gives the command, the expected output, and what we learned. A spike counts as done only when a second member, the component's backup owner, reruns it successfully on their own machine. That way at least two people understand each technology.

Table 35. Technology spikes, one per technology the prototype depends on.

ID	Spike	Technology exercised	Owner	Success criterion	Status
S1	Serve Qwen3-0.6B Q4_K_M GGUF in `llama-server`; issue one OpenAI-style tool call	llama.cpp, Metal, Jinja tool templates	Ekant	`tool_calls` parsed for a two-tool prompt; `timings` and Metal memory limit logged	In progress
S2	Same request through `mlx_lm.server` on a 4-bit MLX conversion	mlx, mlx-lm, `mlx_lm.convert`	Ekant	Same tool call parsed; conversion command and tokens/s recorded	Planned, Sprint 3
S3	Constrain tool-call output with a JSON-schema grammar (llama.cpp) and XGrammar (MLX)	GBNF, `json_schema`, XGrammar	Ekant	100 sampled calls at temperature 1.0 all validate; unconstrained path confirmed grammar-free	Planned, Sprint 3
S4	Run 5 BFCL multi-turn tasks through the official evaluator against `llama-server`	`bfcl-eval` (`--skip-server-setup`), OpenAI-compatible endpoint	Vritika	`bfcl generate` and `bfcl evaluate` complete; our parser agrees on all 5 tasks	In progress
S5	Run 3 tau2-bench-verified airline tasks with the user routed through the cache, then replay	tau2 CLI, LiteLLM, SQLite cache	Vritika	Replay makes zero paid calls; transcripts identical at temperature 0	Planned, Sprint 3
S6	Sample `powermetrics` around one generation	`powermetrics` (plist), step timestamps	Ekant	Energy per token after idle subtraction; spread over 10 repeats reported	Planned, Sprint 3
S7	50-step toy QAD run on a Colab T4, killed at step 25 and resumed	PyTorch, Transformers, Accelerate, FP16 autocast, checkpoints on HF Hub	Yashashav	Resumed loss curve matches an uninterrupted run; peak GPU memory recorded	In progress
S8	Export a fine-tuned checkpoint to GGUF Q4_0 and MLX affine 4-bit with group size 32 (which maps Q4_0 exactly; the default 64 would re-quantize); compare logits	`convert_hf_to_gguf.py`, `llama-quantize`, `llama-perplexity`, llama-cpp-python	Yashashav	Tensor types as intended; export-vs-fake-quant KL small next to FP16-vs-4-bit KL (Chapter 7 audit)	Planned, Sprint 3
S9	Two-copy step-selective precision switch (Q4_0 and Q8_0 of one model, as in R2c)	Two `llama-server` instances, prompt caching, C3 routing	Yashashav	Each step equals the chosen copy run alone; extra prefill tokens per switch logged	Planned, Sprint 4

S3 also answers a question the study depends on. It checks whether `llama-server`'s native tool-call path already constrains output. If it does, our "unconstrained" 4-bit baseline would be partly repaired already, so the baseline must disable that constraint explicitly. S9 measures the main engineering cost of step-selective precision. Like the step-routing study of Li et al., which keeps a BF16 and a 4-bit copy of the model [21],

our switch holds two model copies. The copies keep separate KV caches, so each switch forces the other copy to prefill every token added since it last ran. The second copy's memory counts against the repair in the cost comparison.

Prototype Implementation Plan

Walking skeleton

The first prototype is a walking skeleton: the thinnest end-to-end path that passes through all eleven components and all three repair families. Its purpose is to show that every technology connects before any of them is scaled up. It is milestone M1 (target 2026-10-23). A single command, `uv run python -m testbed.run +experiment=skeleton`, runs one configuration: Qwen3-0.6B at FP16 versus Q4_K_M (llama.cpp) and 4-bit MLX, on 20 BFCL base multi-turn tasks and 3 tau2-bench-verified airline tasks, with 2 seeds, followed by one minimal variant of each repair. The command writes a report. Its numbers are development data and are excluded from the study under the pre-registration rule.

Table 36. Minimal version of each component in the walking skeleton.

Component	Minimal version in the skeleton	Owner
C1 Benchmark Adapter	Loaders for the 23 tasks, tool schemas normalized to one JSON-schema form	Vritika
C2 Episode Runner	Synchronous turn loop (model, tool, user) with fixed seeds	Ekant
C3 Inference Backends	`llama-server` and mlx-lm behind one `generate(messages, tools, constraint)` call	Ekant
C4 Simulated-User Cache	Caching proxy, spend counter, $5 skeleton budget, hard stop	Vritika
C5 Step Logger	One JSONL record per step, converted to Parquet after each run	Ekant
C6 Failure Classifier	Rules for hallucinated tool, malformed call, invalid arguments, premature termination; 50 hand-labeled steps	Vritika
C7 R1	JSON-schema grammar for tool calls (llama.cpp) and XGrammar (MLX)	Ekant
C7 R2	Uniform Q8_0 variant plus one step-selective rule (from S9)	Yashashav
C7 R3 / C8	Toy QAD checkpoint from S7, exported through the S8 path	Yashashav
C9 Cost Profiler	TTFT, tokens/s, peak memory, and energy for one episode	Ekant
C10 Analysis & Stats	Draft-plan tests on toy data, plus a planted-effect check on synthetic data	Vritika
C11 Config & Registry	One Hydra config; run registry keyed by config hash	Yashashav

Incremental build by milestone

After M1, each milestone widens one layer of the skeleton instead of building a new system. The dates match the milestone table in Chapter 9. M7 (paper submission) and M8 (release) are scheduled there.

- **M1 Walking skeleton (2026-10-23).**
  - Build: the skeleton above.
  - Done when: the skeleton report is reproduced from a clean clone on two machines (Ekant's M4 and the M4 Pro); every step has a C5 record with model hash, backend, constraint, and cost fields; C4 unit tests cover cache hit, miss, and budget stop; C6 disagreements with the 50 hand labels are listed and reviewed.
- **M2 Full-precision references and frozen analysis plan (2026-11-06).**
  - Build: full C1 coverage (BFCL multi-turn base, missing-parameter, missing-function, and long-context categories; tau2-bench-verified domains; an easier tool-calling set such as BFCL single-turn); FP16/BF16 conversions of all study models on both runtimes; repeated reference runs; the inclusion filter that keeps only tasks the full-precision model solves.
  - Done when: our harness gives the same per-task pass/fail as the official BFCL and tau2 evaluators on the same outputs; run-to-run variance of the references is measured; the inclusion set is stored in the registry; the pre-registered analysis plan (hypotheses, metrics, tests, correction, exclusions, analysis code) is frozen on OSF before the first 4-bit run of the main study.
- **M3 Damage map v1, RQ1 (2026-12-04).**
  - Build: 4-bit variants of every model (llama.cpp Q4_K_M and Q4_0; MLX affine 4-bit, group 64), each checked with `llama-perplexity` KL divergence against its FP16 file; batch execution on the M4 Pro; C6 taxonomy v1; step-type and depth annotations.
  - Done when: failure counts by type, step type, and depth bucket, with confidence intervals, exist for every model, format, and benchmark; the failure codebook and 300 hand-labeled pilot steps are finished, with a Cohen's κ report of classifier-human agreement; one command regenerates the damage map from the registry.
- **M4 Repairs R1 and R2 evaluated (2026-12-18).**
  - Build: R1 grammars for each model's tool-call format (JSON for Qwen3 and SmolLM3; Pythonic calls for LFM2.5) on both runtimes. R2 uniform variants (Q5_K_M, Q6_K, Q8_0; MLX 5-, 6-, and 8-bit), layer-selective variants at matched average bits (`llama-quantize --tensor-type`, MLX `--quant-predicate`), and the step-selective switch.

    - Done when: every variant runs on the same inclusion set and seeds as the 4-bit baseline; paired comparisons against the baseline and the FP reference are produced in the pre-registered format; bits per weight and resident memory are recorded for each R2 variant. By the end of CMPE 295A, repository v0.1 (C1 to C6, C11, tests, CI) is tagged and the uv lockfile and Dockerfile are committed.
- **M5 Repair R3 and its control evaluated (2027-01-29).**
    - Build: QAD and a "train longer at full precision" control with identical data, steps, and seeds, full-parameter for Qwen3-0.6B and parameter-efficient (a low-rank update applied before fake quantization) for LFM2.5-1.2B-Instruct and Qwen3-1.7B, as scoped in Chapter 3; export of both arms to GGUF and MLX.
    - Done when: exported checkpoints pass the S8 logit check; both arms are evaluated on the real 4-bit code path; the RQ3 analysis (overall score versus failure counts) covers all three repair families.
- **M6 Cost profiling complete, RQ4 (2027-02-05).**
    - Build: the C9 protocol on the M4 Pro for every repair variant: peak memory, TTFT, decode tokens/s, extra tokens (grammar, switch prefill), and energy per task, plus training GPU-hours for R3.
    - Done when: each cost is reported with repeat-to-repeat spread; comparison matrix v1 (effect on failures versus cost) is generated from the registry. Repository v0.5 (R1 to R3, C8 export checks, C9 profiler) is tagged with the paper at M7.

Progress Tracking

We track progress with four mechanisms:

- **GitHub Projects board.** Each task in the Chapter 9 sprint plan is one issue with a single assignee, a component label (C1 to C11, R1 to R3), and a milestone (M1 to M8). The board has Backlog, Sprint, In progress, Review, and Done columns. Each pull request is reviewed by the component's backup owner (Chapter 9, Task Assignment Summary), so no component depends on one person.
- **Sprint cadence.** Sprints run for two weeks, Monday to Sunday. A 30-minute sprint review on the final Friday demonstrates what runs, updates the board, and plans the next sprint.
- **Weekly advisor meeting** with Dr. Bertin Cordova-Diba (meeting day and time). It covers progress on the board, blockers, and decisions. Decisions go into `docs/decisions.md`, and deviations from the pre-registered plan are logged there with their reasons.
- **Burn-down checks.** We keep three running counts: (1) milestone exit criteria, one checkbox each; (2) planned versus completed runs, drawn from the C11 registry;

(3) budget burn, meaning simulated-user dollars from C4 and GPU-hours from C8, each against its weekly plan.

Table 37. Progress log as of 2026-09-27.

Date	Item	Owner	Status
September 2026	Project abstract completed	All	Done
September 2026	Abstract presentation completed	All	Done
Sprint 1 (by 2026-09-13)	Scoping literature review (runtimes, benchmarks, repairs)	All	Done
Sprint 2 (by 2026-09-27)	Division of work agreed	All	Done
Sprint 2 (by 2026-09-27)	GitHub repository and Projects board created	Ekant	Done
Sprint 2 (by 2026-09-27)	M4 development environment and llama.cpp Metal build (S1)	Ekant	In progress
Sprint 2 (by 2026-09-27)	`bfcl-eval` and tau2-bench-verified environments (S4)	Vritika	In progress
Sprint 2 (by 2026-09-27)	Colab and Kaggle GPU access checked (S7)	Yashashav	In progress
Sprint 2 (by 2026-09-27)	Simulated-user API key with $300 cap	Vritika	Not started
Sprint 2 (by 2026-09-27)	Access to the M4 Pro measurement machine	owner of the M4 Pro	Arranged
2026-09-27	Experiments	All	Not started

Chapter 9. Project Schedule

We plan the project in two-week sprints that run Monday to Sunday. Sprint 1 starts on 2026-08-31 and Sprint 16 ends on 2027-05-16, covering CMPE 295A (Fall 2026) and CMPE 295B (Spring 2027). The winter break (2026-12-21 to 2027-01-24) is one reduced-load sprint, W, used for unattended QAD training and as schedule buffer. Each sprint ends with a review that doubles as a checkpoint. Eight milestones (M1 to M8) carry exit criteria, and the advisor reviews them at the weekly meeting nearest each target date. Ownership follows the proposed division of work division of work as proposed, summarized at the end of this chapter. Ekant Kapgate owns the testbed core (C2, C3, C5), repair R1, and the Cost Profiler (C9). Vritika Malhotra owns the Benchmark Adapter (C1), Simulated-User Cache (C4), Failure Classifier (C6), and Analysis & Stats (C10). Yashashav Devalapalli Kamalraj owns repairs R2 and R3, the QAD Training Pipeline (C8), model conversion, and the Experiment Config & Registry (C11). The paper targets EuroMLSys 2027, whose call for papers has not been published as of 2026-09-27. EuroSys 2027 runs April 19 to 23, 2027 in Rabat, with workshops on April 19 [80]. The 2025 and 2026 EuroMLSys deadlines fell seven to nine weeks before their workshops: February 11, 2025 for a March 31 workshop [108], and February 24, 2026 for April 27 [79]. We therefore plan to submit on 2027-02-19 EuroMLSys 2027 deadline, the earliest plausible date, and to have a paper-ready draft one week earlier. If the real deadline is later, the extra time goes to the extended study.

Milestones

Table 38. Project milestones and exit criteria.

ID	Milestone	Target date	Exit criteria
M1	Walking skeleton runs end to end	2026-10-23	One command runs Qwen3-0.6B at FP16, Q4_K_M, and MLX 4-bit on 23 BFCL and tau2 tasks; every step logged and classified; one minimal run of R1, R2, R3; toy stats and cost report; reproduced on two machines
M2	Full-precision references reproduced; analysis plan frozen	2026-11-06	FP references for all models, benchmarks, and runtimes with repeats; per-task results match the official evaluators; inclusion set stored; pre-registration frozen on OSF before any main-study 4-bit run
M3	Damage map v1 (RQ1)	2026-12-04	Failure counts by type, step type, and depth with CIs for every model, 4-bit format, and benchmark; failure codebook and 300 labeled pilot steps with κ report
M4	Repairs R1 and R2 evaluated	2026-12-18	Grammar, uniform, layer-selective, and step-selective variants run on the inclusion set; paired comparisons against the 4-bit baseline and the FP reference; repository v0.1, uv lockfile, and Dockerfile tagged (end of 295A)
M5	Repair R3 (QAD) and train-longer control evaluated	2027-01-29	Both arms trained with identical data and steps; exports pass the logit check; evaluated on the real 4-bit path; RQ3 analysis covers all three repairs
M6	Cost profiling complete (RQ4)	2027-02-05	Memory, TTFT, tokens/s, extra tokens, and energy for every repair on the M4 Pro, plus R3 GPU-hours; comparison matrix v1
M7	Workshop paper submitted	2027-02-19 EuroMLSys 2027 deadline	Six-page paper on RQ1 to RQ4 submitted; repository v0.5 (R1 to R3, export checks, profiler) tagged for the artifact
M8	Open-source release v1.0 and final results	2027-05-14	Release v1.0 archived with a DOI; QAD and control checkpoints with model cards on the Hub; step-log data and damage-map report public; outside reproduction within CIs; Expo demo runs on the M4 Pro

The six-page length follows the EuroMLSys 2025 format [108]. EuroSys 2027 holds its workshops on 2027-04-19, during Sprint 15. If EuroMLSys stays co-located and the paper is accepted, that is when we would present it.

Sprint Plan

Each member has one distinct line of work in every sprint. Shared work (literature review, paper, release) is split so that each person writes or delivers a separate part.

Table 39. Sprint plan with per-member tasks (Fall 2026 to Spring 2027; S1 to S9 refer to spikes in Chapter 8).

Sprint	Dates	Goal	Ekant Kapgate	Vritika Malhotra	Yashashav Devalapalli Kamalraj
1	Aug 31–Sep 13	Scope and literature	Survey runtimes and 4-bit formats	Survey agent benchmarks, error-budget evidence	Survey repairs (mixed precision, QAD)
2	Sep 14–27	Environments	M4 setup, llama.cpp Metal build; S1	Harness environments; S4	Colab and Kaggle setup; S7
3	Sep 28–Oct 11	Spikes and interfaces	C3 backend API; S2, S3, S6	C1 loaders; C4 proxy; S5	C11 config schema; S8
4	Oct 12–25	Walking skeleton (M1)	C2 loop, C5 logger; minimal R1, C9; CI	C6 rules v0; toy stats; plan draft	R2 Q8_0; S9 switch; R3 toy export
5	Oct 26–Nov 8	FP references (M2)	Backend parity tests; batch runner	Full C1; inclusion filter; OSF pre-registration	All model conversions; KL checks
6	Nov 9–22	4-bit damage runs	R1 grammars per tool-call format	Failure codebook; 300 labeled pilot steps, κ	R2 uniform, layer-selective; QAD data
7	Nov 23–Dec 6	Damage map (M3)	R1 runs; energy protocol	Damage map v1 (RQ1)	R2 step-selective; QAD on 0.6B
8	Dec 7–20	R1 and R2 done (M4)	R1 and R2 cost runs; repository v0.1	R1, R2 analysis; RQ3 metrics	R2 runs; launch QAD and control; Dockerfile
W	Dec 21–Jan 24	Reduced: training, buffer	R2 switch-cost profiling; cost section outline	Analysis automation; results outline	QAD and control on other models; exports
9	Jan 25–Feb 7	R3 and costs (M5, M6)	RQ4 cost matrix, including R3	R3 analysis; comparison matrix	R3 eval on 4-bit path; GPU-hours
10	Feb 8–21	Paper (M7)	Paper: testbed, cost; v0.5 (R1, C9)	Paper: RQ1, RQ3 results, stats	Paper: repairs, QAD, related work; v0.5 (R2, R3)
11	Feb 22–Mar 7	Extended study	Testbed docs; Expo demo design	Step-log dataset v1; replications	QAD on larger models
12	Mar 8–21	Extended study	Costs for added models; depth sweep	Damage map v2	Combined repair (R1 with R3)
13	Mar 22–Apr 4	Camera-ready or revision	Revise testbed, cost sections	Revise results sections	Revise repair sections
14	Apr 5–18	Release preparation	Expo demo build	Damage-map report (static site)	Checkpoints and model cards on Hub
15	Apr 19–May 2	Final results, outside rerun	Support outside rerun from cache	Final statistics and figures	Support outside rerun from scratch; registry freeze
16	May 3–16	Release v1.0 (M8)	Expo demo final; release CI	Report from release data; dataset cards	Release v1.0 with DOI

Gantt Chart

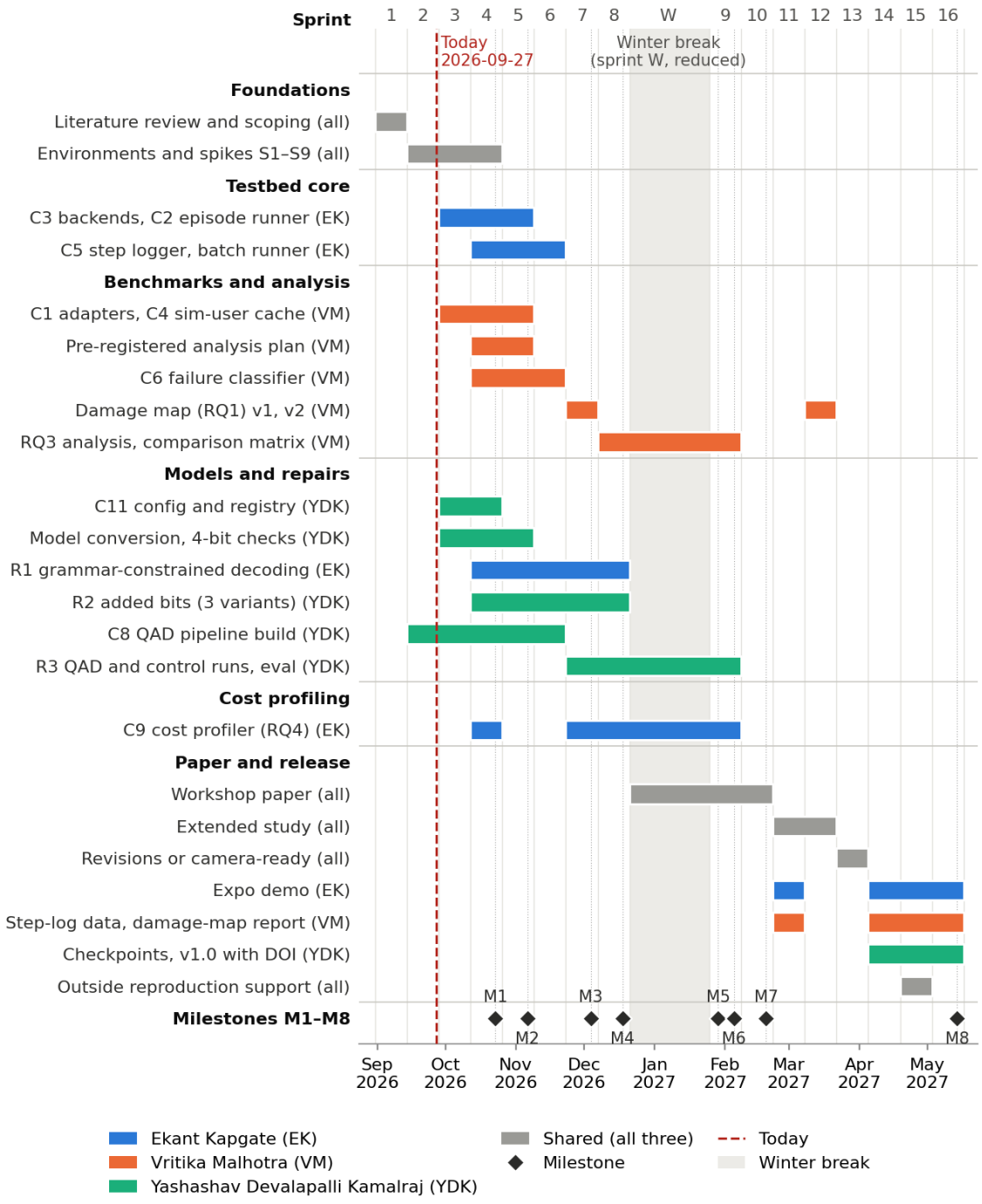


Figure 6. Project Gantt chart, Fall 2026 to Spring 2027. Bars are colored by owner and labeled with the owner's initials; diamonds mark milestones M1 to M8; the dashed line marks 2026-09-27; the shaded band is the winter break (reduced sprint W).

Task Assignment Summary

Release items follow the checklist in Chapter 7. Every component has one primary owner, who is accountable for it, and one backup, who reviews its pull requests, reruns its spike, and can take it over if needed. Each member backs up four components, and nobody backs up their own work.

Table 40. Ownership, deliverables, and backups per member.

Member	Primary ownership	Key deliverables	Backup for
Ekant Kapgate	C2 Episode Runner, C3 Inference Backends, C5 Step Logger, R1 grammar decoding, C9 Cost Profiler	Testbed core on llama.cpp and mlx-lm; repository v0.1 and CI; R1 grammars for both runtimes; RQ4 cost matrix; Expo demo; paper sections on the testbed and costs	C1, C4, R2, C8
Vritika Malhotra	C1 Benchmark Adapter, C4 Simulated-User Cache, C6 Failure Classifier, C10 Analysis & Stats	Adapters and inclusion set; budget-capped cache; failure codebook and labeled sample; OSF pre-registration; damage map (RQ1); RQ3 analysis; step-log datasets; damage-map report; paper results and statistics	C2, C5, C9, C11
Yashashav Devalapalli Kamalraj	R2 added bits, R3 with C8 QAD Training Pipeline, model conversion and 4-bit checks, C11 Config & Registry	Uniform, layer-selective, and step-selective variants; QAD and control checkpoints on the Hub; verified GGUF and MLX exports; uv lockfile, Dockerfile, run registry; release v1.0 with DOI; paper sections on repairs and related work	C3, C6, R1, C10

Schedule Risks and Buffers

- **Free GPU quota slips.** Colab's free limits are dynamic and unpublished, and Kaggle's quota is about 30 GPU-hours per week [81, 107]. Buffers: QAD starts in Sprint 7 on the smallest model; sprint W is reserved for training; runs checkpoint to the Hugging Face Hub and resume on either platform; each member can run a training job under their own account from a pinned configuration. If runs slip past M5, the paper reports R3 for the models that finished, and the rest move to the extended study (Sprints 11 and 12).
- **Simulated-user budget ($300).** Only tau2-bench-verified spends money, because BFCL multi-turn fixes its user turns in the dataset [47]. Buffers: development replays from the C4 cache for free; C4 refuses calls past the cap; we release the budget in tranches of at most $75 for the skeleton, pilots, and FP references, $150 for the main study, and a $75 reserve held until Sprint 11 for replications.
- **The field moves fast.** A paper on one of these repairs, or a comparison of them, could appear before our submission, as the abstract anticipates. Buffers: each member checks new work in their own area weekly (Ekant: runtimes and constrained decoding; Vritika: benchmarks and evaluation; Yashashav: precision

allocation and QAD), and related work is updated continuously. The controlled comparison, the controls, and the cost measurements remain our contribution, and Sprints 11 and 12 leave room to add a newly published repair.

- **Single measurement machine, finals, and an unknown deadline.** All reported numbers come from one M4 Pro, and Sprints 7 and 8 overlap Thanksgiving and final exams. Buffers: long runs are batched overnight by the batch runner, so Sprint 8 mostly finishes runs launched in Sprint 7; cost profiling gets dedicated sessions; we plan to the earliest plausible paper date (2027-02-19), and a later deadline adds slack.
- **Machine time for the repair runs.** Chapter 3 estimates the Essential grid at about 785 single-stream machine-hours, and the R1 and R2 runs between M3 and M4 alone need about 280. One M4 Pro cannot do that one episode at a time in two weeks. Buffers: the M2 pilot measures throughput with parallel `llama-server` slots against the NFR-5 target of at least 120 episodes per hour; correctness runs move to the 16 GB M4 if FR-D6 shows its outputs are token-identical; and if the pilot misses the target, the pre-registered reduction ladder in NFR-5 applies (3 trials on BFCL, then 200 BFCL tasks, then R2 on two models) before any milestone moves.
- **Built-in slack.** Each sprint is planned at about 80% of capacity. Sprint W and Sprints 11 and 12 absorb slips without moving M7 or M8.